

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 18:54:07
Файлов исходного кода: 87
Суммарный объём кода: 1.19 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts

25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts

27. src/data/tickets/ticket14_20.ts

28. src/data/tickets/ticket21_24.ts

29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx

31. src/components/BellmanFordViz.tsx

32. src/components/BfsViz.tsx

33. src/components/ChapterImage.tsx

34. src/components/ChapterNav.tsx

35. src/components/ChapterView.tsx

36. src/components/DfsBridgesSimulator.tsx

37. src/components/DijkstraViz.tsx

38. src/components/DPVisualizer.tsx

39. src/components/EulerSimulator.tsx

40. src/components/ExpressQuiz.tsx

41. src/components/FloydViz.tsx

42. src/components/GraphTraversalViz.tsx

43. src/components/HeapViz.tsx

44. src/components/hints/demoKit.tsx

45. src/components/hints/demosExtra.tsx

46. src/components/hints/demosGraph.tsx

47. src/components/hints/demosStruct.tsx

48. src/components/hints/MiniDemo.tsx

49. src/components/hints/SimulatorModal.tsx

50. src/components/hints/TermPopover.tsx

51. src/components/JohnsonViz.tsx

52. src/components/KosarajuViz.tsx

53. src/components/KruskalSimulator.tsx

54. src/components/MnemonicCards.tsx

55. src/components/MobileToc.tsx

56. src/components/Navbar.tsx

57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

## `github/workflows/deploy-pages.yml`

### Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

```
- name: Upload Pages artifact
  uses: actions/upload-pages-artifact@v5
  with:
    path: ./dist

deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+ push:
+ branches:
+ - main
+ # Позволяет проверить Pages прямо из рабочей ветки
+ - "arena/**"
+ workflow_dispatch:
+
```

## Универсальное пособие • полный исходный код

```

+
+ deploy:
+ name: Publish to GitHub Pages
+ needs: build
+ runs-on: ubuntu-latest
+ environment:
+ name: github-pages
+ url: ${ steps.deployment.outputs.page_url }
+ steps:
+ - name: Deploy
+ id: deployment
+ uses: actions/deploy-pages@v5
+ ,,,
+

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml
name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:
#
#   исходный код  →  full_code_all_pages.txt  (script)
#                   |
#                   →  page-0001.png ... page-NNNN.png  (
#                   |
#                   →  full_code_all_page
#
# Готовые TXT и PDF коммитаются обратно в ветку (их отда
# промежуточные PNG сохраняются как artifacts запуска.

on:
  push:
    branches:
      - "*"
    paths-ignore:
      # чтобы коммит самого workflow не запускал бескон
```

```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text -> p
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| \`\${public}/export/full\_code\_all\_pages`"

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from
import { chapters } from "../data/content";
```



```

 }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
);

```

```

 </footer>
 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (
    <main>

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-800" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-400 hover:text-slate-200 transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-400 hover:text-slate-200 transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

-----  
### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
-none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

<div className="border-t border-slate-800 b
  <label className="block px-4 pt-3 text-[1
    Ввод для input() • по одной строке на к
  </label>
  <textarea
    id="python-stdin"
    value={stdin}
    onChange={(event) => setStdin(event.tar
    spellCheck={false}
    rows={2}
    placeholder={'Например:\nАлиса'}
    className="block w-full resize-y bg-tra
eholder:text-slate-700"
  />
</div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"}
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
x] leading-6 text-slate-300">
    {output}
  </pre>

```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {  
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             <label className="block px-4 pt-3 text-slate-600 font-medium">Ввод для input() • по одной строке на строку</label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-transparent border border-slate-800 rounded-md p-2"
+             />
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 20px; margin: 10px 0;}}>
+         <div className="flex items-center justify-between" style={{border: 1px solid #333; border-radius: 10px; padding: 5px; margin-bottom: 10px;}}>
+             <div className="flex items-center gap-2" style={{border: 1px solid #333; border-radius: 10px; padding: 5px; flex: 1;}}>
+                 <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+             </div>
+             <span className={`inline-flex items-center gap-2`} style={{border: 1px solid #333; border-radius: 10px; padding: 5px; flex: 1;}}>
+                 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin-top: 10px;}}>
+             </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin-bottom: 10px;}}>

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: {
    "@": path.resolve(__dirname, "src"),
  },
},
server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});
```
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
```

Универсальное пособие • полный исходный код

-
2. ****Структура JSON/TS:**** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
 3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
 4. ****Не ломай верстку:**** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "version": "1.0.0",
  "description": "A reader for the ExamPrep Remix Remix.",
  "keywords": ["remix", "remix", "exam", "prep", "reader"],
  "author": "Remix Remix",
  "license": "MIT",
  "repository": {
    "type": "git",
    "url": "https://github.com/remix-remix/exam-prep-reader"
  },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.14.2",
    "remix": "^2.10.0"
  },
  "devDependencies": {
    "@remix-run/dev": "^2.10.0",
    "@types/react": "^18.2.14",
    "@types/react-dom": "^18.2.6",
    "@types/react-router-dom": "^6.14.2",
    "typescript": "^5.1.6",
    "vite": "^4.3.9"
  }
}
```

```
"devDependencies": {
  "@tailwindcss/vite": "4.1.17",
  "@types/node": "22.19.17",
  "@types/pdfkit": "^0.17.6",
  "@types/react": "19.2.7",
  "@types/react-dom": "19.2.3",
  "@vitejs/plugin-react": "5.1.1",
  "esbuild": "^0.28.2",
  "tailwindcss": "4.1.17",
  "typescript": "5.9.3",
  "vite": "7.3.2",
  "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash
npm install
npm run dev # предпросмотр на http://0.0.0.0:5174
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- * ****Содержание**** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- * ****Переходы между темами**** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- * ****Всплывающие подсказки по терминам**** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`
- * ****Реестр интерактивов**** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | – оболочка, привязка глав к визуализации |
| components/ | – интерактивные симуляторы (Действия) |
| data/ | – контент пособия (главы/билеты) |
| scripts/export/ | – пайплайн код → txt → png → pdf |
| public/export/ | – сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | – автоматизация |
| ... | |

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```
-----
> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|--------------------|-----------|----------------------|
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет | Тема | Статус |
|---------------|--------------------------------|---|
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую. |
| **7** | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии. |
| **11** | Мосты в графах | Номерной главы нет; есть блок аналогии. |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; есть блок аналогии. |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

| | | | |
|----|---|---|-------|
| 9 | 9. Графы. Топологическая сортировка | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | |
| 12 | 12. Графы. Точки сочленения | 1 | - - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | |
| 16 | 16. Графы. Кратчайший путь. Флойд | - | - |
| 17 | 17. Графы. Алгоритм Джонсона | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - - |
| 22 | 22. Z-функция | 4 | - - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - - |
| 24 | 24. Классы сложности, сведение задач | 4 | - - |
| - | Обзор: Кратчайшие пути и Графы | - | - - |
| - | Алгоритмы на карте | - | - - |
| - | Эйлер: Путь ≠ Цикл | - | - - |
| - | Мосты в графах: код + визуализация | 1 | - - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» — минимум `inli`
4. ****Решить судьбу `alg-map`**** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
```



```
-----
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algov0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {
    requestAnimationFrame(() => {
      document.getElementById("chapter-viz)?.scrollIntoView();
    });
  });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight" && next) goTo(next.id);
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar
      activeTab={activeTab}
      setActiveTab={setActiveTab}
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) => !o)}
    />

    <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
      {activeTab === "guide" ? (
        <div>
          <MobileToc

```

```

        next={next}
        index={index}
        total={chapters.length}
        onGo={goTo}
        onOpenSimulator={setModalVizId}
        onOpenCompiler={openCompilerWithViz}
      />
    </main>
  </div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={({id}) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<footer className="p-8 text-center text-slate-600" >
  © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
  setModalVizId(null);
}} />

{compilerOpen && (
  <PythonCompiler
    chapterId={activeChapter.id}
    chapterTitle={activeChapter.title}
    onOpenGuide={() => setActiveTab("guide")}
    onClose={() => setCompilerOpen(false)}
  />
)}
</div>

```

```
* <span class="term-hint" data-term-id="...">, чтобы на
* повесить всплывающую карточку (как превью ссылок в В
*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.
      if (!node.nodeValue || node.nodeValue.trim().length === 0)
        return NodeFilter.FILTER_ACCEPT;
    },
  ));

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
```

```
    try {
      annotateTerms(el);
    } catch {
      /* если браузер не умеет lookbehind – просто оста
    }
  };

  // при смене главы
  // eslint-disable-next-line react-hooks/exhaustive-dep
  useEffect(run, deps);

  // страховка: контент перерисовали, а подсказок в нём
  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
  });
}
```

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 KB

```
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
```

```

    from {
      opacity: 0;
      transform: translateY(4px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }

.term-popover {
  animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
  cursor: help;
  border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
  color: #c7d2fe;
  border-radius: 3px;
  padding: 0 1px;
  transition:
    background-color 0.15s ease,
    color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
  background: rgba(99, 102, 241, 0.22);
  color: #fff;
  outline: none;
}
@media (hover: none) {
  .term-hint {
    border-bottom-style: solid;
    background: rgba(99, 102, 241, 0.12);
  }
}

```

```
.chapter-body :where(h2) {
  font-size: 1.25rem !important;
  line-height: 1.3 !important;
}
.chapter-body :where(h3) {
  font-size: 1.05rem !important;
  line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
  padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
  font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
  font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
  font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}
```

```
    description?: string;
    category?: string;
}
```

ФАЙЛ 16/87: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

Аналогия 2: Ступени

```

    </div>
    <p class="text-slate-300 text-sm mb-6">
        нужно покрыть отрезок, мы берем две самые бо
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
    <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приори
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

    } else {
      auto [L, R] = split(t->left, x);
      t->left = R;
      return {L, t};
    }
  }</pre>
</div>
</details>
</div>
</div>
</section>`,
  },
  {
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">

    <!-- 0. Определение -->
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-lg sm:text-xl font-mono">
        поиска <span class="text-slate-500">без</span>
        : серия поворотов, которая поднимает v в кор
        <p class="text-sm text-slate-400 text-c">
        т – поворот.</p>
      </div>

```

```

<ul class="list-disc list-inside text-slate-300">
  <li><span class="text-white font-bold">Важно!</span>
    каждой вставки чинят дерево, чтобы оно <em>не</em>
  </li>
  <li><span class="text-white font-bold">Важно!</span>
    менно быть кривым. Но каждый раз, когда мы
    ровнее становится.</li>
</ul>
</div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
    Аналогия 1: стопка бумаг на столе
  </div>
  <p class="text-slate-300 text-sm">У тебя
    скиваешь его и, поработав, кладёшь <span class="text-white font-bold">Важно!</span>
    неделю самые нужные бумаги сами собой оказыва
    ет ровно это: «взял → положил наверх».</p>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
    Аналогия 2: тропинка через газон
  </div>
  <p class="text-slate-300 text-sm">Ты идёшь по дорожке, ко
    ода дорожка сама укорачивалась вдвое: чем чаще
    оплачивает все следующие. Пойдёшь один раз
  </p>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
  <h3 class="text-lg font-bold text-white mb-4">Поворот
  <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки</span> и

```



```
<p class="text-slate-300 text-sm mb-4">Дерево  
3 шага, а потом делаем Splay(10).</p>  
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-  
0 leading-relaxed">старт           после Zi  
        глубина 3 → 1           10 в корне
```

```
      40  
     /  
    30  
   /  
  20  
 /  
10
```

```
      30  
     /  \  
    10   40  
     \  
    20
```

```
      10  
     \  
    30  
   /  \  
  20   40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра  
инили дерево по пути</span>. Все узлы, мимо  
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2  
<h3 class="text-lg font-bold text-white mb-3">  
<p class="text-slate-300 text-sm mb-3">Стро  
оддерева) по всем узлам, и Zig-Zig/Zig-Zag  
ах идея такая:</p>  
<div class="grid grid-cols-1 md:grid-cols-3  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">c  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">H  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">д
```

```

-----
// один поворот: x встаёт на место своего родителя
function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                          // левый поворот (зеркало)
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                      // родитель уехал вниз
    x.parent = g;                      // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                    // ZIG: деда нет
            rotate(x);
        } else if ((g.left === p) === (p.left === x)) {
            rotate(p);                // ZIG-ZIG: сн
            rotate(x);
        } else {
            rotate(x);                // ZIG-ZAG: сн
            rotate(x);
        }
    }
    return x;                        // теперь x —
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-
  <summary class="p-4 font-bold text-slate-400
  open:border-b border-slate-700/50">
    Вопросы, которые задают на экзамене (
  </summary>
  <div class="p-5 text-sm text-slate-300 space-
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Последний
    енный или преемник). Splay делают всегда, да
    бы, и амортизация сломалась бы.</p>
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Потому что
    еворачивают бамбук в другую сторону (см. бл
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Чередовани
    вает один из них в корень, превращая дерево
    ступает только для <em>одной</em> конкретной
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Treap дер
    play не хранит вообще ничего и держит балан
    p>
    </div>
  </div>
</details>

</div>
</section>`,
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
  }

```

```

        content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-blue-300 italic m
        <p class="text-xl font-mono text-white"
    ый граф можно раскрасить 4 цветами.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4
        rder border-emerald-500 shadow-md">
          Аналогия 1: Карта мира
        </div>
        <p class="text-slate-300 text-sm mb
        . Границы не пересекаются в одной точке по-
        ы не сливались цветами, Всегда можно всего
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

        description: "Разбиение орграфа на макро-вершины. А
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        нвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            БЫХ направлениях по односторонним улицам. Как это отразится
            у на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "graph-bridges",
      title: "11. Графы. Мосты",
      type: "html",
      description: "Рёбра, удаление которых расхреначивает граф",
      category: "Графы. Продвинутые",
      content: `

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
тей > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>
        <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-indigo-400

```

```

        </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                    Аналогия: Копаем туннель под .
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    ней земли. Но если начать копать одновременно
                    в 2 РАЗА меньше работы (геометрически: площ
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "mst-kruskal",
        title: "18. Графы. Остовное дерево. Краскал",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
                <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                    <p class="text-xl font-mono text-white">
                        (проверяем через DSU) - берем в ответ.</p>
                </div>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
      id: "mst-boruvka",
      title: "20. Графы. Остовное дерево. Борувка",
      type: "html",
      description: "",
      category: "Графы. Остовные",
      content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
ается с соседом.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
          Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb-6">
изкому племени для объединения. К вечеру пл

```

```

        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300
            <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-blue-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> —
енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
          </div>
          <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведен
аче B, то B — это <b>NP-Полная</b> (сосредо
        </div>
      </div>
    </div>
  </div>
`

```

ин раз. Вершины повторять можно.</p>

```
<p><strong class="text-white">Эйлер</strong>
<div class="p-3 bg-slate-950 rounded-lg border border-rose-500 shadow-md">
  <p class="text-sm text-blue-300">
    <p><strong class="text-white">Путь:
    <p><strong class="text-white">Цикл:
  </div>
</div>
</div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла)
  <br>• <span class="text-green-400 font-weight-bold">шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font-weight-bold">финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-950 border border-green-500 shadow-md">
      Путь: Нормальная прогулка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)</strong>.
      <strong>Теплый</strong> пришёл в бар <strong>(вторая нечётная)</strong>.
      Домой вернуться не смог, потому что
    </p>
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md bg-slate-950">
    Цикл: Гамильтонов синдром (болезнь)
  </div>
  <p class="text-slate-300 text-sm mb-4">
```

```
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-4">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">tin[v]
        <p><strong class="text-white">low[v]
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font-mono text-sm">
            <p class="text-xl font-bold text-white">Мост: Единственная веревка
            <p class="text-xs text-slate-400">
          </div>
        </div>
      </div>
    </div>
```

```
<p class="text-slate-300 text-sm">
  Если из сына u и всех его потомков <strong>
  Единственная ниточка. Порвётся — граф р-связен.
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов — только веревка.
      Если верёвка (ребро v-u) оборвётся — граф р-связен.
    </p>
  </div>
```



```

int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                 // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `class="text-white">V=5, -white">10 > 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забу

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bord
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6 xt-white">9 ≤ 12</code> – проходит, но он в`

Почему? Потому что у двудольного гр

Отсюда более строгое ограничение: <

```
<code class="text-white">9 &gt; 8</
```

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg bord
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скры

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 spac
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен. $V = 5$, $E = 10$.

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) – районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи – это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
</ul>
    </div>
</div>
</section>`,
    },
];
```

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень) */
  func: string;
  /** Локальные переменные: имя → герг (усечённый). */
  locals: Record<string, string>;
}

export interface DebugResult {
  steps: DebugStep[];
  /** Текст последней необработанной ошибки (или пустая) */
  error: string;
  /** true, если уперлись в лимит шагов. */
  truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
  const srcLiteral = JSON.stringify(userSrc); // JSON-с
  return `import sys as __sys, json as __json`
```

```

    })
    OUT
  }

  /**
   * Какие переменные изменились на этом шаге относительно
   * они подсвечиваются (как amber-подсветка изменений в
   * Новые переменные тоже считаются изменёнными.
   */
  export function changedVars(
    prev: Record<string, string> | null | undefined,
    cur: Record<string, string>
  ): Record<string, boolean> {
    const out: Record<string, boolean> = {};
    for (const k of Object.keys(cur)) {
      out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
    }
    return out;
  }

  /** Базовое имя из карточки переменной: "tin[v]" → "tin"
  export const baseVarName = (name: string): string => name

  /**
   * Порядок показа в панели переменных: сначала переменные
   * страницы (те, что подсвечивает визуализация), остальные
   */
  export function orderVars(
    cur: Record<string, string>,
    syncNames: string[]
  ): [string, string][] {
    const inSync = new Set(syncNames);
    const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
    const rest = Object.entries(cur)
      .filter(([k]) => !inSync.has(k))
      .sort(([a], [b]) => a.localeCompare(b));
    return [...syncEntries, ...rest];
  }

```

```

    short:
        "Волна от источника: сначала все соседи, потом со
formal:
        "Очередь (FIFO). Кладём старт, достаём вершину —
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
},
{
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS — обход в глубину",
    short:
        "Прёшь вперёд до упора, упёрся — откатываешься на
formal:
        "Стек (или рекурсия). Из DFS растут: времена вход
complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
},
{
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
    title: "Куча (priority queue)",
    short:
        "Не отсортированный список, а «мешок, из которого
formal:
        "Полное бинарное дерево в массиве: дети узла i ле
complexity: "вставка/извлечение O(log n), «посмотре
demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO",
    title: "Стек (LIFO)",
    short: "Стопка тарелок: кладёшь сверху и берёшь све

```



```

    "Заранее считаем ответ для всех отрезков длины 1,
      м.",
    formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",
    simulator: "sparse-table",
  },
  {
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дер"],
    title: "Дерево отрезков",
    short:
      "Матрёшка из отрезков: корень отвечает за весь массив,
        токовых кусков.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O(\log n)$ ",
    demo: "segment-tree",
    simulator: "segment-trees",
  },
  {
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
      "Ребро, после удаления которого граф разваливается на
        две части.",
    formal: "Ребро  $(u, v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где  $\text{tin}[u]$  — время
        первого посещения вершины  $u$  (то есть  $\text{low}[v] > \text{tin}[u]$  означает, что
        ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
  },
  {
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точки разрыва"],
    title: "Точка сочленения",
    short:
      "Вершина-незаменимый-сотрудник: уволь её — и коллапс.",
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists \text{ребро } (u, v) \text{ такое, что } \text{low}[v] \geq \text{tin}[u]$ ",
    demo: "articulation",
  }

```

```

        ые в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полн",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов",
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  – ра",
        к путей сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты",
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до кажд",
    formal: "Косарайю: DFS с сохранением порядка выхода",
    complexity: " $O(V + E)$ ",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-фу",
        "префикс", "префикса", "суффиксом", "суффикс", "с",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начал",
        от кусок.",
    formal:

```

```

    short:
        "Самый дешёвый маршрут из A в B. «Дешёвый» — по с
    formal:
        "Дейкстра — неотрицательные веса,  $O(E \log V)$ . Фор
        ы,  $O(V^3)$ .",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "negative-cycle",
    labels: ["отрицательный цикл", "отрицательного цикл
    title: "Отрицательный цикл",
    short:
        "Круг, пройдя по которому вы становитесь «богаче»
    formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "greedy",
    labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
    formal:
        "Корректность обычно доказывается «леммой о разре
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
    title: "Эйлеров путь и цикл",
    short:

```

```

    formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Список
},
{
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс",
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: " $O(E \log V)$  с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман",
    title: "Алгоритм Форда-Беллмана",
    short:
        "Тупой, но честный:  $V-1$  раз проходим по ВСЕМ рёбра
        ",
    formal:
        "После  $k$ -й итерации гарантированно верны все крат
        ось — есть отрицательный цикл.",
    complexity: " $O(V \cdot E)$ ",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину  $k$ ",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки

```

```

        "Сортируем все рёбра по весу и берём подряд те, ч
formal:
    "Сортировка  $O(E \log E)$  +  $E$  операций union/find. C
complexity: "O(E log E)",
demo: "mst",
simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        " $G = (V, E)$ . Ребро бывает ориентированным (улица
demo: "graph",
simulator: "graph-dfs-bfs",
},
{

```

```

    labels: [
        "включений-исключений", "включения-исключения", "
        "вырезание прямоугольников",
    ],
    title: "Включения-исключения на префиксных суммах",
    short:
        "Чтобы получить сумму прямоугольника, берём больш
        рый отрезали дважды.",
    formal:
        "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P
    complexity: "запрос O(1) после O(nm) предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён
    title: "Сведение задач",
    short:
        "«Я не умею решать A, зато умею B». Если A можно
    formal:
        "A ≤p B: есть полиномиальная функция, переводящая
        NP-полную задачу к новой.",
    demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы – префиксные суммы O(1). Минимум без изменен
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{

```

```

    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
    title: "Zig-Zig – x и родитель с одной стороны",
    short:
      "Дерево вытянулось в «бамбук»: g → p → x идут в о
        м (p, x). Именно это вдвое сокращает глубину вс
    formal:
      "Если крутить наивно (два раза поднимать x), бамб
    demo: "zig-zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – прав
        p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => e.labels.map((label) => ({ label, id: e.id })))
    .sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

```
import { Chapter } from "../types";
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
  <p class="text-xl font-mono text-white">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
      Аналогия 1: Позитивное планирование
    </div>
    <p class="text-slate-300 text-sm mb-4">
(нельзя поехать и заработать). Он всегда выбирает
    <div id="slot-chapter-image-dijkstra">
    </div>
```

```
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
      Ограничения
    </div>
    <p class="text-slate-300 text-sm mb-4">
охождение улицы), Дейкстра сломается, так как
    </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg mb-4">
  <summary class="p-4 font-bold text-slate-300">
- b border-slate-700/50">
    Строгое доказательство (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>Алгоритм использует приоритетную очередь (min
g V). Гарантирует корректность только для неориентированной
  </div>
```



```

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px; margin: 10px 0;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
                Отрицательный цикл
            </div>
            <p class="text-slate-300 text-sm mb-0">
                г станет ЕЩЕ короче – значит мы попали во в
            </p>
        </div>

        <div id="slot-bellman-viz" class="mt-8"></div>
    </div>
</section>`
    },
    {
        id: "floyd",
        title: "Билет 16. Флойд-Уоршелл",
        type: "html",
        category: "Графы",
        content: `<section id="floyd-content" class="mb-12">
            <div class="flex items-center mb-6">
                <span class="bg-blue-600 text-white px-4 py-1 rounded">
                    <h2 class="text-3xl font-bold text-white">Флойд
            </div>

            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border">
                    <h3 class="text-xl font-bold text-indigo-400">

                    <div class="bg-slate-900 p-4 rounded-lg mb-4">
                        <p class="text-lg text-indigo-300 italic">
                            <p class="text-xl font-mono text-white">
                                </p>
                        </div>

                        <div class="grid grid-cols-1 xl:grid-cols-2">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">0c
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
    </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

  <div class="bg-slate-900 p-6 rounded-lg border-2
    <div class="absolute -top-3 left-4 bg-rose-900
      -500 shadow-md">
      Аналогия 2: Матёшка (Терминальные)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      нашли и "he", потому что оно спрятано внутри
      минальные ссылки</b> (term_link) – прямые мос
      p>
    </div>
  </div>

</div>

<details class="bg-slate-900/40 rounded-lg border b

```

```

        nodes[child].term_link = nodes[nodes[child].term_link];
    }

    q.push(child);
}
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `

<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">
        <p class="text-slate-300 text-sm mb-4">Когда
        ут быть большими дробными числами (долгота
        ву от 0 до N, и уже по ним строим графы.</p>
    </div>
    </div>
</section>`
    }
};


```

```

    "Всё ещё можно пройти по всем рёбрам ровно раз",
    "Такого графа не существует",
    "Эйлера пути нет – нужно удалять лишние рёбра",
  ],
  correctIndex: 2,
  explanation: "Если нечётных вершин больше 2, эйлеров
    ровно 2 нечётные или 0."
}
],
"bridges-code": [
  {
    question: "После DFS у нас  $low[u] = 4$ , а  $tin[v] = 2$ ",
    options: [
      "Да, так как  $low[u] (4) > tin[v] (2)$ ",
      "Нет,  $low[u]$  должно быть меньше или равно",
      "Только если граф связный и неориентированный"
    ],
    correctIndex: 0,
    explanation: "Условие моста:  $low[u] > tin[v]$ . В на
  },
  {
    question: "Для чего нужен массив  $low[v]$ ?",
    options: [
      "Для того чтобы было больше переменных в коде",
      "Чтобы знать минимальный  $tin$  предка, до которого",
      "Чтобы хранить глубину рекурсии"
    ],
    correctIndex: 1,
    explanation: " $low[v]$  хранит минимальное время входа
      поиска мостов и точек сочленения."
  },
  {
    question: "Какова временная сложность алгоритма поиска мостов",
    options: [
      " $O(V^2)$  – ищем мосты в квадрате",
      " $O(V + E)$  – каждый элемент один раз",
      " $O(2^V)$  – полный перебор"
    ],
  },

```

ФАЙЛ 24/87: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 113 | РАЗМ

```
import { createContext, useContext, useEffect } from "react";
```

```
/**
```

```
 * Связь «компилятор ↔ открытая визуализация».
```

```
 *
```

```
 * Панель компилятора в режиме пошагового отладчика транслирует
```

```
 * текущего шага трассы вместе с id текущей главы; пошаговые шаги
```

```
 * главы (через useVizStepSync) подхватывают его и прыгают к нужной
```

```
 * визуализации идёт в ногу с листингом (каждая исполняемая строка
```

```
 * = очередной шаг подсветки, см. vizSync.ts).
```

```
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-step";
```

```
export interface VizStepEventDetail {
```

```
  chapterId: string;
```

```
  step: number;
```

```
}
```

```
/** Транслировать текущий шаг отладчика визуализациям трансляции
```

```
export function emitVizStep(chapterId: string, step: number) {
```

```
  if (typeof window === "undefined") return;
```

```
  window.dispatchEvent(new window.CustomEvent<VizStepEventDetail>("viz-step", {
```

```
  }));
```

```
/**
```

```
 * Подписка визуализации на шаги отладчика.
```

```
 * Вызывается внутри компонента демонстрации:
```

```
 * useVizStepSync(currentStep, setCurrentStep, steps.length);
```

```

}

/** Сколько шаговых («помеченных») строк успело выполни
export function vizHitsAtTrace(
  steps: ReadonlyArray<{ line: number }>,
  idx: number,
  srcLines: readonly string[],
  markedContents: ReadonlySet<string>
): number {
  let hits = 0;
  for (let t = 0; t <= idx && t < steps.length; t++) {
    const content = srcLines[steps[t].line - 1];
    if (content !== undefined && markedContents.has(con
  }
  return hits;
}

/* — Выбор демонстрации внутри страницы (вкладки ID /
const DEMO_CHANNEL = "algo:viz-demo";

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас откр
export function emitVizDemo(chapterId: string, demoId:
  if (typeof window === "undefined") return;
  window.dispatchEvent(new window.CustomEvent<VizDemoEv
}

/** Подписка: панель компилятора следует за активной вк
export function onVizDemo(chapterId: string, cb: (demoId
  if (typeof window === "undefined") return () => {};
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>

```

```

variables: SyncVariable[];
/**
 * Минимальный исполняемый скелет для редактора: стро
 * шаг-в-шаг с подсветкой визуализации (без словесных
 */
code?: string;
/**
 * Номера строк ВНУТРИ code (с 1), каждое выполнение
 * шаг визуализации: по ним отладчик точно переводит
 * шаг демонстрации (иначе – грубо «строчный индекс =
 */
stepCodeLines?: number[];
/**
 * Может ли демонстрация страницы ходить за отладчиком
 * false/нет – интерактив руками (клики/наведение): ч
 */
stepDriven?: boolean;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем tr
    code: `n = 8                                # длина массива
tree = [0] * (2 * n)
i = 0                                           # лист: tree[n + i]

v, l, r = 1, 0, n - 1                          # вершина и её отрезок
m = (l + r) // 2                               # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
      { name: "n", role: "длина исходного массива; лист
      { name: "i", role: "индекс элемента массива; его
      { name: "v", role: "текущая вершина дерева отрезк
      { name: "l, r", role: "границы отрезка, за которы
      { name: "m", role: "середина отрезка m = (l + r)
    ],

```

```

-----
    stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j - 1], st[i - 1][j])$ ",
    stepDriven: true,
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
st = [[2, 0, 0, 0], [3, 0, 0, 0], [5, 0, 0, 0], [62, 0, 0, 0],
for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])
    stepCodeLines: [3, 7],
    variables: [
        { name: "n", role: "длина массива / сторона квадрата", range: "1..8" },
        { name: "k", role: "уровень таблицы: ячейка хранит сумму", range: "1..LOG" },
        { name: "i", role: "начало блока в строке уровня", range: "0..n - (1 << k) + 1" },
        { name: "j", role: "номер уровня при построении (1..LOG)", range: "1..LOG" },
        { name: "r, c", role: "строка и столбец ячейки в таблице", range: "0..n - 1" },
        { name: "kx, ky", role: "уровни по вертикали и горизонтали", range: "1..LOG" }
    ],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного  $P[i] = P[i - 1] + a[i - 1]$ ",
    stepDriven: true,
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`
    stepCodeLines: [3, 5],
    variables: [
        { name: "n", role: "число строк матрицы", range: "1..8" },
        { name: "m", role: "число столбцов матрицы", range: "1..8" },
        { name: "i", role: "индекс текущей строки (таблицы)", range: "0..n - 1" },
        { name: "j", role: "индекс текущего столбца (столбца)", range: "0..m - 1" },
        { name: "S[i][j]", role: "сумма прямоугольника (1..n) x (1..m) на каждом шаге" },
    ],
},

```



```

        variables: [
            { name: "n", role: "текущее число элементов в куче"},
            { name: "i", role: "индекс элемента, который сейчас обрабатываем"},
            { name: "(i-1)//2", role: "индекс родителя вершины i"},
            { name: "2·i+1, 2·i+2", role: "индексы левого и правого ребёнка"},
        ],
    },
    "stack-dfs": {
        vizTitle: "Стек и обход в глубину",
        stepNote: "Шаг = push новой вершины на стек или pop",
        code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень выше
print("pop →", top, "стек:", st)` ,
        variables: [
            { name: "top", role: "вершина стека — то, откуда мы вышли"},
            { name: "v", role: "текущая вершина графа, которую мы обрабатываем"},
        ],
    },
    "queue-bfs": {
        vizTitle: "Очередь и обход в ширину",
        stepDriven: true,
        stepNote: "Шаг = dequeue вершины из головы очереди",
        code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
        variables: [
            { name: "front", role: "голова очереди — вершина, которую мы обрабатываем"},
            { name: "v", role: "вершина, извлечённая из очереди"},
            { name: "u", role: "сосед вершины v, которого кладем в очередь"},
        ],
    },
    "everyday-basics": {
        vizTitle: "Бытовой тренажёр: стек, очередь, куча",

```

```

"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepDriven: true,
    stepNote: "Шаг = выход рекурсии из вершины v: она д
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)           # выход из v!

for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])           # разворот – ответ`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "вершина, куда ведёт ребро из
        { name: "i", role: "перебор стартовых вершин во в
        { name: "order", role: "список выхода из рекурсии
    ],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)"
    stepDriven: true,
    stepNote: "Шаг = вершина из order (в обратном поряд
    code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {v: [] for v in adj}
for v in adj:
    for to in adj[v]:
        adjT[to].append(v)    # транспонирование

order = [3, 2, 1, 0, 4]      # фаза 1 уже дала порядок в
comp = {}                    # фаза 2: вершина → № SCC

```

```

v, to = "B", "G"                # шаг: возврат из G
is_bridge = low[to] > tin[v]      # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "сосед; ребро (v, to) проверя
        { name: "tin[v]", role: "время входа в вершину –
        { name: "low[v]", role: "минимум tin, куда можно
        { name: "timer", role: "глобальный счётчик времени
    ],
},
"euler-path-vs-cycle": {
    vizTitle: "Эйлеров путь и цикл",
    stepNote: "Шаг = один клик по следующей вершине мар
    code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges:              # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
# 0 → цикл • 2 → путь • иначе – нельзя`,
    variables: [
        { name: "path", role: "текущий маршрут – последов
        { name: "last", role: "последняя вершина пути – о
        { name: "deg(v)", role: "степень вершины: чётная
    ],
},
"planarity-euler-formula": {
    vizTitle: "Планарность: K5 и K3,3",
    stepNote: "Шаг = попытка перетащить вершину так, что
    code: `V, E = 5, 10          # K5      (K3,3: V=
F = 2 - V + E                    # формула: V - E + F = 2

not_planar = E > 3 * V - 6 # 10 > 9 → K5 непланарен
print(f"V={V} E={E} F={F} непланарен={not planar}")`,

```

```

        { name: "dist[v]", role: "лучшее известное расстояние"},
    ],
},
"bellman-ford": {
    vizTitle: "Форд–Беллман",
    stepDriven: true,
    stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
    code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
    u, v, w = "S", "A", 5 # шаг: очередное ребро
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")
        break`,
    variables: [
        { name: "n", role: "число вершин графа – равно n"},
        { name: "m", role: "число рёбер, по которым проходят"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
        { name: "dist[v]", role: "расстояние; если обновлено, то новое"},
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepDriven: true,
    stepNote: "Шаг = инициализация матрицы, смена промежуточного узла",
    code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
      [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0],
      [INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):
    print(f"посредник k={k}")
    for i in range(n):
        for j in range(n):
            if d[i][k] < INF and d[k][j] < INF:
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    print(d)
    print()
print(d)
print()
print("Готово!")
    `,
    variables: [
        { name: "n", role: "число вершин графа – равно n"},
        { name: "m", role: "число рёбер, по которым проходят"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
        { name: "dist[v]", role: "расстояние; если обновлено, то новое"},
    ],
},

```

```

parent = {v: v for v in "ABC"}    # DSU
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

mst = []
for w, u, v in edges:             # шаг: очередное ребро
    if find(u) != find(v):        # разные компоненты → берём
        parent[find(u)] = find(v)
        mst.append((w, u, v))
print(mst)\,
    variables: [
        { name: "n", role: "число вершин; в остов войдёт n",
        { name: "m", role: "число рёбер, сортируем по весу",
        { name: "i", role: "ребро сортируем по весу w; ра",
        { name: "parent[v]", role: "DSU: кто представитель",
        { name: "rank[v]", role: "высота дерева DSU – по k",
    ],
},
"mst-prima": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

start = "A"
heap = [(0, start, "start")]      # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start}                  # дерево растёт от start
print(f"берём {v} за {w} (из {parent})")\,
    variables: [
        { name: "v", role: "вершина, которую только что д",
        { name: "u", role: "ещё не взятый сосед v – канди",
        { name: "w", role: "вес ребра (v, u) – ключ в куче",
        { name: "start", role: "стартовая вершина, из неё",
    ],

```

```

        ица" },
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepDriven: true,
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
    code: `s = "abacaba"
n = len(s)
z = [0] * n
l = r = 0                # правый Z-блок [l, r]

for i in range(1, n):    # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l])    # из блока
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1                        # досчёт в лоб
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
print(z)`,
    variables: [
        { name: "n", role: "длина строки s", range: "n = "},
        { name: "i", role: "позиция, для которой считаем z[i]"},
        { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]"},
        { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[l:r+1]"},
    ],
},
"aho-corasick": {
    vizTitle: "Ахо-Корасик",
    stepNote: "Шаг = построение бора по образцам, затем поиск в боре",
    code: `trie = {}                # бор: вершина → словарь
state, c = "", "a"                # шаг: автомат идёт по символу
nxt = trie.get(state, {}).get(c, "")    # go(state, c) с бора
print(f"δ({state!r}, {c!r}) → {nxt!r}")`,
    variables: [
        { name: "state", role: "текущее состояние автомата"},
        { name: "c", role: "очередной символ текста – перемещает"},
    ],
},

```

```

        { name: "k", role: "уровень: блоки  $2^k \times 2^k$  (шаг
        { name: "st2[{r,c,k,k}]", role: "минимум квадрати
    },
},
"sparse-table#2d-query": {
    vizTitle: "Разреженная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (
        { name: "S[i][j]", role: "сумма прямоугольника (0
    },
    { name: "r1, c1, r2, c2", role: "углы запроса; от

```

```
-----
    const t = line.trim();
    if (/^(for|while|def|class|if\s|elif\s|else|try|except|
    keep.push(line);
  }
  // убрать пустые строки с конца
  while (keep.length > 0 && keep[keep.length - 1].trim() === '') {
    keep.pop();
  }
  return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, demoId: string) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n# синхронизация-«глазом»\n`
    : `# «${chapterTitle}»\n# демо на странице нет — синхронизация отсутствует\n`;

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/87: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМЕР: 1.2 КБ

```
import { Chapter } from '../../../types';
```



```

        </summary>
        <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; border-radius: 10px; padding: 10px; margin-top: 10px;">
            <pre class="bg-slate-950 p-4 rounded" style="margin: 0; font-family: monospace; font-size: 0.9em;">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
            </div>
        </details>
    </div>
</div>
</section>`
},
{
    id: "treap",
    title: "3. Декартово дерево (Treap)",
    type: "html",
    description: "Дерево поиска по ключу и Куча по приоритету",
    category: "Продвинутые структуры",
    content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">3</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Декартово дерево (Treap)</h2>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-blue-400">Описание</h3>
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">Сочетание бинарного дерева поиска и кучи.</p>
                <p class="text-xl font-mono text-white">Операции: Insert, Delete, Find, Range Query, Split, Merge.</p>
            </div>
            <p>Метода: Split и Merge.</p>
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                    Аналогия 1: Удар катаной (Sp)
                </div>
                <p class="text-slate-300 text-sm mb-4">

```

```

id: "splay-tree",
title: "4. Splay-дерево",
type: "html",
description: "Дерево поиска, которое самобалансируется",
category: "Продвинутые структуры",
content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизи
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты ч
            ка почти не требуется времени!</p>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
              Аналогия 2: Балансировка чере
            </div>
            <p class="text-slate-300 text-sm mb-4">
              и становиться кривым как бамбук. Но как тол
            </p>
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```

        ащая дерево в вытянутый бамбук для другого.
        . Постоянное свинг-переключение превратит п
        </div>

        <div>
            <strong class="text-white block">
                <p>Хотя один поиск может стоить
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.</p>
            </div>
        </div>
    </div>
</div>
</section>`
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                    <p class="text-xl font-mono text-white">
                        ,  $y1...y2$ ) =  $P[x2][y2]$  -  $P[x1-1][y2]$  -  $P[x2$ 
                </div>
                <div class="grid grid-cols-1 xl:grid-cols-2">

```

```

<h3 class="text-xl font-bold text-emerald-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
  <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Позитивное планирование
    </div>
    <p class="text-slate-300 text-sm mb-4">
      (нельзя поехать и заработать). Он всегда в
    </div>
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Ограничения
    </div>
    <p class="text-slate-300 text-sm mb-4">
      дный и не умеет пересматривать уже "зафиксир
    </div>
  </div>
  <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
  },
  {
    id: "bellman-ford",
    title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="bellman-ford-content" class="mb-12 scroll-my-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="bg-slate-800 p-6 rounded-lg border">
        <div class="absolute -top-3 left-4 bg-slate-700/50
          border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Главный герой – внешний цикл
          шаг class="bg-slate-900 text-pink-400"
          шу себе проходить через вершину k?»</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
          <li><b>Шаг k=0:</b> Ты знаешь только
          <li><b>Шаг k=1:</b> Разрешаем пересадки
          e> через путь class="text-indigo-300"
          <li><b>Шаг k=2:</b> Разрешаем пересадки
          внутри этих путей уже могут быть пересадки ч
          <li>...</li>
          <li><b>Шаг k=N:</b> Ты проверил все
        </li>
      </ul>
    </div>
  </div>
</div>
</section>`

```

```

        </div>
      </div>
      <div id="slot-johnson-viz" class="mt-8"></div>
    </div>
  </div>
</section>`
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">18</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Остовное дерево</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Остовное дерево</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф, который является деревом и содержит все вершины графа.</p>
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет-кабелей.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Мы строим сеть дорог, начиная
            с самых дешевых дорог в стране". Он строит
            единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царств
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`
  }
};

```



```

</code>, а это начало нашего слова! Не надо
</p>
</div>

```

```

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg" style="background-color: #1a2b3c; color: white;">
  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md" style="border: 2px solid #e91e63; box-shadow: 0 4px 8px #e91e63;">
    Аналогия 2: LLM стриминг
  </div>
  <p class="text-slate-300 text-sm mb-0" style="color: #a6b8c8; font-size: 0.875em; margin-bottom: 0;">
    Для LLM, которая стримит токены
    каждый шаг. Если мы частично совпали с запятой,
    какого места этого слова мы можем продолжить?
  </p>
</div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg" style="background-color: #1a2b3c; opacity: 0.4; border-radius: 0.5em;">
  <summary class="p-4 font-bold text-slate-300" style="padding: 0.5em 0.5em 0.5em 0.5em; color: #a6b8c8; font-weight: bold;">
    -b border-slate-700/50">
    Пример вычисления (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300" style="padding: 0.5em 0.5em 0.5em 0.5em; color: #a6b8c8; font-size: 0.875em;">
    <p>Тот же пример: <b>abcaabcd</b></p>
    <ul class="list-disc pl-5 space-y-2" style="list-style-type: none; padding-left: 1.5em;">
      <li><b>a</b>: Префиксов нет. <code><b>a</b></code></li>
      <li><b>ab</b>: Из начала ("a") не совпало. <code><b>ab</b></code></li>
      <li><b>abc</b>: Опять мимо. <code><b>abc</b></code></li>
      <li><b>abca</b>: 0! Начало ("a") совпало. <code><b>abca</b></code></li>
      <li><b>abcab</b>: Начало ("ab") совпало. <code><b>abcab</b></code></li>
      <li><b>abcabc</b>: Начало ("abc") совпало. <code><b>abcabc</b></code></li>
      <li><b>abcaabcd</b>: "d" всё испортило. <code><b>abcaabcd</b></code></li>
    </ul>
  </div>

```

```
    <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
са 0) и её суффиксом, начинающимся с i.</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Линейка-шаблон
    </div>
    <p class="text-slate-300 text-sm mb
      Тут мы берём всю строку и «прик
я начну читать строку отсюда, насколько длин
      Это самый быстрый способ найти
>, считаешь Z-функцию, и там, где <code>Z[i
    </p>
  </div>
```

```
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Аналогия 2: LLM Самокопипаст
    </div>
    <p class="text-slate-300 text-sm mb
      В LLM Z-функция может применять
[i]</code> (где <i>i</i> указывает назад на
сама себя.
    </p>
  </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
    category: "Строки",
    content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">

      <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
        <p class="text-sm text-blue-300 italic mb-2">Осн
        <p class="text-lg font-mono text-white">Бор (Тр
      </div>
      <p class="text-slate-300 text-sm">Позволяет найти
        го один проход.</p>
    </div>

    <!-- Аналогии -->
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg border bo">
        <div class="absolute -top-3 left-4 bg-slate-700
          emerald-500 shadow-md">
          Аналогия 1: Паутина (Бор)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
          ет — мы падаем по <b>суффиксной ссылке</b> ("
        </div>

      <div class="bg-slate-900 p-6 rounded-lg border-2">
        <div class="absolute -top-3 left-4 bg-rose-900
          -500 shadow-md">
          Аналогия 2: Матёшка (Терминальные)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
          нашли и "he", потому что оно спрятано внутри
```

```

    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-purple-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-purple-300 italic">
      <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Судoku (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
      </div>
      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
          Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведенн
аче B, то B – это <b>NP-Полная</b> (сосредо
      </div>
    </div>
  </div>
</div>
</section>`
  }
];

```

```
        глупо, но проверка "знает ли Вася Петю" мгр
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
            Аналогия: Списки = Контакты
        </div>
        <p class="text-slate-300 text-sm mb-1">
т. Оптимально для почти всех задач.</p>
    </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border-1
    <h3 class="text-xl font-bold text-indigo-400">

    <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия DFS -->
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
rder border-indigo-500 shadow-md">
                ♂ DFS (Поиск в глубину) = Жадный
            </div>
            <p class="text-slate-300 text-sm mb-1">
                <b>Что делает:</b> Жадный алгоритм находит ближайшую
ую вершину (например, минимальную по номеру) и пробует другой путь.
            </p>
            <ul class="text-xs text-indigo-300">
                <li><b>Где используется:</b></li>
                <li>Топологическая сортировка (линейное время)</li>
                <li>Поиск мостов и точек сочленения</li>
                <li>Сильно связанные компоненты Киркпатрика</li>
                <li>Поиск циклов и просто проверка</li>
            </ul>
        </div>
    </div>
```

```
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

</details>

</section>`

4.

f

```
id: "graph-planar-colors",
```

title: "7. Графы. Планарные. Покраска",

```
type: "html",
```

description: "Формула Эйлера и теорема о 4 красках."

category: "Графы. Теория",

```
content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m
```

```
<div class="flex items-center mb-6 flex-wrap gap-3"
```

```
<span class="bg-blue-600 text-white px-4 py-1 rounded" style="float: right; margin-top: 10px;">
  100%

```

class="text-2xl sm:text-3xl font-bold text-

<div class="space-y-8">

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
```

class="text-xl font-bold text-blue-400"

<div class="bg-slate-900 p-4 rounded-lg mb-

text-blue-300 text-lg

```
<p class="text-xl font-mono text-white"
```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
  <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
    Аналогия 1: Острова
  </div>
  <p class="text-slate-300 text-sm mb-4">
    карту – остались ли серые (неисследованные)
    ь через океан – столько и компонент.</p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-blue-300 italic">
          <p class="text-xl font-mono text-white">
            перед.</p>
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">

```

```
void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

    </div>
    </div>
    </details>
    </div>
    </div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жару",
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-6">
            <p class="text-lg text-emerald-300 italic">Макро-вершиной называют компоненту сильной связности
            <div class="text-left font-mono text-sm">
                <p><strong class="text-white">Компонента сильной связности (SCC) — это максимальный подграф ориентированного графа, в котором каждая вершина достижима из каждой другой вершины.
            </div>
        </div>
    </div>
`
    }
}

```



```

<details class="bg-slate-900/40 rounded-lg mb-6"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">
      Душный мод: Код Алгоритм Косарайю
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <p class="text-emerald-400 font-bold">
        <ol class="list-decimal list-inside"
          <li>Запускаем DFS на исходном графе
        </li>
          <li>Разворачиваем этот список (то есть делаем reverse)
          <li>Переворачиваем все ребра в графе
          <li>Идем по <code>order</code>:
        </li>
      </ol>
    </div>
  </details>
</div>
</div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых расхреначивает граф",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6"
        <p class="text-lg text-rose-300 italic">

```

```

    рои увеличивает число компонент связности.<
    <div class="p-3 bg-slate-950 rounded
      <span class="text-rose-400 font
      <p class="text-xl font-bold tex
      <p class="text-xs text-slate-40
    </div>
  </div>
</div>

```

```

<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong
  ег - это тупик со степенью 1).
</p>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap
  <div class="bg-slate-800 p-6 rounded-lg bor
    <div class="absolute -top-3 left-4 bg-s
      rder-rose-500 shadow-md">
        Аналогия: Главный перекресток
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь город, где два района сое
        её перекроют (удалят вершину) — районы буд
        g>хрупкость</strong> системы.
      </p>
    </div>

```

```

<div class="bg-slate-900 p-6 rounded-lg bor
  <div class="absolute -top-3 left-4 bg-em
    er border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одно
      абелем (мостом). Сами свитчи — это точки со
    </p>

```

```

        </div>
    </details>

    <div class="bg-indigo-950/60 p-6 rounded-xl border" >
        <h4 class="text-lg font-bold text-indigo-400" >
        <p class="text-slate-300 text-sm mb-4">
            Это глубокий дуальный мост между ориентир
        </p>
        <ul class="list-disc list-inside text-sm text" >
            <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
            <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
            <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400" >
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border" >
            <h3 class="text-xl font-bold text-indigo-400" >

```

```

    y: number;
    label: string;
}

interface Edge {
    u: number;
    v: number;
}

const nodes: Node[] = [
    { id: 0, x: 250, y: 50, label: "0" },
    { id: 1, x: 100, y: 150, label: "1" },
    { id: 2, x: 250, y: 250, label: "2" },
    { id: 3, x: 400, y: 150, label: "3" },
    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
    { length: Object.keys(nodes).length },
    () => [],
);

edges.forEach((e) => {
    adj[e.u].push(e.v);
    adj[e.v].push(e.u);
});

```

```

const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], currentTin[to]);
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${v}]`
      );
    } else {
      children++;
      dfs(to, v);
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);

      recordStep(
        `Возврат в ${v} из ${to}. Обновляем low[${v}]`
      );

      if (currentLow[to] >= currentTin[v] && p !== -1) {
        ap.add(v);
        recordStep(
          `Найдена точка сочленения: вершина ${v}`
        );
      }
    }
  }
}

if (p === -1 && children > 1) {
  ap.add(v);
}

```

```

        setIsPlaying(false);
    }
    return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
};

return (
    <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
            <h3 className="font-bold text-white flex items-
                <Info className="w-5 h-5 text-rose-400" />
                Моделирование поиска точек сочленения
            </h3>

        <div className="flex items-center gap-2 bg-slate-
            <button
                onClick={togglePlay}
                className="flex items-center gap-2 px-3 py-
                text-white"
            >
                {isPlaying ? (
                    <Pause className="w-4 h-4 ml-1" />
                ) : (
                    <Play className="w-4 h-4 ml-1" />
                )}
                {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button
                onClick={reset}
                className="flex items-center justify-center
                ors"
            >

```

```

        ? "#3b82f6"
        : isAp
        ? "#e11d48"
        : isVisited
        ? "#10b981"
        : "#1e293b";
const stroke = isCurrent
  ? "#60a5fa"
  : isAp
  ? "#f43f5e"
  : isVisited
  ? "#34d399"
  : "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-300"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>
  </g>
)

```

</h4>

```
<div className="bg-slate-900 border border-slate-800" >
  <p className="text-white font-medium min-h-[100px]">
    {currentStep.desc}
  </p>
</div>
```

```
<div className="space-y-4 flex-1 overflow-y-auto" >
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 font-medium">Шаг 1</h5>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-slate-400">
        Точка сочленения
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-slate-400">
          Посещена
        </div>
        <div className="flex items-center gap-2">
          <div className="w-3 h-3 rounded-full bg-slate-400">
            Текущая
          </div>
          <div className="flex items-center gap-2">
            <div className="w-8 h-1 bg-rose-500 border border-rose-400">
              Мост
            </div>
          </div>
        </div>
      </div>
    </div>
```

```
<div>
  <h5 className="text-xs text-slate-500 font-medium">ЛОГ ДЕЙСТВИЙ:</h5>
  <div className="space-y-2">
    {steps
      .slice(0, currentStepIndex + 1)
      .reverse()
      .map((step, idx) => {
```



```

interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  useVizStepSync(currentStepIndex, setCurrentStepIndex, setSteps);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'A', v: 'D', w: -5 }];
    }
    return defaultEdges;
  };

```

```

    log: '    Фура заведена. Начинаем с Базы (S). Расст
    activeCodeLine: 2,
  });

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

```

```

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v
      distances: { ...dist }, previous: { ...prev }
      log: `Итерация ${iter}: Проверяем ${edge.u} →
        updated ? ` Релаксация успешна! dist[${edge
      }`,
      activeCodeLine: updated ? 6 : 5,
    });
  }

```

```

if (!anyUpdate && iter < vCount - 1) {
  simulationSteps.push({
    iteration: iter, checkingEdge: null, distance
    log: `    Ранний выход после итерации ${iter}.
    activeCodeLine: 10
  });
  break;
}
}

```



```

        return (
          <g key={idx}>
            <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={
              asNegativeCycle && isCyclePart} ? 4 : 2} strokeWidth={
              g ? '4,4' : 'none'} />
            <circle cx={({uNode.x + vNode.x} / 2} cy={({uNode.y + vNode.y} / 2}) r={
              Checking ? '#f59e0b' : isNeg ? '#f43f5e' : '#f8a48f'} />
            <text x={({uNode.x + vNode.x} / 2} y={({uNode.y + vNode.y} / 2}) text={
              8fafc'} fontSize="11" fontWeight="bold" textAnchor="middle" />
          </g>
        );
      }
    }
  }

  {nodes.map((node) => {
    const dist = currentStep.distances[node.id];
    const isCheckingNode = currentStep.checkingNode === node.id;

    return (
      <g key={node.id} className="transition-all duration-300">
        <circle cx={node.x} cy={node.y} r={isCheckingNode ? 10 : 5} stroke={
          isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWidth={2} />
        <text x={node.x} y={node.y + 5} fill="white" fontSize="10" />
        <text x={node.x} y={node.y - 28} fill="white" fontSize="10" />
        <text x={node.x} y={node.y + 35} fill="white" fontSize="10" />
      </g>
    );
  })}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </p>
</div>
</div>

```

```

        </div>
      );
    }}}
  </div>
</div>
</div>
</div>

  { /* Код алгоритма */
  <div className="w-full lg:w-1/2 bg-slate-900/60" >
    <div>
      <h4 className="text-lg font-bold text-slate-100">
        <div className="bg-slate-950/80 rounded-lg p-4">
          {codeLines.map(item => (
            <div key={item.line} className={`py-1.5 px-4
              e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}`>
              <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
              <span className="whitespace-pre flex-grow-1">{item.code}</span>
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
</div>
</div>
  );
}

```

ФАЙЛ 32/87: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useCallback } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';

const NODES = [

```

```

    logs: "Инициализация: начинаем с корня (0)."
```

 });

 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"
 });

 while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: \${queue.length}`
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (\${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],

```
const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);
  useVizStepSync(stepIdx, setStepIdx, STEPS.length - 1)

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
    const timer = setTimeout(handleNext, 1200);
    return () => clearTimeout(timer);
  }, [isPlaying, stepIdx, handleNext]);

  return (
    <div className="bg-slate-900 border-2 border-slate-
      <div className="flex flex-col md:flex-row items-c
        <div>
          <h3 className="text-2xl font-bold text-white
            <span className="text-blue-400"> </span> Ал
          </h3>
          <p className="text-slate-400 text-sm mt-1">Виде
        </div>
```

```

        <line
          key={idx}
          x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
          className={`transition-all duration-500`}
        >`}
```

```

    </>
  );
}
}

{/* Вершины */}
{NODES.map(n => {
  const isCurrent = step.currentNode === n.id;
  const isInQueue = step.queue.includes(n.id);
  const isVisited = step.visited.includes(n.id);

  let fill = '#1e293b'; // slate-800
  let stroke = '#475569'; // slate-600
  let scale = 1;
  void scale;

  if (isCurrent) {
    fill = '#3b82f6'; // blue-500
    stroke = '#60a5fa'; // blue-400
    scale = 1.3;
  } else if (isInQueue) {
    fill = '#059669'; // emerald-600
    stroke = '#34d399'; // emerald-400
    scale = 1.1;
  } else if (isVisited) {
    fill = '#0f172a'; // slate-900
    stroke = '#3b82f6'; // blue-500
  }

  return (
    <g key={n.id} className="transition-tran
      {isCurrent && {
        <circle cx={n.x} cy={n.y} r="30" fi
      }}
    )
  );
})}

```



```

        { line: 6, code: '    for neighbor in graph.neighbors(cl)' }
        { line: 7, code: '        if neighbor not in visited:' }
        { line: 8, code: '            visited.add(neighbor)' }
        { line: 9, code: '            queue.push(neighbor)' }
    ].map((cl) => {
      const isActive = step.activeLine === cl.line
      return (
        <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5`} : 'text-slate-500'>
          <span className="opacity-40 w-6 inline-block">{cl.line}</span>
          <span className="whitespace-pre">{cl.code}</span>
        </div>
      )
    })
  </div>
</div>

<div className="bg-slate-800/80 border border-slate-700 p-4">
  <div className="text-sm font-bold text-emerald-400">
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500">{step.queue.length}</span>
  </div>
  <div className="flex items-center gap-2 mt-2">
    {step.queue.length === 0 ? (
      <span className="text-slate-500 text-sm">Очередь пуста</span>
    ) : (
      step.queue.map(({qId, i}) => (
        <span key={`_${qId}-${i}`} className="text-slate-500 text-sm">
          {qId} {i}
        </span>
      ))
    )}
  </div>
  <div className="mt-2 text-sm text-amber-300 font-mono">
    {step.logs}
  </div>
</div>

```

```
    },  
  };  
  
export default function ChapterImage({ vizType }: { vizType: string }) {  
  const info = imageMap[vizType];  
  if (!info) return null;  
  
  return (  
    <div>  
      <div className={`rounded-2xl overflow-hidden border-2 border-gray-200`} >  
        <img src={info.src} alt={info.alt} className="w-full h-full" />  
      </div>  
      <p className={`text-center text-xs mt-2 font-bold`} >{info.title}</p>  
    </div>  
  );  
}
```

ФАЙЛ 34/87: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";  
import { ChevronLeft, ChevronRight } from "lucide-react";  
import type { Chapter } from "../types";
```

```
interface Props {  
  prev?: Chapter;  
  next?: Chapter;  
  index: number;  
  total: number;  
  onGo: (id: string) => void;  
  compact?: boolean;  
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» — как на Code
```

```

        className="group text-left p-4 rounded-xl bg-
          transition-all"
      >
        <div className="flex items-center gap-1.5 tex
          400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Пре
        </div>
        <div className="text-sm font-bold text-slate-
        </button>
      ) : (
        <div className="hidden sm:block" />
      )}

    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-
          transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end
          text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3
        </div>
        <div className="text-sm font-bold text-slate-
        </button>
      )}
    </nav>
  );
};

```

ФАЙЛ 35/87: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useEffect, useRef, useStat
import { Check, Download, Loader2, Maximize2, MousePoin

```

```

    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(
  }, [cancelHide]);

  useEffect(() => {
    setAnchor(null);
    setSaving("idle");
  }, [chapter.id]);

  const saveHtml = useCallback(async () => {
    setSaving("work");
    try {
      await downloadChapterHtml(chapter);
      setSaving("done");
      window.setTimeout(() => setSaving("idle"), 2500);
    } catch {
      setSaving("idle");
    }
  }, [chapter]);

  const open = useCallback(
    (el: HTMLElement) => {
      const termId = el.dataset.termId;
      if (!termId) return;
      cancelHide();
      setAnchor({ termId, rect: el.getBoundingClientRect()
    },
    [cancelHide]
  );

  const handlePointerOver = (e: React.MouseEvent) => {
    if (isTouch()) return;
    const el = (e.target as HTMLElement).closest<HTMLElement>
    if (el) open(el);
  };

  const handlePointerOut = (e: React.MouseEvent) => {
    if (isTouch()) return;

```

```

        {saving === "done" ? "Файл сохранён" : "Скачать"}
      </button>
      <button
        onClick={() => window.print()}
        title="Распечатать или сохранить в PDF среде"
        className="flex items-center gap-1.5 text-x-sm font-medium
          300 hover:text-white hover:border-indigo-500"
      >
        <Printer className="w-3.5 h-3.5" /> Печать
      </button>
      <span className="text-[11px] text-slate-500">
    </div>

    <div
      ref={contentRef}
      className="prose prose-invert max-w-none"
      onMouseOver={handlePointerOver}
      onMouseOut={handlePointerOut}
      onClick={handleClick}
      onFocus={handleFocus}
      dangerouslySetInnerHTML={{ __html: chapter.content }}
    />

    <p className="mt-6 flex items-start gap-2 text-sm font-medium
      ed-lg px-3 py-2">
      <MousePointerClick className="w-4 h-4 shrink-0" />
      <span>
        Подчёркнутые термины – интерактивные: наведение
        объяснение на пальцах, мини-анимацию и кнопки
      </span>
    </p>

    {viz && (
      <section id="chapter-viz" className="mt-8 scroll-mt-16">
        <div className="flex items-center justify-between">
          <h3 className="flex items-center gap-2 text-sm font-medium">
            <Sparkles className="w-4 h-4 text-indigo-500" />
            Демонстрация: {viz.title}
          </h3>
        </div>
      </section>
    )}
  </div>
</div>

```

```

        }}
        onCardEnter={cancelHide}
        onCardLeave={scheduleHide}
      />
    </>
  );
};

```

ФАЙЛ 36/87: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Undo, Play, Pause, SkipForward, RefreshCw } from 'react-

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },

```

```

    { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 1, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: null, backEdge: null,
    log: "У G нет соседей (кроме B). Возврат. low[G]=3.",
    activeCodeLine: [13,14,15]
  },
  {
    currentNode: 2, visitedIndices: [0,1,6,2],
    tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
    activeEdge: [1,2], backEdge: null,
    log: "Из B идём в C. tin[C]=4, low[C]=4.",
    activeCodeLine: [4,5,7,10,11]
  }
];

```

```

tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост,
activeCodeLine: [13,14,15,16,17]
},
{
currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм
activeCodeLine: [11,13,16,17,18]
},
{
currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм

```



```

        {dfsAuto ? <><Pause className="h-3 w-3" />
        </button>
        <button onClick={() => { setDfsAuto(false); s
          disabled={dfsIdx === DFS_STEPS.length-1}
          className="p-1.5 hover:bg-slate-700 rounded
          <SkipForward className="h-3.5 w-3.5" />
        </button>
        <button onClick={() => { setDfsAuto(false); s
          className="p-1.5 hover:bg-slate-700 rounded
          <RefreshCw className="h-3.5 w-3.5" />
        </button>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-5 g
      {/* Graph visualization */}
      <div className="md:col-span-3 bg-slate-950 roun
        <div className="absolute inset-0 bg-[radial-g
          >
        <svg className="w-full h-[280px] z-10 relative
          {GRAPH_EDGES.map(e => {
            const u = GRAPH_NODES.find(n => n.id ===
            const v = GRAPH_NODES.find(n => n.id ===
            const bridge = step.bridges.includes(e.key
            const activeTree = step.activeEdge && (
              (step.activeEdge[0]===e.u && step.active
              (step.activeEdge[0]===e.v && step.active
            ));
            const activeBack = step.backEdge && (
              (step.backEdge[0]===e.u && step.backEdge
              (step.backEdge[0]===e.v && step.backEdge
            ));
            let sc = "#475569", sw = 2, dash = "none"
            if (bridge) { sc = "#eab308"; sw = 4; }
            else if (activeTree) { sc = "#10b981"; sw
            else if (activeBack) { sc = "#f43f5e"; sw
            else if (step.visitedIndices.includes(e.u
            return <line key={e.key} x1={u.x} y1={u.y

```

```

    </div>
  </div>

  {/** Code panel + stack */}
  <div className="md:col-span-2 space-y-3">
    {/** Code panel */}
    <div className="bg-slate-950 rounded-lg border" style={{borderColor: 'black'}}>
      <div className="bg-slate-900 px-3 py-1.5 text-slate-300">
        >
        <div className="p-2 overflow-x-auto">
          <pre className="text-[10px] font-mono leading-relaxed">
            {CODE_LINES.map(({cl, i}) => {
              const hl = step.activeCodeLine.includes(cl)
              return <div key={i}
                className={`px-1 transition-all duration-200
                  ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-300"}
                `}>
                <span className="text-slate-600 mr-1 font-mono font-size-[0.8em]">
                  {cl.line || ' '}
                </span>
              </div>;
            })}
          </pre>
        </div>
      </div>

      {/** Stack */}
      <div className="bg-slate-900/50 rounded-xl border" style={{borderColor: 'black'}}>
        <div className="text-[10px] font-bold text-slate-300">
          <span>Стек DFS</span>
          <span className="text-indigo-400">{step.stack.join(' ')}</span>
        </div>
        <div className="flex flex-col-reverse gap-1">
          {step.stack.length === 0
            ? <div className="text-[10px] text-slate-300">Стек пуст</div>
            : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0.5 text-[0.8em]
                ${i === step.stack.length - 1 ? "bg-emerald-600/20 text-emerald-400" : "text-slate-300"}
              `}>
                {id}
              </div>
            })}
        </div>
      </div>
    </div>
  </div>

```

```

    checkingEdge: { u: string; v: string } | null;
    distances: { [key: string]: number };
    visited: { [key: string]: boolean };
    previous: { [key: string]: string | null };
    log: string;
    activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

  const adjList: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (adjList[n.id] = []));
  edges.forEach(e => {
    adjList[e.u].push({ v: e.v, w: e.w });
  });
}

```

```

        minD = dist[n.id];
        u = n.id;
    }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `  Выбираем вершину ${u} с мин. непосещённой
    activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
        currentNode: u, checkingEdge: { u, v },
        distances: { ...dist }, visited: { ...visited },
        log: `  Смотрим соседа ${v}. Текущее расстояние
        ${dist[u] + edge.w}.`,
        activeCodeLine: 8,
    });

    if (dist[u] + edge.w < dist[v]) {
        dist[v] = dist[u] + edge.w;
        prev[v] = u;
        simulationSteps.push({
            currentNode: u, checkingEdge: { u, v },
            distances: { ...dist }, visited: { ...visited },
            log: `  Улучшение (Релаксация)! Новое кратчайшее
            activeCodeLine: 9,
        });
    }
}

```

```

        <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-indigo-300">Полн
    </div>
</div>

<div className="flex items-center gap-2">
    <button
        onClick={() => { setIsPlaying(false); setCu
        className="flex items-center gap-1 bg-slate
        ion-colors"
    >
        Сброс
    </button>
    <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py-2
            isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-indigo-600 hover:bg-indigo-500'
        }`
    >
        {isPlaying ? 'Пауза' : '▶ Пуск'}
    </button>
    <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length
        className="flex items-center gap-1 bg-indigo-600
        px-3 py-2 rounded-lg text-sm font-medium tr
    >
        ⏮
    </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml-6">
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/20
        stify-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indigo-950/20
            ont-bold shadow-sm">

```

```

        strokeDasharray={isChecking ? '4,4'
      />
    <circle
      cx={({uNode.x + vNode.x) / 2} cy={({uNode.y + vNode.y) / 2}
      r="12" fill="#1e293b"
      stroke={isChecking ? '#f59e0b' : '#f87192'}
    />
    <text
      x={({uNode.x + vNode.x) / 2} y={({uNode.y + vNode.y) / 2}
      fill={isChecking ? '#f59e0b' : '#f87192'}
      fontSize="11" fontWeight="bold" text={edge.w}
    >
      {edge.w}
    </text>
  </g>
  );
}
}

{/* Вершины */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id
  const isVisited = currentStep.visited[node.id]
  const dist = currentStep.distances[node.id]

  return (
    <g key={node.id} className="transition-all duration-300">
      <circle
        cx={node.x} cy={node.y}
        r={isCurrent ? 22 : 18}
        fill={isCurrent ? '#4f46e5' : isVisited ? '#f87192' : '#1e293b'}
        stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#f87192'}
        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-300"
      />
      <text x={node.x} y={node.y + 5} fill="#1e293b">
        {node.id}
      </text>
      <text x={node.x} y={node.y - 28} fill="#1e293b">
        {node.dist}
      </text>
    </g>
  )
})}
```

```

        <span key={idx} className={`px-3 py-2
            isCheckingThis ? 'bg-amber-500' : 'bg-white'
            600 text-slate-300'
            }`}>
            <span>{edge.v}</span><span cl
            <span className={isCheckingTh
        an>
            </span>
        });
    }}}
  </div>
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Таблица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        <div className="overflow-x-auto">
          <table className="w-full text-left border-collapse">
            <thead>
              <tr className="border-b border-rose-950">
                <th className="py-2 px-3">Вершина</th>
                <th className="py-2 px-3">Расстояние</th>
                <th className="py-2 px-3">Из (пред.
              </tr>
            </thead>
            <tbody className="text-sm">
              {nodes.map((n) => {
                const dist = currentStep.distances[n.i]
                const prev = currentStep.previous[n.i]
                const vis = currentStep.visited[n.i]
                const isCurr = currentStep.currentN

```

```

        </span>}
        </div>
    );
    }}}
    </div>
  </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 38/87: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}

```

```

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "  if (n in memo) return memo[n];",
  /* 3 */ "  if (n <= 2) return 1;",
  /* 4 */ "  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
  /* 5 */ "  return memo[n];",
  /* 6 */ "}"}

```



```

    if (n <= 2) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр
        codeLine: 3,
        memoState: { ...memoMap }
      });
      return 1;
    }

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
      codeLine: 4,
      memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'calc',
      desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
      codeLine: 5,
      memoState: { ...memoMap }
    });

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

```

```

        </div>
        <h3 className="text-2xl font-extrabold text-slate-400">
    </div>
    <p className="text-sm text-slate-400">
        Смотри, как таблица DP кэширует результаты
    </p>
</div>

<div className="flex flex-wrap items-center gap-2">
    <div className="flex items-center gap-2 bg-slate-950 p-2">
        <span>N:</span>
        <select
            value={targetN}
            onChange={(e) => setTargetN(Number(e.target.value))}
            disabled={isPlaying}
            className="bg-transparent text-white font-mono"
        >
            <option value={4}>N = 4</option>
            <option value={5}>N = 5</option>
            <option value={6}>N = 6</option>
            <option value={7}>N = 7</option>
        </select>
    </div>

    <div className="flex items-center bg-slate-950 p-2">
        <button
            onClick={() => { setIsPlaying(false); setTargetN(4); }}
            className="p-2 text-slate-400 hover:text-white"
            title="Сбросить"
        >
            <RotateCcw className="w-4 h-4" />
        </button>
        <button
            onClick={() => { setIsPlaying(false); setTargetN(4); setStepIndex(currentStepIndex - 1); }}
            disabled={currentStepIndex === 0}
            className="p-2 text-slate-400 hover:text-white"
            title="Шаг назад"
        >

```



```

<div className="bg-slate-950 rounded-2xl border-1 border-slate-800" style={{width: 100%, height: 100%;}}
  <div className="bg-slate-900 px-6 py-3 border-1 border-slate-800" style={{width: 100%; height: 100%;}}
    <span className="text-xs font-bold font-mono text-slate-300" style={{display: inline-block; width: 100px;}}>Шаг</span>
    <span className="text-xs text-emerald-400 font-mono" style={{display: inline-block; width: 100px;}}>{currentStepIndex}</span>
  </div>
  <pre className="p-6 font-mono text-sm space-x-2" style={{width: 100%; height: 100%;}}>
    {DP_CODE_LINES.map((line, idx) => {
      const lineNum = idx + 1;
      const isActive = currentStep?.codeLine === lineNum;
      return (
        <div
          key={lineNum}
          className={`flex items-center px-4 py-2 ${
            isActive
              ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
              : 'text-slate-300 hover:bg-slate-900'
          }`}
        >
          <span className="w-8 text-xs text-slate-300" style={{display: inline-block; width: 100px;}}>{lineNum}</span>
          <span>{line}</span>
        </div>
      );
    })}
  </pre>
  <div className="p-6 bg-slate-900/50 border-1 border-slate-800" style={{width: 100%; height: 100%;}}
    <span className="p-1.5 bg-emerald-500/20 text-white font-mono" style={{display: inline-block; width: 100px;}}>{currentStep?.desc}</span>
  </div>
</div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800" style={{width: 100%; height: 100%;}}
  <div className="flex items-center justify-between" style={{width: 100%; height: 100%;}}
    <span>Прогресс выполнения</span>
    <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
  </div>

```

```

const [c1Msg, setC1Msg] = useState("Клики на вершину");
const [c1Done, setC1Done] = useState(false);

const resetC1 = () => {
  setC1Path([]); setC1VisitedEdges([]); setC1Msg("Клики на вершину");
};

const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы пройти все рёбра");
    return;
  }
  const last = c1Path[c1Path.length - 1];
  const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
  if (!edge) {
    setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину");
    return;
  }
  const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
  if (c1VisitedEdges.includes(key)) {
    setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений");
    setC1Done(true); return;
  }
  const nextEdges = [...c1VisitedEdges, key];
  const nextPath = [...c1Path, vId];
  setC1VisitedEdges(nextEdges);
  setC1Path(nextPath);

  if (nextEdges.length === C1_EDGES.length) {
    const startDegree = C1_EDGES.filter(e => e.u === c1Path[0]).length;
    const endDegree = C1_EDGES.filter(e => e.v === c1Path[0]).length;
    const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
    if (isCycle && nextPath[0] === vId) {
      setC1Msg("    ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, факт!");
    } else {
      setC1Msg(`    ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! Сделай клик на вершину ${vId} и получишь цикл`);
    }
  }
};

```

```

        stroke={isLast ? "#fff" : visited ? "#666" : "#ccc"}
        strokeWidth={2.5}
        className="transition-all duration-200"
        <text x={n.x} y={n.y+4} textAnchor="middle">
        bel}</text>
      </g>
    )}
  </svg>
  <div className="absolute top-2 left-2 bg-slate-900 text-white p-2">
    Ребёп: {c1VisitedEdges.length}/{C1_EDGES.length}
  </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
  c1Done && c1VisitedEdges.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
    : c1Done
    ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
    : "bg-slate-900/50 border-slate-700 text-slate-50"}
  `}>
  {c1Msg}
</div>
</div>
);
}

```

ФАЙЛ 40/87: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
  id: number;
  question: string;
  options: string[];
  correctIndex: number;
}

```

```

    'Он использует меньше памяти благодаря коммунистиче
    'Он умеет работать с отрицательными весами без ци
    'Он позволяет выполнять шаги слияния колхозов (ко
    'Он был разработан в Токио и поэтому самый быстрый
  ],
  correctAnswer: 2,
  explanation: 'Абсолютно верно! В Боровке каждая ком
    делить вычисления на GPU или многопроцессорных кл
},
{
  id: 4,
  question: 'В чем главная суть Динамического програм
  options: [
    'Писать код очень быстро, динамично нажимая на кл
    'Запоминать (кешировать) результаты уже решенных
    'Использовать динамическую типизацию как в Python
    'Постоянно менять требования к проекту в процессе
  ],
  correctAnswer: 1,
  explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
  id: 5,
  question: 'Какая структура данных идеально помогает
  options: [
    'СНМ (Система непересекающихся множеств / Disjoin
    'Стек (LIFO).',
    'Красно-черное дерево.',
    'Хэш-таблица со списками коллизий.'
  ],
  correctAnswer: 0,
  explanation: 'Блестяще! СНМ (DSU) позволяет за прак
    '
  }
];

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num

```

```

    <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-600 shadow-xl shadow-indigo-500/30">
      <Award className="w-10 h-10 text-white" />
    </div>
    <h3 className="text-3xl font-bold text-white mb-6">Ты настоящий сеньор-раскрасчик?</h3>
    <p className="text-slate-400 text-sm mb-6">Ты набрал {score} баллов из {quizQuestions.length} возможных. Это означает:

    <div className="bg-slate-950 border border-slate-800 p-6">
      <p className="text-slate-400 text-sm uppercase font-medium">Оценки</p>
      <p className="text-5xl font-black text-indigo-400">{score}</p>
      <p className="text-slate-300 text-sm">
        {score === quizQuestions.length
          ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
          : score >= 3
            ? ' Отличный результат! Главные концепции ты освоил!'
            : ' Стоит повторить главы пособия и еще раз пройти тест.'
        }
      </p>
    </div>

    <button
      onClick={handleRestart}
      className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
    >
      <RotateCcw className="w-5 h-5" />
      <span>Пройти тест заново</span>
    </button>
  </div>
);
}

return (
  <div className="bg-slate-900/90 border border-slate-800 p-6">
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-4 py-2 rounded-lg">Экспресс-тест</span>
      </div>
    </div>
  </div>
);
}

```



```

        ) : isAnswered && isSelected && !isCo
        <XCircle className="w-6 h-6 text-ro
    ) : (
        <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
        {String.fromCharCode(65 + idx)}
        </div>
    )}
    </div>
    <div className="flex-1 text-sm md:text-l
        {option}
    </div>
</div>
    );
    }}}
</div>
</div>

{isAnswered && (
    <div className="bg-slate-950 border border-slat
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
        <p className="text-slate-300 text-sm leading-
            {currentQ.explanation}
        </p>
    </div>
)}

<div className="flex justify-end pt-4 border-t bo
    <button
        onClick={handleNext}
        disabled={!isAnswered}
        className={
            "px-8 py-3.5 font-bold rounded-xl transition
            (!isAnswered
                ? "bg-slate-800 text-slate-500 border bor

```

```
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];
```

```
const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
];
```

```
const adjList: { [key: number]: { v: number; w: number }[] } = {
  0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
  1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
  2: [{ v: 4, w: 2 }],
  3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
  4: [{ v: 6, w: 1 }],
  5: [{ v: 6, w: 5 }],
  6: [{ v: 1, w: 1 }],
};
```

```
const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V)' },
  { line: 2, text: '  dist = copy(matrix)' },
  { line: 3, text: '  for k from 0 to V - 1: // Поочередно для каждого узла' },
  { line: 4, text: '    for i from 0 to V - 1: // Для каждого узла i' },
  { line: 5, text: '      for j from 0 to V - 1: // Для каждого узла j' },
  { line: 6, text: '        if dist[i][k] + dist[k][j] < dist[i][j]' },
  { line: 7, text: '          dist[i][j] = dist[i][k] + dist[k][j]' },
  { line: 8, text: '  return dist' },
];
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);
```

```

    });

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Зарядка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-emerald-600 text-white rounded" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-emerald-300" >Полн
          </div>
        </div>
      </div>

      <div className="flex items-center gap-2" >
        <button onClick={() => { setIsPlaying(false);
          text-sm" > 60 секунд </button>
        <button onClick={() => setIsPlaying(!isPlaying
          -amber-600' : 'bg-emerald-600'}`}>{isPlaying

```

```

        isVia ? '#818cf8' : '#64748b'} strokeWidth
        <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
        </g>
      );
    });
  }}}
  {nodesSvg.map((node) => {
    const isK = currentStep.k === node.id;
    const isI = currentStep.i === node.id;
    const isJ = currentStep.j === node.id;
    let fill = '#1e293b'; let stroke = '#64748b';
    if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8'; }
    else if (isI) { fill = '#b45309'; stroke = '#a8a8a8'; }
    else if (isJ) { fill = '#047857'; stroke = '#a8a8a8'; }
    return (
      <g key={node.id} className="transition-colors">
        <circle cx={node.x} cy={node.y} r={isK ? 4 : 2} className="transition-colors duration-200">
          <text x={node.x} y={node.y + 5} fill="white" font-size="10" font-weight="bold">K</text>
          <text x={node.x} y={node.y + 32} fill="white" font-size="10" font-weight="bold">I</text>
        </g>
      );
    });
  }}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">Список смежности</p>
  <p className="min-h-[40px] flex items-center justify-center"></p>
</div>
</div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
  <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>
  <div className="space-y-2 text-sm font-mono">
    {Object.keys(adjList).map((uStr) => {

```

```

        {nodeNames.map((n, idx) => <th key={idx}
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
        </tr></thead>
        <tbody className="text-xs font-mono">
          {currentStep.matrix.map((row, i) => (
            <tr key={i} className="border-b borde
              <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`}>
                {nodeNames[i].split(' ')[0]} {i}
              </td>
              {row.map((val, j) => {
                const isUpd = currentStep.i === i
                const isVia = currentStep.k !== -
tep.j === j));
                let bg = 'bg-slate-900/40 text-sl
                if (isUpd) bg = 'bg-emerald-900/8
                else if (isVia) bg = 'bg-indigo-9
                else if (currentStep.i === -1 &&
                  bg = 'bg-indigo-950/60 text-ind
                else if (i === j) bg = 'bg-slate-
                return (<td key={j} className={`p
              /td>));
            })))
          </tr>
        })))
      </tbody>
    </table>
  </div>
</div>

```

```

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60
    <div>
      <h4 className="text-lg font-bold text-slate
      <div className="bg-slate-950/80 rounded-lg p
        {codeLines.map(item => (
          <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-ambe

```

```

    { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
  ];

const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};

type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
  const stack: string[] = []; // for visualization, j

  function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Замечено: ${v}`,
      visitedNodes: Array.from(vis) });

    for (const u of adj[v]) {
      if (!vis.has(u)) {
        steps.push({ type: 'process', node: v, desc: `Обработка: ${v}`,
          queueOrStack: stack, visitedNodes: Array.from(vis) });
        dfs(u);
      }
    }
  }
}

```

```

        om(vis)] });

    return steps;
}

export function GraphTraversalViz() {
    const [mode, setMode] = useState<"dfs" | "bfs">("df
    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);

    const steps = useMemo(() => mode === 'dfs' ? genera
    useVizStepSync(stepIdx, setStepIdx, steps.length -

    useEffect(() => {
        setStepIdx(0);
        setAutoPlay(false);
    }, [mode]);

    useEffect(() => {
        if (autoPlay) {
            const t = setInterval(() => {
                setStepIdx(p => {
                    if (p >= steps.length - 1) {
                        setAutoPlay(false);
                        return p;
                    }
                    return p + 1;
                });
            }, 1200);
            return () => clearInterval(t);
        }
    }, [autoPlay, steps.length]);

    const step = steps[stepIdx] || steps[0];

    const isVisited = (nodeId: string) => step.visitedN
    const isCurrent = (nodeId: string) => step.node ===

```

```

const current = isCurrent(n)

let fillColor = "fill-slate"
let strokeColor = "stroke-slate"
let textColor = "fill-slate"
let radius = 18;

if (visited) {
  fillColor = mode === 'done' ? 'fill-slate' : 'fill-white';
  strokeColor = mode === 'done' ? 'stroke-slate' : 'stroke-white';
  textColor = "fill-white";
}

if (current && step.type !== 'done') {
  strokeColor = "stroke-amber";
  textColor = "fill-amber";
  radius = 22;
}

return (
  <g key={n.id} className={className} >
    <circle cx={n.x} cy={n.y}
      className={`stroke-${strokeColor} fill-${fillColor} radius-${radius}`}
    />
    <text x={n.x} y={n.y}
      className={`text-${textColor} font-size-12 font-weight-normal`}
      {n.label}
    />
    {
      /* Ripple for BFS */
      current && mode === 'done' &&
      <circle cx={n.x} cy={n.y}
        className="stroke-white fill-white radius-22"
      />
    }
  </g>
)
  </svg>
</div>

```



```

        <button onClick={() => setAutoPlay(
          justify-center min-w-[80px] ${mode === 'df
        >
          {autoPlay ? <><Pause size={16} .
        </button>
        <button onClick={() => {setAutoPlay
          = steps.length - 1} className="p-2 text-slate-500
          hover:bg-transparent">
          <SkipForward strokeWidth={3} size={16} />
        </button>
        <button onClick={() => {setAutoPlay
          g-slate-800 rounded-lg ml-2 border-l border-l-slate-500
          <RefreshCw strokeWidth={3} size={16} />
        </button>
      </div>

      {/* Description Log */}
      <div className="flex-1 bg-slate-900/50 p-4 overflow-hidden">
        <div className="text-[10px] text-slate-400">
          {steps.length}</div>
        <div className="text-sm text-slate-400">
          {step.desc}
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 43/87: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';

```

```

    }
    return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
    return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
    { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
    let arr: Patient[] = [];
    const start = [
        { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤' },
        { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
        { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
        { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
        { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
        { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
    ];
    return arr;
})();

```

```

        symptom: finalSymptom,
        priority,
        animState: 'in',
    });

    const nextHeap = heapInsert(heap, newPatient).map(x => {
    // Mark specifically this new patient as 'in' in the heap
    const target = nextHeap.find(x => x.priority === pr
    const marked = nextHeap.map(x => x.id === (target?.id || 0));

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

    setTimeout(() => {
        setHeap(nextHeap);
        setBusy(false);
    }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
    e.preventDefault();
    if (!customName.trim()) return;
    insertPatient(customName, customPriority);
    setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
    if (busy || heap.length === 0) {
        setShake(true);
        addLog('⚠ Нет пациентов для экстренной госпитализации');
    }
}, [busy, heap]);

```

```

    }, [busy, heap]));

const reset = () => {
    setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
    setLog([' База данных скорой помощи перезагружена.']);
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))), 1000);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
    const res: React.ReactElement[] = [];
    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < heap.length) {
        const lp = nodePos(l);
        res.push(
            <line key={`el${idx}`}
                x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
                x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
                stroke="#475569" strokeWidth="2"
            />
        );
    }
    if (r < heap.length) {
        const rp = nodePos(r);
        res.push(
            <line key={`er${idx}`}
                x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
                x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
                stroke="#475569" strokeWidth="2"
            />
        );
    }
    return res;
});

```

```

        placeholder="Имя пациента"
        className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
    />
    <div className="flex flex-col items-center min-w-[300px]
      <span className="text-[9px] font-mono text-slate-400"
      <input
        type="range"
        min="10"
        max="99"
        value={customPriority}
        onChange={e => setCustomPriority(Number(e.target.value))}
        className="w-20 accent-emerald-500 cursor-pointer"
      />
    </div>
    <button
      type="submit"
      disabled={busy || !customName.trim() || heap.length === 0}
      className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-800 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
    >
      Везти
    </button>
  </form>

  {/* Забрать самого тяжелого */}
  <button
    onClick={extractMax}
    disabled={busy || heap.length === 0}
    className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium transition-all cursor-pointer flex items-center justify-center"
  >
    <span> В реанимацию (EXTRACT MAX)</span>
  </button>

  <button
    onClick={reset}

```

```

        top: `${pos.y}%`,
        width: isRoot ? 60 : 52,
        height: isRoot ? 60 : 52,
      }}
    >
    <span className="text-2xl leading-none">
    <span className="text-[10px] font-mono">
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full m-1"
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-white">
      <div className="text-emerald-400">
      </div>
    </div>
    </div>
    </div>
  );
  }}}
</div>

<div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Операционная койка
  </div>

  <div className="flex-1 flex flex-col justify-between p-4">
    {healingPatient ? (
      <div className="flex flex-col items-center justify-center h-100">
        <div className="relative w-full h-full">
          <span className="text-6xl block animate-pulse text-center">
            <span className="absolute -top-4 -right-4 text-2xl font-mono">

```

```
        <span>{entry}</span>
      </div>
    )))
  </div>
</div>
);
};
```

ФАЙЛ 44/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { createContext, useContext, useEffect, use } from 'react';

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими,
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше 1000 мс.
 * export const MOVE_MS = 800;
 * export const EASE = "cubic-bezier(.34,.01,.2,1)";
 * export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу — можно
 * export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);
```

```

    });
};

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);

```



```

    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
  </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [ "A", "B"], [ "B", "C"],
    [ "D", "E"],
    [ "F", "G"], [ "G", "H"], [ "F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => {
          <Edge key={` ${ci} - ${i}` } a={pos[u]} b={pos[v]}
        })
      })
    }
    {Object.entries(pos).map(([k, [x, y]]) => {
      const ci = compOf(k);
      return <Node key={k} x={x} y={y} r={12} label={
    }}}
  )}

```

```

    });
  }
  {Object.entries(pos).map(([k, [x, y]]) => {
    <Node key={k} x={x} y={y} label={k} fill={C.idle} />
  })}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={30} />
          {v}
        </text>
        </g>
      })}

      {step >= 1 &&
        sorted.map((v, i) => {

```

```

        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      )}}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="middle">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": 130, "x": 78}],
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": 130, "p": 182}],
    ]}
  />
);

/** Zig-Zig – x и p с одной стороны, крутим сначала вершину. */
export const ZigZigDemo: React.FC = () => (
  <SplayFrames
    captions={["Линия g → p → x («бамбук»)", "Поворот В"]}

```

```

const step = useLoop(order.length + 1, 850);
const visited = new Set(order.slice(0, step).flat());
const wave = step > 0 && step <= order.length ? order
return (
  <Svg caption={`Уровень ${Math.max(0, Math.min(step,
    {GRAPH_EDGES.map(([u, v], i) => {
      <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
    }}}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
      }}}
    </Svg>
  );
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS
      }}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={k ===
      }}}
    </Svg>
  );
};

export const ToposortDemo: React.FC = () => {

```

```

const improved = step === 2;
return (
  <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь" : ""}>
    <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
    <text x={130} y={54} textAnchor="middle" fontSize={16} />
    <Node x={60} y={70} r={20} fill={C.active} label="3" />
    <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
    <text x={60} y={106} textAnchor="middle" fontSize={16} />
    <text x={200} y={106} textAnchor="middle" fontSize={16} />
    <text x={130} y={22} textAnchor="middle" fontSize={16} />
  </Svg>
);
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active} />
      ))}
    </Svg>
  );
};

export const ArticulationDemo: React.FC = () => {

```

```

        <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
        <text x={({pos[e.u][0] + pos[e.v][0]) / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
        </g>
    );
  }}}
  {Object.entries(pos).map(([k, [x, y]]) => (<Node
</Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step
      })}
    </Svg>
  );
};

```

ФАЙЛ 47/87: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```
import React from "react";
```

```
export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к"
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>выход</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>вход</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};
```

```
export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f44336"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = объединение"
      {p.map((par, i) => (
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`} />
        )
      ))}
    </Svg>
  );
};
```

```

        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
    ]}]
    ]}]
    {TRIE_NODES.map((n) => {
        <Node key={n.id} x={n.x} y={n.y} label={n.ch}
        fill={wave.includes(n.id) ? C.active : doneId
    ]}]
    ]}]
    <text x={6} y={12} fontSize={9} fill={C.dim}>очер
</Svg>
    ]];
};

export const SuffixLinkDemo: React.FC = () => {
    const links: [number, number][] = [[1, 0], [2, 0], [3
    const step = useLoop(links.length + 1, 900);
    return (
        <Svg caption="Суффиксная ссылка: «куда откатиться,
        {TRIE_NODES.filter((n) => n.parent !== null).map(
            const p = TRIE_NODES[n.parent as number];
            return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
        ]}]
        ]}]
        {links.slice(0, step).map(([from, to], i) => {
            const a = TRIE_NODES[from];
            const b = TRIE_NODES[to];
            return (
                <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
                fill="none" stroke={C.warn} strokeWidth={1.
            ]];
        ]}]
        {TRIE_NODES.map((n) => {
            <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
        ]}]
        ]}]
        </Svg>
    ]];
};

export const SegmentTreeDemo: React.FC = () => {
    const step = useLoop(3, 1000);

```



```

        </g>
      )}}
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={12}
          fill={show && i === l ? C.bad : show && i === l ? C.dim : C.idle}
          stroke={C.idleStroke} style={{ transition: 'stroke 0.5s' }} />
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    )}}
  </Svg>
);
};

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.idle} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s' }} />
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      )}}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.activeStroke} style={{ transition: 'stroke 0.5s' }} />
      )}}
    </Svg>
  );
};

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];

```

```

<Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
  {s.split("").map((ch, idx) => (
    <g key={idx}>
      <rect x={18 + idx * 33} y={26} width={28} height={24}
        fill={step > 0 && (idx < len || (idx > i - 1) ? C.active : C.idle)}
        style={{ transition: "fill .25s" }} />
      <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={10} fill="black">
        {ch}
      </text>
      <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={10} fill="black">
        {idx <= i && step > 0 ? C.warn : C.idle}
      </text>
    </g>
  )
  )
  <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
  <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
  <text x={130} y={106} textAnchor="middle" font-size={10} fill="black">
    π[i] — длина самого длинного «префикс = суффикс»
  </text>
</Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step / 320 * 100}%`}
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : filled ? C.filled : C.empty}
                style={{ transition: "fill .25s" }} />
              {filled && (
                <text x={57 + c * 38} y={36 + r * 32} text-align="center" font-size={10} font-weight="700" fill="black">
                  {idx}
                </text>
              )}
            </g>
          )
        })
      )
    )
  );
};

```

```

    dfs: DfsDemo,
    heap: HeapDemo,
    stack: StackDemo,
    queue: QueueDemo,
    dsu: DsuDemo,
    trie: TrieDemo,
    "trie-bfs": TrieBfsDemo,
    "suffix-link": SuffixLinkDemo,
    toposort: ToposortDemo,
    relax: RelaxDemo,
    "prefix-sum": PrefixSumDemo,
    "sparse-table": SparseTableDemo,
    "segment-tree": SegmentTreeDemo,
    bridge: BridgeDemo,
    articulation: ArticulationDemo,
    mst: MstDemo,
    amortized: AmortizedDemo,
    np: NpDemo,
    scc: SccDemo,
    "prefix-function": PrefixFunctionDemo,
    dp: DpDemo,
    floyd: FloydDemo,
    components: ComponentsDemo,
    graph: GraphBasicsDemo,
    "coord-compress": CoordCompressDemo,
    zig: ZigDemo,
    "zig-zig": ZigZigDemo,
    "zig-zag": ZigZagDemo,
  };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
  const [paused, setPaused] = useState(false);
  const Cmp = REGISTRY[kind];
  if (!Cmp) return null;
  return (
    <DemoPauseContext.Provider value={paused}>
      <div
        className="bg-slate-950/70 rounded-lg border bo

```

```

    }, [vizId, onClose]);

    const viz = getViz(vizId ?? undefined);
    if (!vizId || !viz) return null;
    const Component = viz.Component;

    return createPortal(
      <div className="fixed inset-0 z-[110] flex items-end">
        <div className="absolute inset-0 bg-slate-950/80">
          <div className="relative w-full sm:max-w-5xl max-h-2xl shadow-2xl flex flex-col">
            <div className="flex items-center gap-3 px-4 py-4">
              <div className="min-w-0 flex-1">
                <h3 className="font-bold text-white text-sm">
                  {viz.hint && <p className="text-[11px] text-white">
                    {viz.description}
                  }
                </h3>
              </div>
              <button
                onClick={onClose}
                className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
                aria-label="Заккрыть симулятор"
              >
                <X className="w-5 h-5" />
              </button>
            </div>
            <div className="overflow-y-auto overscroll-contain">
              <Component />
            </div>
          </div>
        </div>,
        document.body
      );
  };
};

```

```

    let left = anchor.rect.left + anchor.rect.width / 2
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor

    setPos({ top: Math.max(8, Math.min(top, vh - height
    }, [anchor]));

useEffect(() => {
  if (!anchor) return;
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  window.addEventListener("scroll", onClose, true);
  return () => {
    window.removeEventListener("keydown", onKey);
    window.removeEventListener("scroll", onClose, true);
  };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD W, typeof window !== "unde

```

```
        >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    })
  </div>
</div>
</div>,
document.body
);
};
```

ФАЙЛ 51/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync } from '../data/vizStepBus';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP);

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 2, labelDx: 0, labelDy: 0 },
  ];
```

```

    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "";
    if (step === 1) return "url(#arrow-amber)";
    if (step === 2) return "url(#arrow-pink)";
    return "url(#arrow-slate-dim)";
  }
  if (e.id === 'AB' && step === 4) return "url(#a
  if (e.id === 'AB' && step > 4) return "url(#arr
  if (e.id === 'BC' && step === 5) return "url(#a
  if (e.id === 'BC' && step > 5) return "url(#arr
  if (e.id === 'CA' && step === 6) return "url(#a
  if (e.id === 'CA' && step > 6) return "url(#arr

  if (step >= 4) return "url(#arrow-slate-dim)";
  return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

  if (step < 4) return e.weight;

  if (e.id === 'AB') {
    if (step === 4) return `${e.weight} + (${0}
    return e.newWeight;
  }
  if (e.id === 'BC') {
    if (step < 5) return e.weight;
    if (step === 5) return `${e.weight} + (${-2}
    return e.newWeight;
  }
}

```

```

case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгоритм
        на всех вершинах:
        <span className="text-pink-300 ml-1">
             $h(v) = \min_{(u,v) \in E} (h(u) + w(u,v))$ 
        </span>
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый вес:
        <div className="text-center font-monospace">
             $w'(u,v) = w(u,v) + h(u) - h(v)$ 
        </div>
        Это повышает или понижает вес ребра.
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-green-500">
             $w(A,B) = 2$ 
        </span>
        Вычисляем:  $-2 + 0 - (-2) = 0$ 
    </div>
);
case 5: return (
    <div>
        <b>Шаг 3.2: Пересчет ребра  $B \rightarrow C$ .</b>
        Старый вес: 1. <br/>
        Вычисляем:  $1 + (-2) - (-1) = 0$ 
    </div>
);
case 6: return (
    <div>
        <b>Шаг 3.3: Пересчет ребра  $C \rightarrow A$ .</b>
        Старый вес: 2. <br/>
        Вычисляем:  $2 + (-1) - 0 = 1$ 
    </div>
);
case 7: return (

```



```

        </marker>
        <marker id="arrow-emerald"
o-start-reverse">
            <path d="M 0 0 L 10 5 L
        </marker>
    </defs>

```

```

{/* Edges */}
{edges.map((e, i) => {
    const u = nodes.find(n => n
    const v = nodes.find(n => n

    const isS = e.u === 'S';
    if (isS && (step === 0 || s

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getE
    const textHigh = isEdgeText
    const textEmer = isEdgeText

    return (
        <g key={i}>
            <line
                x1={u.x} y1={u.y}
                className={`str
                markerEnd={getM
            />

            <text
                x={midX} y={midY}
                textAnchor="mid
                dominantBaselin
                className={`tex
                    ${textHigh
                    textEmer
                    isS ? 'fi

```

```

        )
      }}}}
    </svg>
  </div>

  <div className="w-full md:w-[320px] flex
    {/* Log Box */}
    <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
      {getDesc()}
    </div>

    {/* Controls */}
    <div className="flex bg-slate-900 border
      <button onClick={() => setStep(1)}
        className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
        <RotateCcw strokeWidth={3}
      </button>
      <button onClick={() => setStep(1)}
        className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
        <Undo strokeWidth={3} size={24}
      </button>
      <button onClick={() => setStep(
        className={`flex-1 font-bold text-white
          ${step === MAX_STEP
            ? 'bg-emerald-600/50'
            : 'bg-indigo-600 hover:bg-indigo-500'}
          {step === MAX_STEP ? "Готово" : "В процессе"}
          {step !== MAX_STEP && <Skip
        </button>
    </div>

    {/* Progress Dots */}
    <div className="flex justify-center gap-2"
      {Array.from({length: MAX_STEP + 1}, (_, i) =>
        <div key={i} className={`w-4 h-4 rounded-full

```

```

    { id: 1, x: 300, y: 100, label: "1" },
    { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
    { id: 3, x: 450, y: 160, label: "3" },
    { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

```

```

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

```

```

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

```

```

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const [isPlaying, setIsPlaying] = useState(false);

```

```

  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
    const visited = new Set<number>();
    const order: number[] = [];

```

```

    visited.add(u);
    recordStep(
        "dfs1",
        `DFS 1: зашли в вершину ${u}, помечаем посещение
        u,
        false,
        -1,
    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs1(to);
        }
    }

    order.push(u);
    recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи
        u,
        false,
        -1,
    );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологиче
    ебра.`,
    null,

```

```

        u,
        true,
        sccId,
    );
    dfs2(u, sccId);
} else {
    recordStep(
        "dfs2",
        `DFS 2: вершина ${u} из стека уже покрашена в
        u,
        true,
        sccId,
    );
}
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,

```

```

        d-500 text-white"
    >
    {isPlaying ? {
        <Pause className="w-4 h-4 ml-1" />
    } : {
        <Play className="w-4 h-4 ml-1" />
    }}
    {isPlaying ? "Пауза " : "Старт "}
</button>

<button
    onClick={reset}
    className="flex items-center justify-center
ors"
>
    <RotateCcw className="w-4 h-4" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 >
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
    <svg className="w-full h-full max-w-[650px]" >
        {/* Markers for directed arrows */}
        <defs>
            <marker
                id="arrow"
                viewBox="0 0 10 10"
                refX="22"
                refY="5"
                markerWidth="6"
                markerHeight="6"
                orient="auto-start-reverse"
            >
                <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
            </marker>
            <marker

```

```

        let stroke = "#475569";
        let markerId = "arrow";

        if (step.transposed) {
            stroke = isSccEdge ? "#818cf8" : "#4f46e5";
            markerId = "arrow-transposed";
        } else {
            if (
                step.currentNode === edge.u ||
                step.currentNode === edge.v
            ) {
                stroke = "#10b981";
                markerId = "arrow-active";
            }
        }

        // Adjust slightly for bidirectional link
        const isBidi =
            (edge.u === 3 && edge.v === 4) ||
            (edge.u === 4 && edge.v === 3);
        const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

        return (
            <line
                key={`edge-${idx}`}
                x1={x1}
                y1={y1 + dy}
                x2={x2}
                y2={y2 + dy}
                stroke={stroke}
                strokeWidth="2.5"
                markerEnd={`url(#${markerId})`}
                className="transition-all duration-300"
            />
        );
    }
}

{/* Nodes */}

```

```

        {sccId && {
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        }}
    </g>
};
}}
</svg>

```

```

{step.transposed && {
    <div className="absolute top-4 left-4 bg-in
    1 rounded">
        Граф транспонирован (Ребра обратные)
    </div>
}}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
    <h4 className="text-xs font-bold text-slate-4
        Порядок обхода и стек
    </h4>

```

```

<div className="bg-slate-900 border border-sl
    <p className="text-xs text-white leading-re
        {step.desc}
    </p>
</div>

```

```

-----
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
  ];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal');
  const [vertices, setVertices] = useState<Vertex[]>([]);
  const [edges, setEdges] = useState<Edge[]>(initialEdges);
  const [stepIndex, setStepIndex] = useState<number>(0);
  const [heapQueue, setHeapQueue] = useState<string[]>([]);
  const [activeCheatTab, setActiveCheatTab] = useState<string>('');

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Представим, что мы начинаем!',
      type: 'info',
    },
  ]);

  // --- KRUSKAL STEPS ---

```



```

    },
    // Step 13: Inspect D-G (8)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-6' ? {
            setLogs(prev => [{
                title: 'Шаг 7: Берем ребро D-G (вес 8)',
                description: 'Оно соединяет фиолетовую вершину I',
                type: 'info'
            }, ...prev]));
    },
    // Step 14: Include D-G
    () => {
        setVertices(prev => prev.map(v => v.id === 6 ? {
            setEdges(prev => prev.map(e => e.id === '3-6' ? {
            setLogs(prev => [{
                title: 'Успех: Ребро D-G в осто́ве',
                description: 'Вершина G окрашена в фиолетовый.',
                type: 'success'
            }, ...prev]));
    },
    // Step 15: Inspect C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Шаг 8: Берем ребро C-F (вес 9)',
                description: 'Оно соединяет вершины C и F, кото',
                type: 'warning'
            }, ...prev]));
    },
    // Step 16: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Отказ: Цикл обнаружен!',
                description: 'Выбрасываем ребро C-F.',
                type: 'error'
            }, ...prev]));
    },

```

```

    () => {
      setEdges(prev => prev.map(e => e.id === '7-8' ? {
      setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю',
        type: 'info'
      }, ...prev]));
    },
  // Step 22: Include H-I
  () => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
      title: '    Успех: MST построено Краскалом!',
      description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Сумма весов ребер: 3 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
      type: 'success'
    }, ...prev]));
  },
];

// --- PRIM STEPS ---
const primSteps = [
  // Step 1: Start at E
  () => {
    setVertices(prev => prev.map(v => v.id === 4 ? {
    setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],
    setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8']),
    setLogs(prev => [{
      title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
      description: 'Мы выбрали вершину E как стартовую вершину. Добавили в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8).',
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Pop D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {

```

```

    },
    // Step 6: Pop A-B (2)
    () => {
        setEdges(prev => prev.map(e => e.id === '0-1' ? {
            setLogs(prev => [{
                title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
                description: 'Плесень стремительно бежит к вершине A.',
                type: 'info'
            }, ...prev]));
    },
    // Step 7: Include A-B, add B's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 1 ? {
            setEdges(prev => prev.map(e => e.id === '0-1' ? {
                eap' } : e));
        setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
        setLogs(prev => [{
            title: 'Успех: Вершина B захвачена плесенью',
            description: 'В кучу добавлено B-C(4). Ребро B-A (2) включено.',
            type: 'success'
        }, ...prev]));
    },
    // Step 8: Pop B-C (4)
    () => {
        setEdges(prev => prev.map(e => e.id === '1-2' ? {
            setLogs(prev => [{
                title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
                description: 'Плесень тянется к вершине C.',
                type: 'info'
            }, ...prev]));
    },
    // Step 9: Include B-C, add C's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 2 ? {
            setEdges(prev => prev.map(e => e.id === '1-2' ? {
                eap' } : e));
        setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
        setLogs(prev => [{

```

```

    setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
    setLogs(prev => [{
      title: 'Успех: Вершина F захвачена',
      description: 'Ребро F-I(13) добавлено в кучу. R
      type: 'success'
    }, ...prev]));
  },
  // Step 14: Pop D-G (8)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
      title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
      description: 'Плесень расширяется вниз к G.',
      type: 'info'
    }, ...prev]));
  },
  // Step 15: Include D-G, add G's edges
  () => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
      eap' } : e));
    setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
    setLogs(prev => [{
      title: 'Успех: Вершина G захвачена',
      description: 'В кучу добавлено G-H(10).',
      type: 'success'
    }, ...prev]));
  },
  // Step 16: Pop C-F (9) - Cycle!
  () => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
      title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
      description: 'Вершины C и F уже заражены плесенью',
      type: 'warning'
    }, ...prev]));
  },
  // Step 17: Reject C-F

```

```

    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
            setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setHeapQueue(['F-I (13 - цикл)']));
        setLogs(prev => [{
            title: 'Успех: Плесень захватила весь граф',
            description: 'Все 9 вершин поглощены. Итоговый граф',
            description: 'иной – мы разрастались из одной точки с помощью',
            type: 'success'
        }, ...prev]));
    },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {

```



```

    setLogs(prev => [{
      title: 'Слияние: Деревни объединились в колхозы'
      description: 'Все выбранные дороги построены ра
        оз {D, E, F, G, H, I}.',
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Five-Year Plan 2 (Inspecting bridge betw
  () => {
    // Now both kolkhozes look at all outgoing roads
    // Outgoing roads between {A, B, C} and {D, E, F,
    // A-D (5), B-E (6), C-F (9).
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
      title: 'Вторая пятилетка: Колхозы выбирают мост
      description: 'Теперь Красный и Синий колхозы од
        -D с весом 5. Остальные мосты B-E(6) и C-F(9)
      type: 'warning'
    }, ...prev]));
  },
  // Step 4: Concurrently merge into single Kolkhoz
  () => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
      e.status === 'included' || e.id === '0-3' ? { .
    }));
    setLogs(prev => [{
      title: 'Объединение завершено: Один гигантский
      description: 'Оба колхоза объединились по мосту
      type: 'success'
    }, ...prev]));
  },
  // Step 5: Check remaining edges & mark as rejected
  () => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
      title: ' Успех: MST построен Борувкой за 2 шаг

```

```

        едь (Heap) и захватываем соседей!',
        type: 'info',
    },
  ]);
} else {
  setLogs([
    {
      title: 'Введение в Коллективизацию (Борувка)',
      description: 'Логика «Борувки»: Каждая из 9 д
        чтобы объединиться в колхоз.',
      type: 'info',
    },
  ]);
}
};

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

```

```

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
  }

```

```

        <span>Краскал (Билет 18)</span>
      </button>
      <button
        onClick={() => handleReset('prim')}
        className={
          "flex items-center gap-1.5 px-3 py-2
+
          (activeAlgo === 'prim'
            ? 'bg-emerald-600 text-white shadow-md rounded-xl border
            : 'text-slate-400 hover:text-slate-500'
          )
        >
        <Layers className="w-3.5 h-3.5" />
        <span>Прима (Билет 19)</span>
      </button>
      <button
        onClick={() => handleReset('boruvka')}
        className={
          "flex items-center gap-1.5 px-3 py-2
+
          (activeAlgo === 'boruvka'
            ? 'bg-rose-600 text-white shadow-md rounded-xl border
            : 'text-slate-400 hover:text-slate-500'
          )
        >
        <Users className="w-3.5 h-3.5" />
        <span>Борувка (Билет 20)</span>
      </button>
    </div>

    <button
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-center gap-2 py-2
-200 font-semibold text-xs rounded-xl border
    >
      <RotateCcw className="w-3.5 h-3.5" />
      <span>Сбросить</span>
    </button>

```

```

        <div className="flex items-center gap-4">
          <div className="w-3 h-3 rounded-full bg-slate-300">
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
          </div>
        </div>
      ) : activeAlgo === 'prim' ? (
        <div className="flex items-center gap-4">
          <div className="w-3 h-1 bg-sky-600">
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
          </div>
        </div>
      ) : (
        <div className="flex items-center gap-4">
          <div className="w-3 h-3 rounded-full bg-slate-300">
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
            <div className="flex items-center gap-4">
              <div className="w-3 h-3 rounded-full bg-slate-300">
              </div>
            </div>
          </div>
        </div>
      )
    </div>

    <div className="absolute top-4 right-4 bg-slate-300 p-4 text-slate-300">
      {Math.floor(stepIndex / currentSteps.length)}
    </div>

    { /* SVG Edges */ }
    <svg className="absolute inset-0 w-full h-full" />

```

```

    }}}
  </svg>

  {/* HTML Overlay for Vertices */}
  <div className="absolute inset-0 w-full h-full" >
    {vertices.map(vertex => {
      <div
        key={vertex.id}
        style={{ top: vertex.y + "%", left: vertex.x + "%" }}
        className={"absolute -translate-x-1/2 -translate-y-1/2"}
        font-bold text-base border-2 transition-all duration-200
        id={vertex.id} >
          <span>{vertex.label}</span>
          {vertex.id === 4 && activeAlgo === 'prim' && !isHighlighted} &&
            <span className="text-[8px] block -mt-10px" >
              {vertex.id}
            </span>
          </div>
        </div>
      </div>
    </div>

    {stepIndex >= currentSteps.length && (
      <div className="absolute inset-x-6 bottom-6"
        shadow-xl z-20">
        <p className={"font-bold text-base " +
          "text-rose-300"} >
          MST построено с помощью {activeAlgo}
        </p>
        <p className="text-slate-300 text-xs mt-2">
          Общий вес минимального остова: 51. Всего
        </p>
      </div>
    )}
  </div>

  {/* Logs / Heap */}
  <div className="bg-slate-950 rounded-2xl border-2 border-slate-900" >
    <div className="p-4 bg-slate-900 border-b border-slate-950" >

```

```

    }
  >
    <div className="flex items-center gap-2">
      {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
      {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
      {log.type === 'error' && <XCircle className="text-red-500"/>
      {log.type === 'info' && <Play className="text-blue-500"/>
      <h5 className="font-bold text-sm text-slate-600">{log.message}</h5>
    </div>
    <p className="text-xs text-slate-300">{log.timestamp}</p>
  </div>
  )))
</div>
</div>
</div>
</div>

```

```

{/* Cheat Sheet: Complexity & Code for all 3 algo */}
<div className="bg-slate-900/80 border border-slate-700 p-4">
  <div className="flex flex-col sm:flex-row items-center gap-2">
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-400"/>
      <h4 className="text-xl font-bold text-white">Cheat Sheet</h4>
    </div>
    <div>
      <div className="flex bg-slate-950 p-1 rounded">
        <button
          onClick={() => setActiveCheatTab('kruskal')}
          className={
            "px-3 py-1.5 rounded text-xs font-medium" +
            (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300")
          }
        >
          Краскал (Билет 18)
        </button>
        <button
          onClick={() => setActiveCheatTab('prim')}
          className={
            "px-3 py-1.5 rounded text-xs font-medium" +
            (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-900 text-slate-300")
          }
        >
          Прим
        </button>
      </div>
    </div>
  </div>

```

```

<p className="text-slate-400 text-xs font-medium">
  <Code className="w-3.5 h-3.5 text-indigo-500 font-mono">
</p>
<pre className="text-xs text-emerald-400 font-mono">
  80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
</pre>
</div>
</div>
</div>
</div>
{activeCheatTab === 'prim' && (
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-emerald-400">
</p>
        <p className="text-2xl font-black text-white">

```

```
        heapq.heappush(heap, (next_w, v, u))
    return mst`}
```

```
    </pre>
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
}}
```

```
{activeCheatTab === 'boruvka' && {
```

```
  <div className="grid grid-cols-1 lg:grid-cols-
```

```
    <div className="lg:col-span-1 space-y-4">
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
          <Clock className="w-3.5 h-3.5 text-ro
```

```
        </p>
```

```
        <p className="text-2xl font-black text-r
```

```
        <p className="text-slate-400 text-xs mt
```

```
      <p>
```

```
    </div>
```

```
    <div className="bg-slate-950 p-4 rounded-
```

```
      <p className="text-slate-400 text-xs for
```

```
        <Award className="w-3.5 h-3.5 text-ro
```

```
      </p>
```

```
      <p className="text-2xl font-black text-r
```

```
      <p className="text-slate-400 text-xs mt
```

```
    </div>
```

```
    <div className="bg-slate-950 p-4 rounded-
```

```
      <p className="text-slate-400 text-xs for
```

```
ении:</p>
```

```
      <p className="text-xs text-slate-300 le
```

```
, организуя массовое распараллеленное слиян
```

```
    </div>
```

```
</div>
```

```
<div className="lg:col-span-2">
```

```
  <div className="bg-slate-950 p-4 rounded-
```

```
    <p className="text-slate-400 text-xs for
```

```
      <Code className="w-3.5 h-3.5 text-ros
```

```
    </p>
```



```
const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '    Все ко Все (Флойд)',
    desc: 'Матрица + три цикла (',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: ')). Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
    titleColor: 'text-emerald-400',
```

```

    setSelectedChapterId: (id: string) => void;
    /** Заголовок текущей темы — показываем в кнопке, что
    currentTitle: string;
    index: number;
    total: number;
  }

  /**
   * Мобильное содержание: липкая панель снизу-сверху + в
   * На десктопе не рендерится (там обычный сайдбар).
   */
  export const MobileToc: React.FC<Props> = ({ selectedCh
    const [open, setOpen] = useState(false);

    useEffect(() => {
      document.body.style.overflow = open ? "hidden" : ""
      return () => {
        document.body.style.overflow = "";
      };
    }, [open]);

    useEffect(() => {
      const onKey = (e: KeyboardEvent) => e.key === "Escap
      window.addEventListener("keydown", onKey);
      return () => window.removeEventListener("keydown",
    }, []));

    return (
      <>
        <div className="lg:hidden sticky top-[57px] z-40
          <button
            onClick={() => setOpen(true)}
            className="w-full flex items-center gap-2.5 p
            aria-label="Открыть содержание"
          >
            <span className="p-1.5 rounded-lg bg-indigo-600
              <List className="w-4 h-4" />
            </span>

```

ФАЙЛ 56/87: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, compilerOpen, onToggleCompiler }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 text-white p-3 rounded-lg">
```

```

<button
  id="tab-compiler-btn"
  onClick={onToggleCompiler}
  aria-pressed={compilerOpen}
  title="Боковая панель Python-компилятора (на
    страницах)"
  className={`flex-1 lg:flex-none flex items-
    uration-200 whitespace-nowrap ${
      compilerOpen
        ? 'bg-emerald-600 text-white shadow-lg
        : 'text-slate-400 hover:text-white'
    }`}
>
  <Terminal className="w-4 h-4" /> Python
</button>
<a
  id="download-pdf-btn"
  href={` ${import.meta.env.BASE_URL}export/full
  download="full_code_all_pages.pdf"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
  over:bg-emerald-600/20 border border-emerald
  title="Скачать полный PDF сборник всех стра
>
  <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
  id="download-txt-btn"
  href={` ${import.meta.env.BASE_URL}export/full
  download="full_code_all_pages.txt"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
  :bg-sky-600/20 border border-sky-500/30 tra
  title="Скачать полный TXT файл со всем кодом
>
  <FileText className="w-4 h-4" /> TXT

```

```

        {id:1,x:270,y:100},
        {id:2,x:230,y:230},
        {id:3,x:90,y:230},
        {id:4,x:50,y:100}
    ];
    const edges = [
        [0,1],[0,2],[0,3],[0,4],
        [1,2],[1,3],[1,4],
        [2,3],[2,4],
        [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return
        const p1=findPt(a),p2=findPt(b),p3=find
        function ccw(ax:number,ay:number,bx:num
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1]
                if (!crosses.includes(i)) crosses.p
                if (!crosses.includes(j)) crosses.p
            }
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
            stroke={xing ? "#f43f5e" : "#4f46e5"}
        }
    }
    const labels = ["A","B","C","D","E"];

```

```

        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
            if (!crosses.includes(i)) crosses.push(i);
            if (!crosses.includes(j)) crosses.push(j);
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
            stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
    }
    const labels = ["\u04141", "\u04142", "\u04143"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
            <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
  })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" style={z-20">
  V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

```

<div className="bg-amber-950/40 border border-amber-950/40 padding-10">
  ▲ Запомни: K5 и K3,3 – два кита непланарности.
  гомеоморфный K5 или K3,3 – он непланарен. Теорема Кюри.
</div>

```

```

    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

interface PythonCompilerProps {
  /** Страница, с которой синхронизируется редактор. */
  chapterId: string;
  chapterTitle: string;
  /** Перейти к странице пособия (посмотреть визуализацию) */
  onOpenGuide: () => void;
  /** Скрыть панель. */
  onClose: () => void;
}

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL);
  }
}

```

```

        label: "sorted(key=...)",
        tip: "Сортировка по ключу: рёбра по весу, пары по...",
        code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "A", "C")]\nedges\n',
    },
],
},
{
    group: "Циклы",
    items: [
        {
            label: "for i in range(n)",
            tip: "Вложенные циклы по i и j – как обход матрицы",
            code: "n, m = 3, 4\nfor i in range(n):",
        },
        {
            label: "while ...",
            tip: "Классический while-счётчик – как n-1 итерация",
            code: "i = 1\nwhile i < 5:\n    print(f\"итерация {i}\")\ni += 1",
        },
        {
            label: "for i, x in enumerate(...)",
            tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
            code: "for i, x in enumerate(['a', 'b', 'c']):",
        },
    ],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop = pop",
            code: 'st = []\nst.append("A")    # push\nst.pop()    # pop',
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop(0) тоже, но медленнее",
            code: 'from collections import deque\nq = deque()\nq.append("A")    # push\nq.popleft()    # pop',
        },
    ],
},

```



```

const [demoId, setDemoId] = useState<string | null>(null);
const sync = useMemo(() => getPageSync(chapterId, demoId));
const syncVarBases = useMemo(() => sync.variables.map((v) => v.bases));
const syncVarSet = useMemo(() => new Set(syncVarBases));
/** Полный эталонный код – подсматривается «глазом», не редактируется */
const template = useMemo(() => buildSyncTemplate(chapterId));
/** Дефолт редактора: шапка + только инициализированные переменные */
const initTemplate = useMemo(() => buildInitTemplate(chapterId));
/** Помеченные строки эталона по содержимому: каждое слово – строка */
const markedContents = useMemo(() => {
  const marks = sync.stepCodeLines;
  if (!marks || marks.length === 0) return undefined;
  const lines = template.split("\n");
  return new Set(marks.map((l) => (lines[l + 2] ?? "")));
}, [sync, template]);
/** Режим «подсмотреть эталон»: показывает полный код */
const [showRef, setShowRef] = useState(false);

const [code, setCode] = useState(() => {
  if (!savedEditor) return initTemplate;
  if (savedEditor.code === savedEditor.template || savedEditor.code.trim() === "")
    return savedEditor.code;
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Нажмите «Запустить»");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "highlight">("highlight");
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

const [desyncedFrom, setDesyncedFrom] = useState<string | null>(
  savedEditor.code !== savedEditor.template ||
  savedEditor.chapterId !== chapterId ||
  savedEditor.code.trim() !== ""
);

```

```

    setCode(initTemplate);
    setDesyncedFrom(null);
    setStripTab("vars");
    setDebug(null);
    setShowRef(false);
  }, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(showRef ? template : code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – бу");
  }
}, [code, showRef, template]);

/* — Пошаговый отладчик —————

/** Редактор – в точности эталон (для подписи chip).
const lockedToViz = code === template;
/** Пользователь дописал тело сам, перепечатав помощью
const contentMatched = useMemo(() => {
  if (!markedContents || markedContents.size === 0) return true;
  const written = new Set(code.split("\n").map((l) => l.trim()));
  return [...markedContents].every((m) => written.has(m));
}, [code, markedContents]);
/** Демонстрация реально умеет ходить по шагам за отладкой
const stepCapable = !!sync.code && sync.stepDriven === true;
/** Связь кода с демонстрацией: по эталону, по перепечатанному
type VizLink = "exact" | "content" | "none";
const vizLink: VizLink = stepCapable ? (lockedToViz ? "exact" : "content") : "none";

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d) return d;
      const idx = Math.max(0, Math.min(d.result.steps.length - 1, nextIdx));
      return { ...d, result: { ...d.result, steps: d.result.steps[idx] } };
    });
  }, [setDebug]
);

```

```

        setRuntimeReady(true);
        runtime.setStdout({ batched: (message) => stdout.log(message) });
        runtime.setStderr({ batched: (message) => stderr.log(message) });
        runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

        const raw = await runtime.runPythonAsync(buildDebugCode(code));
        const result = JSON.parse(String(raw)) as DebugResult;

        setDebug({ result, idx: 0, src: code });
        if (stepCapable) emitVizStep(chapterId, vizStepForStep, result, true);
        // на первом шаге
        const printed = [...stdout, ...stderr].join("\n");
        setOutput(printed || "Программа ничего не вывела");
        setRuntimeMessage(
            result.truncated
                ? "Трасса обрезана лимитом шагов — листайте с 1 по 1000"
                : "Трасса готова — листайте шаги стрелками ← →"
        );
    } catch (error) {
        setOutput(`Ошибка:\n${errorText(error)}`);
        setRuntimeMessage("Не удалось протрассировать — попробуйте другой код");
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady, chapterId, map]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
    if (!debug) return;
    const onKey = (e: KeyboardEvent) => {
        const target = e.target as HTMLElement | null;
        if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
        if (e.metaKey || e.ctrlKey || e.altKey) return;
        if (e.key === "ArrowRight") {
            e.stopPropagation();
            e.preventDefault();
            stepBy(1);
        }
    };
    document.addEventListener("keydown", onKey);
    return () => document.removeEventListener("keydown", onKey);
}, [debug]);

```

```

const padded = (start > 0 && !current.slice(0, start)) ? ' ' : '';
const next = current.slice(0, start) + padded + current.slice(start);
requestAnimationFrame(() => {
  ta.focus();
  ta.selectionStart = ta.selectionEnd = start + padded.length;
});
return next;
});
}, []);

```

```
const hasViz = sync.variables.length > 0;
```

```

return (
  <aside
    aria-label="Python-компилятор"
    className="fixed z-40 inset-x-0 bottom-0 h-[58dvh] flex flex-col bg-slate-900 border-t lg:border-l"
  >
    /* — Заголовок панели —————
    <div className="shrink-0 flex items-center gap-2 p-2"
      <span className={`w-2 h-2 rounded-full shrink-0`} style={{background-color: 'white'}} />
      <Terminal className="w-4 h-4 text-emerald-400" />
      <span className="text-[13px] font-bold text-white" />
      <span className="hidden sm:inline text-[10px] text-white" />
    </div>

    <div className="ml-auto flex items-center gap-1"
      <Tooltip side="bottom" content="Скопировать код"
        <button
          type="button"
          onClick={() => void copyCode()}
          className="p-1.5 rounded-lg text-slate-400"
          aria-label="Скопировать код"
        >
          {copied ? <Check className="w-4 h-4 text-emerald-400" /> : <Copy className="w-4 h-4 text-slate-400" />}
        </button>
      </Tooltip>
      <Tooltip side="bottom" content="Вернуть комментарий"
        <button

```

```

        content="Запуск = пошаговый отладчик: код в
        трация слева следует за шагами. Листайте ←
    >
    <button
      type="button"
      onClick={() => void debugRun()}
      disabled={running}
      className="ml-1 inline-flex items-center
      -60 text-white text-xs font-bold transition
    >
      {running ? <Loader2 className="w-3.5 h-3.5" /> : ""}
      {running ? "Запуск..." : "Запустить"}
    </button>
  </Tooltip>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргалка */}
<div className="shrink-0 border-b border-slate-800" />
  <div className="flex items-center gap-1 px-2 pt-2" />
    <button
      type="button"
      onClick={() => setStripTab("vars")}
      className={`inline-flex items-center gap-1.5
        stripTab === "vars" ? "bg-slate-800 text-white" : ""
      }`
    >
      <Variable className="w-3.5 h-3.5" /> Переменные
    </button>
    <button
      type="button"
      onClick={() => setStripTab("hints")}
      className={`inline-flex items-center gap-1.5
        stripTab === "hints" ? "bg-slate-800 text-white" : ""
      }`
    >
      <HelpCircle className="w-3.5 h-3.5" /> Подсказки
    </button>
  </div>

```

```

        side="bottom"
        content="В редакторе ваш код, а страниц
т заменён)."
      >
      <button
        type="button"
        onClick={resync}
        className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
      >
        <Unlink className="w-3.5 h-3.5 shrink
        Страница сменилась – подставить её пер
      </button>
    </Tooltip>
  )}
{hasViz && {
  <div className="flex flex-wrap items-centr
    {sync.variables.map((v) => {
      <Tooltip
        key={v.name}
        side="bottom"
        content={
          <span>
            <span className="block font-bol
              <code className="font-mono te
            </span>
            <span className="block">{v.role
              {v.range && <span className="bl
            </span>
          }
        }
      >
        <code
          tabIndex={0}
          className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
        >
          {v.name}
        </code>

```

```

        </div>
      )))
      <p className="text-[10px] text-slate-600">H
    </div>
  })
</div>

```

```

{ /* — Редактор + ввод —————
<div className="flex-1 min-h-0 flex flex-col">
  {debug ? (
    <div className="flex-1 min-h-[80px] flex flex
      { /* панель управления шагами */
    <div className="shrink-0 flex items-center"
      <Tooltip content="В начало трассы">
        <button type="button" onClick={() => st
          text-white hover:bg-slate-800 disabled:opac
        <SkipBack className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="Шаг назад (←)">
      <button type="button" onClick={() => st
        :text-white hover:bg-slate-800 disabled:opa
      <ChevronLeft className="w-4 h-4" />
    </button>
    </Tooltip>
    <Tooltip content="Шаг вперёд (→)">
      <button type="button" onClick={() => st
        className="p-1 rounded-md text-slate-400 hov
      <ChevronRight className="w-4 h-4" />
    </button>
    </Tooltip>
    <Tooltip content="В конец трассы">
      <button type="button" onClick={() => st
        g.result.steps.length - 1} className="p-1 r
        ansition-colors">
      <SkipForward className="w-4 h-4" />
    </button>
    </Tooltip>
  )
}

```

```

        className={`cursor-help inline-flex items-center gap-2
        stepCapable && vizLink !== "none"
        ? "border-emerald-500/40 bg-emerald-500/15 text-emerald-500
        : "border-slate-600 bg-slate-800 text-slate-400
    }`}
  >
    {stepCapable && vizLink !== "none" ? (
      {stepCapable
        ? vizLink === "exact"
        ? "демо следует (эталон)"
        : vizLink === "content"
        ? "демо следует (ваши строки)"
        : "демо отвязано"
        : "демо ручное"}
    ) : null}
  </span>
</Tooltip>
<Tooltip content="Выход из отладчика (Esc)">
  <button type="button" onClick={stopDebug}
  00/30 transition-colors">
    <Square className="w-3.5 h-3.5" />
  </button>
</Tooltip>
</div>

```

```

{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-100px">
  {debug.src.split("\n").map((line, i) => {
    const ln = i + 1;
    const active = ln === curDebug?.step?.line;
    return (
      <div
        key={i}
        data-line={ln}
        className={`flex whitespace-pre px-2 py-2
          active ? "bg-emerald-500/15 text-emerald-500
        }`}
      >
        <span

```



```

        : "border-slate-700 bg-slate-500"
      }` }
    >
    <span className={changed ? "text-indigo-400" : "text-slate-500"}>
      {name}
    </span>
    <span className="text-slate-500">
      <span className={changed ? "text-indigo-400" : "text-slate-500"}>
        {name}
      </span>
    </span>
  );
  }}}
</div>
</div>
</div>
) : showRef ? (
  <div className="flex-1 flex min-h-[70px] flex-direction-column" >
    <div className="shrink-0 flex items-center justify-center" >
      <Eye className="w-3.5 h-3.5 text-indigo-400" />
      <span className="text-[10.5px] font-bold" >
        Эталонный код этого демо • только чтение
      </span>
    </div>
    <div className="ml-auto flex items-center" >
      <button
        type="button"
        onClick={() => {
          setCode(template);
          setShowRef(false);
        }}
        className="rounded-md bg-indigo-600 h-10 text-white"
      >
        вставить в редактор
      </button>
      <button
        type="button"
        onClick={() => void copyCode()}
        className="rounded-md bg-slate-800 h-10 text-white"
      >
        копировать
      </button>
    </div>
  </div>
) : null;

```

```

        />
      </div>
    </div>

    {/* — Вывод ————— */
    <div className="shrink-0 border-t border-slate-800" >
      <button
        type="button"
        onClick={() => setOutputOpen((o) => !o)}
        className="flex items-center justify-between rounded-3xl
          colors"
      >
        <span className="inline-flex items-center gap-2" >
          <Terminal className="w-3.5 h-3.5 text-emerald-500" />
        </span>
        <span className="inline-flex items-center gap-2" >
          {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
          <span className="text-slate-600">{outputOpen ? "Закрыть" : "Открыть"}</span>
        </span>
      </button>
      {outputOpen && (
        <pre className="flex-1 min-h-[70px] overflow-hidden rounded-3xl
          leading-5 text-slate-300" >
          {output}
        </pre>
      )}
      <div className="px-3 py-1.5 border-t border-slate-800" >
        </div>
    </div>
  </aside>
);
}

```

```

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing]          = useState(fa
const [shakeEmpty, setShake]              = useState(fa
const [log, setLog]                        = useState<st

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяже
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {
    id: Date.now().toString(),
    ...preset,
    animState: 'in',
  };

  setQueue(prev => [...prev, newGuest]);
  addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

  setTimeout(() => {
    setQueue(prev => prev.map(c => c.id === newGuest.
  }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (isServing) return;

```

```

    });
    setServedHistory([]);
    setIsServing(false);
    setLog([' Столовая сброшена. Снова голодная толпа!']);
    setTimeout(() => {
      setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })), 500);
    });
  };

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-indigo-950/40 border border-indigo-500 p-6">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
          <p className="text-xs text-slate-300 mt-1 leading-tight">
            В очереди за супом всё честно. Кто первый встал, тот и ест.
            Новые гости встают строго <b>в конец</b> очереди.
            Здесь нет обхода и блата — только строгий порядок.
          </p>
        </div>
      </div>

      {/* УПРАВЛЕНИЕ */}
      <div className="flex items-center gap-3 flex-wrap">
        <button
          onClick={enqueue}
          disabled={!isServing}
          className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
            text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
            active:scale-95 transition-all cursor-pointer">
          <span> Встать в очередь (ENQUEUE)</span>
        </button>

        <button
          onClick={dequeue}

```

```

    </div>
    <div className="text-center">
      <span className="text-3xl block">    <
      <span className="text-[10px] font-bol
ase font-mono tracking-wider">
        ПОВАР
      </span>
    </div>
  </div>

```

```

  { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }

```

```

  <div className="flex flex-col items-cente
    <span className="text-2xl animate-pulse
    <span className="text-[8px] font-mono t
  </div>

```

```

  { /* СПИСОК ОЧЕРЕДИ */ }

```

```

  <div className="flex-1 flex items-end gap
    {queue.length === 0 ? {
      <div className="flex-1 flex flex-col
        <span className="text-4xl mb-1"> </
        <p className="text-xs font-bold fon
        <p className="text-[10px]">Все сыты
      </div>
    } : (
      queue.map((customer, idx) => {
        const isHead = idx === 0;
        const isTail = idx === queue.length
        return (
          <div
            key={customer.id}
            className={`
              flex flex-col items-center ga
              ${customer.animState === 'in'
              ${customer.animState === 'eat
              ${customer.animState === 'out
            `}
          >

```

```
</div>
```

```
{/* Пол кухни */}
```

```
<div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
```

```
</div>
```

```
{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
```

```
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
```

```
<div className="absolute top-4 left-4 text-[12px] font-mono">
```

```
Сытые гости (Архив FIFO)
```

```
</div>
```

```
<div className="absolute top-4 right-4 flex items-center justify-center gap-2" style={{
  order: emerald-800/80, text: [9px], font: mono}}>
```

```
<Sparkles className="w-3.5 h-3.5 animate-pulse text-emerald-400">
```

```
<span>СЫТЫ </span>
```

```
</div>
```

```
<div className="flex-1 flex flex-col justify-between p-4">
```

```
{servedHistory.length === 0 ? (
```

```
<div className="flex-1 flex flex-col items-center justify-center gap-2">
```

```
<span className="text-5xl mb-2"> </span>
```

```
<p className="text-xs font-bold font-mono">Здесь пока нет гостей
```

```
<p className="text-[10px] mt-1">Здесь пока нет гостей
```

```
</div>
```

```
) : (
```

```
<div className="flex flex-col gap-1.5 w-full">
```

```
{servedHistory.map((item, idx) => (
```

```
<div
```

```
key={idx}
```

```
className="w-full py-1.5 px-3 rounded-2xl">
```

```
<div className="flex items-center gap-2 text-[11px] text-emerald-400">
```

```
>
```

```
<span>{item}</span>
```

```
</div>
```

```
)))
```

```
</div>
```

```
})
```

```

    fail: number;
    term_link: number;
    is_terminal: boolean;
    words: string[];
    depth: number;
    x?: number;
    y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([]
  const [currentNode, setCurrentNode] = useState<number
  const [speechText, setSpeechText] = useState<string>('
    'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
      буль!')
  );
  const [isTalking, setIsTalking] = useState<boolean>(f
  const [isBlinking, setIsBlinking] = useState<boolean>

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

  // Эффект разговора при смене текста
  useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false),
    return () => clearTimeout(timer);
  }, [speechText]);

```

```

const layoutDFS = (u: number, depth: number) => {
  const childKeys = Object.values(newNodes[u].children);
  newNodes[u].y = 40 + depth * paddingY;

  if (childKeys.length === 0) {
    newNodes[u].x = 40 + currentX * paddingX;
    currentX++;
  } else {
    let leftX = -1;
    let rightX = -1;
    childKeys.forEach(v => {
      layoutDFS(v, depth + 1);
      if (leftX === -1) leftX = newNodes[v].x!;
      rightX = newNodes[v].x!;
    });
    newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
    if (leftX === -1) currentX++;
  }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
  `Отлично! Я построил базовое префиксное дерево (БПД) для слова "S", чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {

```



```

        оче суффикса просто нет. Жми «Следующий шаг»!'
    });
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полный  

            ь полюбоваться таблицей переходов!'
        );
        return;
    }

    const r = bfsQueue[0];
    const newQueue = bfsQueue.slice(1);
    const updatedNodes = { ...nodes };
    const rNode = updatedNodes[r];
    const childrenIds = Object.values(rNode.children);

    let logs = `Рассматриваем вершину #${r} (${rNode.ch

    for (const [char, u] of Object.entries(rNode.children)) {
        newQueue.push(u);
        let v = rNode.fail;
        while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
            v = updatedNodes[v].fail;
        }
        const failTarget = updatedNodes[v].children[char];
        updatedNodes[u].fail = failTarget;

        // Рассчитываем терминальную ссылку (term_link)
        if (updatedNodes[failTarget].is_terminal) {
            updatedNodes[u].term_link = failTarget;
        } else {
            updatedNodes[u].term_link = updatedNodes[failTa

```

```

        </button>
    </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
    {/* Аватар Лосося (Сеня) с анимацией моргания и
    <div className="flex flex-col items-center just
        <div className="w-40 h-40 relative flex items
            full border border-blue-500/30 shadow-inner
            {/* SVG Лосося */}
            <svg
                viewBox="0 0 200 200"
                className={`w-36 h-36 transition-transform
                    isTalking ? 'scale-105 drop-shadow-[0_0_
                ]`}
            >
                {/* Тело лосося */}
                <ellipse cx="100" cy="100" rx="70" ry="45"
                <path d="M 40 85 Q 100 55 160 85 Q 100 115
                    40 85" fill="#e67e22" stroke="black" stroke-
                width="2"/>
                {/* Хвост */}
                <path
                    d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
                    fill="#be123c"
                    className="origin-[35px_100px] animate-
                />
                {/* Плавники */}
                <path d="M 90 55 L 70 30 L 110 55 Z" fill="#1abc9c"
                <path d="M 90 145 L 70 170 L 110 145 Z" fill="#1abc9c"
                <path
                    d="M 120 115 L 100 135 L 130 130 Z"
                    fill="#f87171"
                    className="origin-[120px_115px] animate-
                />
                {/* Глаз (с морганием) */}

```

```
<div className="flex items-center space-x-2">
  <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
  <span>Прямое вещание из аквариума:</span>
</div>
```

```
<p className="text-slate-200 text-base leading-relaxed">
  {speechText}
</p>
```

```
<div className="mt-4 pt-3 border-t border-slate-200">
  <div className="text-xs text-slate-400 flex justify-between">
    <Info className="w-3.5 h-3.5 text-blue-400" />
    Статус: {simulationStep === 0 ? 'Борется с вирусом!' : 'Восстановился!'}
  </div>
</div>
```

```
<div className="flex gap-2">
  {simulationStep === 0 && (
    <button
      onClick={startBFS}
      className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
    >
      <Play className="w-4 h-4" /> Запустить симуляцию
    </button>
  )}
  {simulationStep === 1 && (
    <button
      onClick={stepBFS}
      className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
    >
      <span>Шаг BFS</span>
      <span className="bg-blue-800 px-1.5 py-0.5 text-white text-xs font-medium">
        {currentStep}
      </span>
    </button>
  )}
  {simulationStep === 2 && (
    <button
      onClick={resetBFS}
      className="bg-red-600 hover:bg-red-500 text-white px-4 py-2 shadow-lg shadow-red-900/40 transition-colors"
    >
      <span>Сброс</span>
    </button>
  )}
</div>
```

```

    { /* Draw Edges */ }
    { Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x || !node.y) return null;

      const isCurrent = currentNode === node.id;

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
            x2={node.x}
            y2={node.y - 16}
            stroke={isCurrent ? '#60a5fa' : '#ccc'}
            strokeWidth="2"
            markerEnd="url(#arrowhead)"
          />
          <text
            x={(parent.x + node.x) / 2 - 10}
            y={(parent.y + node.y) / 2}
            fill="#94a3b8"
            fontSize="12"
            fontWeight="bold"
          >
            {node.char}
          </text>
        </g>
      );
    }) }

    { /* Draw Fail Links */ }
    { simulationStep > 0 && Object.values(nodes).map(node => {
      if (node.id === 0 || node.fail === 0) return null;
      // n draw all! But maybe wait, we can just draw fail link.
      // Wait, all nodes get fail link. Let's
    }) }
  
```

```

    } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
    }

    return (
        <g key={`node-${node.id}`}>
            <circle
                cx={node.x}
                cy={node.y}
                r="16"
                fill={fill}
                stroke={stroke}
                strokeWidth="2"
            />
            <text
                x={node.x}
                y={node.y + 4}
                fill="white"
                fontSize={isRoot ? "10" : "12"}
                fontWeight="bold"
                textAnchor="middle"
            >
                {isRoot ? 'R' : node.char}
            </text>
            {!isRoot && (
                <text
                    x={node.x + 12}
                    y={node.y - 12}
                    fill="#94a3b8"
                    fontSize="9"
                >
                    {node.id}
                </text>
            )}
        </g>
    );
})}

```

```

        maxLength={8}
        className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
      />
      <button
        type="submit"
        className="bg-slate-700 hover:bg-slate-600 transition-colors"
      >
        <Plus className="w-3.5 h-3.5" /> Добавить
      </button>
    </form>
  </div>

```

```

  { /* Таблица Вершин и суффиксных ссылок */ }
  <div className="lg:col-span-2 bg-slate-900/50 p-4" >
    <h5 className="text-white font-bold mb-3 flex" >
      <span className="flex items-center gap-1.5" >
        <span> </span> Таблица переходов и fail-ссылок
      </span>
      <span className="text-xs text-slate-400 font-mono" >
        Всего вершин: {Object.keys(nodes).length}
      </span>
    </h5>

```

```

    <div className="max-h-[220px] overflow-y-auto" >
      <table className="w-full text-left border-collapse" >
        <thead>
          <tr className="border-b border-slate-700" >
            <th className="py-2 px-3" >ID Вершины</th>
            <th className="py-2 px-3" >Символ</th>
            <th className="py-2 px-3" >Родитель</th>
            <th className="py-2 px-3" >fail-ссылка</th>
            <th className="py-2 px-3" >term_link</th>
            <th className="py-2 px-3" >Дети</th>
            <th className="py-2 px-3" >Слова (dict)
          </tr>
        </thead>

```

```

</td>
<td className="py-2.5 px-3">
  {node.id === 0 ? (
    <span className="text-slate-500">
      {node.fail}
    </span>
  ) : (
    <span className="bg-indigo-950 text-white">
      {node.fail}
    </span>
  )}
</td>
<td className="py-2.5 px-3">
  {node.id === 0 || node.term_link ? (
    <span className="text-slate-500">
      {node.term_link}
    </span>
  ) : (
    <span className="bg-rose-950 text-white">
      {node.term_link}
    </span>
  )}
</td>
<td className="py-2.5 px-3 text-slate-500">
  {Object.keys(node.children).length === 0 ? 'Ø' : Object.entries(node.children)
    .map(([c, id]) => `${c}→#id`)
    .join(', ')}
</td>
<td className="py-2.5 px-3">
  {node.words.length === 0 ? (
    <span className="text-slate-600">
      {node.words}
    </span>
  ) : (
    <span className="bg-emerald-900 text-white">
      {node.words.join(', ')}
    </span>
  )}
</td>
</tr>
);

```

```

    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};

  const CODE = [
    "build(v, l, r) {", // 1
    "  if (l === r) {", // 2
    "    tree[v] = A[l]; return", // 3
    "  }", // 4
    "  mid = (l + r) >> 1", // 5
    "  build(2v, l, mid)", // 6
    "  build(2v+1, mid+1, r)", // 7
    "  tree[v] = tree[2v]+tree[2v+1]", // 8
    "}", // 9
  ];
  void CODE;
}

```

```

function go(v: number, l: number, r: number) {
  if (l === r) {
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
      , highlightNodes: [v], arrayHighlight: [l, r] });
    tree[v] = arr[l];
    steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
      , highlightNodes: [v], arrayHighlight: [l, r] });
    return;
  }
}

```



```

        codeLine: 6, treeState: tree, highlightNodes: [v
const left = go(2 * v, l, m, qL, qR);
const right = go(2 * v + 1, m + 1, r, qL, qR);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}:
    rgeChildren: [2 * v, 2 * v + 1], arrayHighlight:
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${q
    ghlightNodes: [1], arrayHighlight: [qL, qR], result
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree

function genQueryBottomUpSteps(arr: number[], qL: number

```

    const n = arr.length;

```

```

// Build iterative tree: leaves at [n..2n-1]

```

```

const tree: Record<number, number> = {};

```

```

for (let i = 0; i < n; i++) tree[n + i] = arr[i];

```

```

for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i

```

```

const steps: Step[] = [];

```

```

let id = 0;

```

```

let l = qL + n;

```

```

let r = qR + n + 1; // half-open [l, r)

```

```

let res = 0;

```

```

steps.push({ id: id++, desc: `Старт: l = qL + n = ${q
    `, codeLine: 3, treeState: tree, highlightNodes: [l

```

```

while (l < r) {

```

```

    const highlights: number[] = [];

```

```

    if (l & 1) {

```

```

        res += tree[l] ?? 0;

```

```

        steps.push({ id: id++, desc: `l=${l} нечётный → l
            0}. Сумма = ${res}. l++.`, codeLine: 5, treeSta

```

```

        highlights.push(l);
    }
}

```

```

    "  build(2*v, l, mid);",
    "  build(2*v+1, mid+1, r);",
    "  tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
  ];
const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
];
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
  "    if (l & 1) res += tree[l++];",
  "    // -----",
  "    if (r & 1) res += tree[--r];",
  "    l >>= 1; r >>= 1;",
  "  }",
  "  return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);

  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

  useEffect(() => {

```



```

        = {step.result}
      </span>
    })
  </div>

  {/* Controls */}
  <div className="px-3 py-2.5 bg-slate-950 border-t-1 border-slate-800" >
    <div className="flex items-center bg-slate-900" >
      <button onClick={() => { setPlaying(false); setSpeed(1200); }}
        className="p-1 rounded" title="Сброс"><RotateCcw className="w-4 h-4" />
      <button onClick={() => { setPlaying(false); setSpeed(700); }}
        className="p-1 rounded" title="Норма"><Play className="w-4 h-4" />
      <button onClick={() => setPlaying(!playing)}
        className="p-1 rounded" title={playing ? "Пауза" : "Воспроизведение"}>
        {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
      </button>
      <button onClick={() => { setPlaying(false); setSpeed(120); }}
        className="p-1 rounded" title="Быстро"><RotateCcw className="w-4 h-4" />
      </button>
    </div>
    <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
      className="w-full text-[9px] font-bold rounded px-1.5 py-1 outline-1 outline-slate-800" >
      <option value={1200}>Медл.</option>
      <option value={700}>Норма</option>
      <option value={350}>Быстро</option>
      <option value={120}>Турбо</option>
    </select>
  </div>

  {/* Progress bar */}
  <div className="h-1 bg-slate-950"><div className="bg-slate-800" style={{width:
    ((currentStep + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT

```

```

<div className="mb-6 space-y-4">
  <div className="flex flex-col lg:flex-row gap-4">
    {/* Array input */}
    <form onSubmit={handleApply} className="flex">
      <div className="flex items-center justify-between">
        <label className="text-[10px] font-bold uppercase">
          label>
        <button type="button" onClick={randomize}>
          "><RefreshCw className="w-3.5 h-3.5" /></button>
        </div>
        <div className="flex gap-2">
          <input
            value={customInput}
            onChange={e => setCustomInput(e.target.value)}
            className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
            placeholder="5, 8, 3, 12, 7, 2"
          />
          <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
            Применить</button>
        </div>
      </form>
      <div className="flex flex-wrap gap-1.5 pt-1">
        {arr.map((v, i) => (
          <span key={i} className="bg-slate-900 border border-indigo-500 padding-2">
            <span className="text-slate-500">A[{i}]</span>
            </span>
        ))}
        <span className="text-xs text-slate-500 font-weight-normal">
          </div>
      </div>
    {/* Query range */}
    <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500">
      <label className="text-[10px] font-bold uppercase">
        Query range</label>
      <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400">From</span>
          <input
            type="text"
            value={queryRangeFrom}
            onChange={e => setQueryRangeFrom(e.target.value)}
            className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
          />
        </div>
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400">To</span>
          <input
            type="text"
            value={queryRangeTo}
            onChange={e => setQueryRangeTo(e.target.value)}
            className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
          />
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        er:text-slate-200'}`}>
        <Search className="w-3.5 h-3.5" /> Запрос: св
    </button>
    <button onClick={() => setView('theory')} class=
        n-colors ${view === 'theory' ? 'border-blue-5
        te-200'}`}>
        <Info className="w-3.5 h-3.5" /> Почему / Спра
    </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
    <SimPanel
        key={`build-${arr.join('-')}`}
        title="Построение дерева (рекурсия)"
        icon=" "
        steps={buildSteps}
        code={BUILD_CODE}
        color="indigo"
        arr={arr}
    />
)}

{view === 'queries' && (
    <div className="grid grid-cols-1 xl:grid-cols-2"
        <SimPanel
            key={`qtd-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Сверху`
            icon="↑ "
            steps={queryTDSteps}
            code={QUERY_TD_CODE}
            color="emerald"
            arr={arr}
        />
        <SimPanel
            key={`qbu-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Снизу-н
            icon="↑ "

```

```

        <p className="text-xs text-slate-300 mb-3">
            Начинаем с корня. Если отрезок узла полн
        наче спускаемся в обоих детей.
        </p>
        <div className="space-y-1.5 text-xs">
            <div className="flex justify-between bo
            <span className="text-rose-400 font-mono">0(
            <div className="flex justify-between bo
            ><span className="text-rose-400 font-mono">
            <div className="flex justify-between"><
            d-400 font-bold">✓ легко</span></div>
            </div>
        </div>
        <div className="bg-slate-950 p-5 rounded-xl
            <div className="absolute -top-3 left-4 bg
            rder border-amber-500 shadow-md">
                † Запрос Снизу-вверх (Итерация)
            </div>
            <p className="text-xs text-slate-300 mb-3">
                Стартуем с двух указателей (l и r) на л
                ём его левого соседа. Поднимаемся пока l {"
            </p>
            <div className="space-y-1.5 text-xs">
                <div className="flex justify-between bo
                <span className="text-emerald-400 font-mono":
                <div className="flex justify-between bo
                ><span className="text-emerald-400 font-mono
                <div className="flex justify-between"><
                00 font-bold">x сложно</span></div>
                </div>
            </div>
        </div>
    </div>

    {/* Verdict */}
    <div className="bg-slate-950 p-5 rounded-xl b
        <h4 className="font-bold text-blue-400 mb-2
        <p className="text-sm text-slate-300 leadin
            <strong className="text-white">Итеративны

```

```

    (map.get(key) as typeof chapters).push(c);
  }
  return Array.from(map.entries());
}, [query]);

return (
  <aside className="w-full bg-slate-900/80 lg:bg-tran
    <div className="p-4 pb-3 shrink-0">
      <h3 className="text-xs font-bold text-slate-400">
        <Compass className="w-4 h-4 text-indigo-400" />
      </h3>
      <div className="relative">
        <Search className="w-4 h-4 text-slate-500 abs
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Поиск по темам.."
            className="w-full bg-slate-800/70 border bo
            500 focus:outline-none focus:border-indigo-!
          />
          {query && (
            <button
              onClick={() => setQuery("")}
              className="absolute right-2 top-1/2 -tran
              aria-label="Очистить поиск"
            >
              <X className="w-3.5 h-3.5" />
            </button>
          )}
        </div>
      </div>

      <div className="flex-1 overflow-y-auto px-3 pb-4">
        {groups.length === 0 && <p className="text-sm t

        {groups.map(([category, items]) => (
          <div key={category}>
            <div className="text-[10px] font-bold upperc

```



```
    <div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-500 to-purple-500 p-3 text-white" style={styles.container}>
      <div className="font-bold text-indigo-400 flex items-center justify-between" style={styles.header}>
        <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
      </div>
      <p className="leading-relaxed">
        Подчёркнутые термины в тексте раскрываются по клику на стрелку.
      </p>
      <p className="text-slate-400 italic">Стрелки ← и → указывают на соответствующие элементы симулятора.
      </p>
    </div>
  </aside>
);
};
```

ФАЙЛ 63/87: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';
```

```
interface Plate {
  id: string;
  name: string;
  icon: string;
}
```

```
interface Person {
  id: string;
  name: string;
  icon: string;
}
```

```
interface Hypothesis {
  id: string;
```

```

    { name: 'Чашка от кофе', icon: '☕' },
    { name: 'Миска с остатками салата', icon: '🥗' },
    { name: 'Тарелка от тако', icon: '🌮' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name} ${newPlate.icon}`);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon}`);
};

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюдец от торта', icon: '🍰' },
  ]);
  setPopMessage("Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
  const presets = [
    { name: 'Любитель борща', icon: '🥘' },
    { name: 'Курьер доставки', icon: '🚚' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPerson = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue(prev => [...prev, newPerson]);
  setAddMessage(`Добавили в очередь: ${newPerson.name} ${newPerson.icon}`);
};

const removePerson = () => {
  if (queue.length === 0) {
    setAddMessage("🚫 Очередь пуста!");
    return;
  }
  const frontPerson = queue[0];
  setQueue(prev => prev.slice(1));
  setRemoveMessage(`Удален из очереди: ${frontPerson.name} ${frontPerson.icon}`);
};

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Повар', icon: '👨‍🍳' },
    { id: '2', name: 'Помощник повара', icon: '👩‍🍳' },
    { id: '3', name: 'Помощник курьера', icon: '👨‍🚚' },
  ]);
  setAddMessage("Очередь восстановлена.");
};

```

```

        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    });
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(`    Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
    if (heap.length === 0) {
        setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
        return;
    }
    // Find highest probability
    let maxIdx = 0;
    for (let i = 1; i < heap.length; i++) {
        if (heap[i].probability > heap[maxIdx].probability) {
            maxIdx = i;
        }
    }
    const best = heap[maxIdx];
    setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
    setHeapMessage(`    Выбран самый "тяжелый" кандидат:`);
};

const resetHeap = () => {
    setHeap([
        { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.3 },
        { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', probability: 0.2 },
        { id: '3', text: 'Кандидат: "Привет, я испанский язык"', probability: 0.1 }
    ]);
    setHeapMessage("    Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return {

```

```

        : 'bg-slate-800/60 border-slate-700/60 text-white'
      }`}
    >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'fifo' ? 'bg-indigo-500/20 border-indigo-500/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">FIFO
          <div className="text-xs opacity-80">FIFO
        </div>
      </div>
      <span className="text-xs bg-indigo-500/20 text-white">
        BFS
      </span>
    </button>

    <button
      onClick={() => setActiveTab('heap')}
      className={`flex items-center justify-between gap-3 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
    >
    <div className="flex items-center gap-3">
      <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-500/20 border-emerald-500/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">Приоритетная
          <div className="text-xs opacity-80">Приоритетная
        </div>
      </div>
      <span className="text-xs bg-emerald-500/20 text-white">
        Beam Search
      </span>
    </button>
  </div>

  {/* STACK TAB */}
  {activeTab === 'stack' && (

```

```

    {popMessage && {
      <div className="bg-blue-950/60 border border-
imate-fade-in">
        <span className="inline-block w-2 h-2 round
        {popMessage}
      </div>
    }}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
  >
    {stack.length === 0 ? {
      <div className="text-center py-12 text-sla
        <div className="text-5xl mb-3"> </div>
        <p className="text-base font-bold">Стор
        <p className="text-xs mt-1">Добавьте гр
      </div>
    } : {
      <div className="w-full max-w-sm flex flex
        <div className="absolute top-0 text-xs
full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map(({plate, ind
        const isTop = index === 0;
        return {
          <div
            key={plate.id}
            className={`w-full flex items-cen
              isTop
                ? 'bg-gradient-to-r from-blue
le-102'
                : 'bg-slate-900/90 border-sla
            }` }
          >

```

```

<div className="flex flex-wrap items-center"
  <button
    onClick={addPerson}
    className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
  >
    <Plus className="w-4 h-4" /> Встать в о
  </button>
  <button
    onClick={servePerson}
    className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
  >
    Налить суп первому
  </button>
  <button
    onClick={resetQueue}
    title="Сбросить"
    className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
  >
    <RotateCcw className="w-5 h-5" />
  </button>
</div>
</div>

```

```

{dequeueMessage && {
  <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
    <span className="inline-block w-2 h-2 roun
    {dequeueMessage}
  </div>
}}

```

```

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
  {queue.length === 0 ? {

```

```

        </span>
        <span className={`text-[10px] font-
          isFirst ? 'bg-indigo-500/30 text-
        }`}>
          {isFirst ? 'Получает сун' : `В
        </span>
      </div>
    );
  }}}

  <div className="flex flex-col items-cent
    -600 min-w-[120px] h-[120px]">
    <Plus className="w-6 h-6 mb-1" />
    <span className="text-xs font-mono up
  </div>
</div>
}}
<div className="mt-6 text-xs text-slate-500
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
}}
```

```

{/* HEAP TAB */}
{activeTab === 'heap' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Приоритетная очередь / Куча</sp
          <span className="text-xs font-mono bg-e
        x-Heap</span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Здесь не важно, кто пришел первым. Важн
        </p>
      </div>
    </div>
  }
}
```

```

        <input
          type="range"
          min="0.01"
          max="0.99"
          step="0.01"
          value={newCustomProb}
          onChange={e => setNewCustomProb(Number(e.target.value))}
          className="w-full accent-emerald-500 cursor-pointer"
        />
      </div>
      <div className="md:col-span-3">
        <button
          type="submit"
          disabled={!newCustomText.trim()}
          className="w-full flex items-center justify-center text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
        >
          <Plus className="w-4 h-4" /> Добавить в список
        </button>
      </div>
    </form>

    {heapMessage && (
      <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">
        <span className="inline-block w-2 h-2 rounded-full bg-emerald-500/40" />
        {heapMessage}
      </div>
    )}

    {/** Visualization */}
    <div className="bg-slate-950/60 p-6 rounded-2xl">
      <div>
        {sortedHeap.length === 0 ? (
          <div className="text-center py-12 text-slate-400">
            <div className="text-5xl mb-3">0</div>
            <p className="text-base font-bold">Куча пуста

```



```
        </div>
      </div>

      <div className="flex items-center" >
        { /* Custom progress bar */ }
        <div className="hidden sm:block" >
          <div
            className={`h-full rounded-
              style={{ width: `${percent}%`
            ></div>
          </div>
          <span className={`font-mono font-
            isTop ? 'text-emerald-400 text-
          }`}>
            {percent}%
          </span>
        </div>
      </div>
    );
  }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500" >
  НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```

```

return (
  <div>
    <div className="mb-6">
      <h2 className="text-xl sm:text-2xl font-extrabo
      <p className="text-sm text-slate-400 leading-re
        Все интерактивные демонстрации в одном месте.
        здесь их можно открыть напрямую.
      </p>
    </div>

    <div className="relative mb-6 max-w-md">
      <Search className="w-4 h-4 text-slate-500 absol
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Поиск тренажёра..."
        className="w-full bg-slate-900 border border-
          focus:outline-none focus:border-indigo-500
      />
      {query && (
        <button
          onClick={() => setQuery("")}
          className="absolute right-2 top-1/2 -transl
          aria-label="Очистить поиск"
        >
          <X className="w-4 h-4" />
        </button>
      )}
    </div>

    {items.length === 0 && <p className="text-slate-5

    <div className="grid gap-3 sm:grid-cols-2 xl:grid
      {items.map(({ id, entry, chapterTitle }) => (
        <div
          key={id}
          className="flex flex-col bg-slate-900 borde
        >

```

import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U

/**

* Песочница поворотов.

*

* Здесь ничего не подсказывается: дано настоящее дерев

* поднять отмеченный узел в корень. Клик по узлу = оди

* Порядок поворотов выбираешь сам; правильный ли он по

* только после того, как узел окажется наверху (или по

*/

export interface TNode {

key: number;

left: TNode | null;

right: TNode | null;

}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>

t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос

export const PRESETS: { name: string; hintCase: string;

{

name: "Линия влево",

hintCase: "Zig-Zig",

target: 1,

build: () => {

const n1 = leaf(1);

const n2: TNode = { key: 2, left: n1, right: leaf

const n4: TNode = { key: 4, left: n2, right: leaf

return { key: 6, left: n4, right: leaf(7) };

},

},

{

name: "Змейка",

hintCase: "Zig-Zag",

```
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
```

```
export function rotateUp(root: TNode, key: number): TNode {
```

```
  const path = findPath(root, key);
```

```
  if (path.length < 2) return root;
```

```
  const x = path[path.length - 1];
```

```
  const p = path[path.length - 2];
```

```
  const gg = path.length >= 3 ? path[path.length - 3] : null;
```

```
  if (p.left === x) {
```

```
    p.left = x.right;
```

```
    x.right = p;
```

```
  } else {
```

```
    p.right = x.left;
```

```
    x.left = p;
```

```
  }
```

```
  if (!gg) return x;
```

```
  if (gg.left === p) gg.left = x;
```

```
  else gg.right = x;
```

```
  return root;
```

```
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
```

```
export function canonicalNext(root: TNode, key: number): TNode {
```

```
  const path = findPath(root, key);
```

```
  if (path.length < 2) return null;
```

```
  const x = path[path.length - 1];
```

```
  const p = path[path.length - 2];
```

```
  const g = path.length >= 3 ? path[path.length - 3] : null;
```

```
  if (!g) return { node: x.key, kind: "Zig" };
```

```
  const xLeft = p.left === x;
```

```
  const pLeft = g.left === p;
```

```
  if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
```

```

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.root);
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() => preset.graph(), [preset]);
  const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
  const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

  const clickNode = (key: number) => {
    if (solved) return;
    const p = findPath(tree, key);
    if (p.length < 2) return; // корень крутить не с кем

    const want = canonicalNext(tree, target);
    setHistory((h) => [...h, clone(tree) as TNode]);
    setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
  };
};

```

```

        className={`px-3 py-1.5 rounded-lg text-[11pt]
          i === presetIdx ? "bg-indigo-600 text-white" : "text-gray-500"
        }`
      >
        {p.name}
      </button>
    )))
  </div>

```

```

<div className="bg-slate-900 rounded-xl border border-gray-700" style={{ width: 400px, height: 400px }}>
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420), height: 400px }}
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
        />
      );
    })}

    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;

```

```

        onClick={() => reset()}
        className="flex items-center gap-1.5 px-3 py-3 border rounded-lg text-slate-500 font-bold transition-colors"
      >
        <RotateCcw className="w-3.5 h-3.5" /> Заново
      </button>
      <button
        onClick={() => setShowAnswer((s) => !s)}
        className={`flex items-center gap-1.5 px-3 py-3 border rounded-lg text-slate-500 font-bold transition-colors
          ${showAnswer
            ? "bg-amber-600/20 border-amber-500 text-amber-500"
            : "bg-slate-800 border-slate-700 text-slate-500"}
        `}
      >
        {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор" />
      </button>

      <span className="text-[11px] text-slate-500 ml-2">
        поворотов: {moves.length} • минимум: {minimalMoves}
      </span>
    </div>

    {showAnswer && !solved && expected && (
      <div className="mt-3 text-[12px] bg-amber-950/40 border rounded-lg padding-3">
        Сейчас splay крутил бы узел{" "}
        <span className="font-mono font-bold">{expected}
      </div>
    )}

    {solved && (
      <div
        className={`mt-3 rounded-lg border px-3 py-2.5 text-slate-500 font-medium
          ${wrong === 0
            ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
            : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
        `}
      >

```

```
* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
*
* Что сделано ради понятности:
*   * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
*   * сразу видно, что дерево остаётся деревом поиска;
*   * под деревом печатается обход слева направо: он ОДНОЭТАПНЫЙ;
*   * это и есть доказательство, что поворот ничего не ломает;
*   * кадр «прицел» ничего не двигает, только подсвечивает;
*   * на кадре «результат» остаются призраки – пунктирные узлы в
*     местах и стрелки «откуда → куда»;
*   * по умолчанию анимация НЕ крутится сама: пользователь
*     идёт в своём темпе. Автопрокрутка включается кнопкой.
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
}
```



```

        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [160, 50], A: [80, 130], p: [245, 130],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 – корень",
        text: "40 стало правым сыном двадцатки. Поддерев
            ева от 40 – это одно и то же место.",
        moved: ["x", "p", "B"],
        ms: RESULT_MS,
    },
    ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
    edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
    pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
    edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",
    when: "x и p – дети с ОДНОЙ стороны (оба слева или об
    rotations: "2 поворота, первым – верхний",
    keys: { x: 20, p: 40, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZZ_S0,

```

```

        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
    key: "zig-zag",
    label: "Zig-Zag",
    when: "x и p – дети с РАЗНЫХ сторон (змейка)",
    rotations: "2 поворота, первым – нижний",
    keys: { x: 40, p: 20, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZG_S0,
            kind: "start",
            title: "Было: 60 → влево на 20 → вправо на 40",
            text: "Узлы идут «змейкой»: сначала шаг влево, по
                ",
            ms: RESULT_MS,
        },
        {
            ...ZG_S0,
            kind: "aim",
            title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",

```

```

g: { fill: "#f59e0b", stroke: "#fcd34d" },
sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" re
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>

      /* призраки: где узел стоял до поворота */
      {ghosts.map((id) => {
        const from = prev[id]!;
        const to = pos[id]!;
        const r = isSubtree(id) ? 15 : 20;
        const dx = to[0] - from[0];
        const dy = to[1] - from[1];
        const len = Math.hypot(dx, dy) || 1;
        const pad = r + 8;
        return (
          <g key={`ghost-${id}`} opacity={0.55}>
            <circle cx={from[0]} cy={from[1]} r={r} fil
              {len > pad * 2 + 6 && (

```

```

const sub = isSubtree(id);
const c = COLOR[sub ? "sub" : id];
const r = sub ? 15 : 20;
const hot = Boolean(focus && focus.includes(id));
return (
  <g
    key={id}
    style={{
      transform: `translate(${p[0]}px, ${p[1]}px)`,
      transition: MOVE,
      opacity: dim(id) ? 0.28 : 1,
    }}
  >
    {hot && <circle r={r + 7} fill="none" stroke="red" />}
    <circle r={r} fill={c.fill} stroke={c.stroke} />
    <text
      textAnchor="middle"
      dominantBaseline="central"
      fontSize={sub ? 12 : 15}
      fontWeight="700"
      fill={sub ? "#94a3b8" : "#fff"}
    >
      {sub ? id : keys[id]}
    </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font-size={10}
        >{id}</text>
    )}
  </g>
);
}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);

```

```

<div className="mb-3 text-xs sm:text-[13px] text-
  <span className="font-bold text-indigo-300">Кор,
  <span className="text-slate-500">({active.rotat
</div>

```

/* шаг + пояснение стоят НАД картинкой: сначала

```

<div className="mb-3 rounded-xl border border-sla
  <div className="flex items-center gap-2 mb-1 fl
    <span
      className={`text-[10px] font-bold uppercase
        current.kind === "aim"
          ? "bg-amber-500/20 text-amber-300"
          : current.kind === "start"
            ? "bg-slate-700 text-slate-300"
            : "bg-indigo-500/20 text-indigo-300"
        }`
    >
      {current.kind === "aim" ? "прицел" : current
    </span>
    <span className="text-sm font-bold text-white
      Шаг {idx + 1} из {total}. {current.title}
    </span>
  </div>
  <p className="text-[13px] text-slate-400 leadin
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
  <Tree frame={current} prev={prevPos} keys={acti

```

/* обход слева направо – одинаковый на всех ка

```

<div className="mt-2 pt-2 border-t border-slate
  <div className="flex items-center gap-1.5 fle
    <span className="text-slate-500 mr-1">Обход
    {active.inorder.map((t, i) => (
      <span
        key={i}
        className={`px-1.5 py-0.5 rounded font-r
          /\d/.test(t) ? "bg-emerald-500/15 tex

```

```

        setPlaying(false);
        setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600
      w-indigo-900/40"
  >
    {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
  onClick={() => setPlaying((p) => !p)}
  className="flex items-center gap-1.5 px-3 py-1
    ext-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" />
      {playing ? "Стоп" : "Авто"}}
</button>
<button
  onClick={() => setSlow((s) => !s)}
  title="Автопрокрутка ещё медленнее"
  className={`flex items-center gap-1.5 px-3 py-1
    slow
      ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
      : "bg-slate-800 border-slate-700 text-slate-200"}
  >
    <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5" : "1"}
</button>
<button
  onClick={() => {
    setFrame(0);
    setPlaying(false);
  }}
  className="flex items-center gap-1.5 px-3 py-1
    ext-xs font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сброс
</button>

```

```
// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};
```

```
// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
  x.left = y.right;
  y.right = x;
  return y;
};
```

```
// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};
```

```
// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
  };
};
```

```

while (current) {
  lastNode = current;
  if (key === current.key) {
    path.push({ node: current, dir: null });
    break;
  } else if (key < current.key) {
    path.push({ node: current, dir: "left" });
    current = current.left;
  } else {
    path.push({ node: current, dir: "right" });
    current = current.right;
  }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
  steps.push(`Поиск ${key}: элемент отсутствует в д...`);
  steps.push(
    `Поиск споткнулся на узле [${lastNode.key}]. И...`
  );
} else {
  steps.push(
    `Поиск ${key}: элемент найден! Запускаем подъем`
  );
}

// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,
): SplayNode | null => {
  if (!node || node.key === k) return node;

  if (k < node.key) {
    if (!node.left) return node;

```



```

    }

    if (!node.right) return node;
    steps.push(
      `Финальный поворот Zag (Левое вращение корня)`
    );
    return leftRotate(node);
  }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
  setHighlightedNode(key);
  const { newRoot, steps } = splayStepByStep(tree, key);
  if (newRoot) {
    setTree(newRoot);
  }
  setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;

  // Standard BST insert, then splay!
  let newTree = cloneTree(tree);
  newTree = insertBST(newTree, val);
  const { newRoot, steps } = splayStepByStep(newTree, val);
  if (newRoot) {
    setTree(newRoot);
  }
  setLog([`Вставили вершину [${val}].`, ...steps]);
  setInputValue("");
};

```

```
// Render lines and nodes
const renderLines = (node: SplayNode | null): React.ReactNode[] => {
  if (!node) return [];
  let lines: React.ReactNode[] = [];
  if (node.left && node.left.x && node.left.y) {
    lines.push(
      <line
        key={`l-${node.key}-${node.left.key}`}
        x1={node.x}
        y1={node.y}
        x2={node.left.x}
        y2={node.left.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.left));
  }
  if (node.right && node.right.x && node.right.y) {
    lines.push(
      <line
        key={`r-${node.key}-${node.right.key}`}
        x1={node.x}
        y1={node.y}
        x2={node.right.x}
        y2={node.right.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.right));
  }
  return lines;
};
```

```
const renderNodes = (node: SplayNode | null): React.ReactNode[] => {
  if (!node) return [];
  let circles: React.ReactNode[] = [];
```

```

<div className="bg-slate-900 rounded-xl border border-
  {/* Header */}
  <div className="bg-slate-800 p-4 border-b border-
    <h3 className="font-bold text-white flex items-
      <BookOpen className="w-5 h-5 text-indigo-400"
      Интерактивное Splay-дерево
    </h3>
    <p className="text-xs text-slate-400">
      Кликните на вершины дерева, чтобы «вытащить»
      Splay.
    </p>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-12
    {/* Playfield */}
    <div className="lg:col-span-7 bg-[#0f172a] p-4
      <svg
        className="w-full h-full max-w-[600px] max-h-
        viewBox="0 0 600 320"
      >
        {renderLines(drawTree)}
        {renderNodes(drawTree)}
      </svg>

      {/* Quick interactive controls */}
      <div className="absolute bottom-4 left-4 right-
        <form onSubmit={handleFind} className="flex
          <input
            type="number"
            placeholder="Поиск..."
            value={inputValue}
            onChange={(e) => setInputValue(e.target
            className="w-24 bg-slate-950 text-white
            00 focus:outline-none"
          />
          <button
            type="submit"
            onClick={handleFind}

```

```

<h4 className="text-xs font-bold text-slate-400">
  Ход вращения (Splay-логи)
</h4>
<div className="flex-1 space-y-2 mb-4 overflow-hidden">
  {log.map((step, idx) => (
    <div
      key={idx}
      className={`text-xs p-2.5 rounded transition-colors
        ${idx === log.length - 1
          ? "border-l-2 border-indigo-500 bg-indigo-50"
          : "text-slate-400 bg-slate-900/50"}
      `}
    >
      {step}
    </div>
  ))}
</div>

<div className="border-t border-slate-800 pt-10">
  <p>
    <span className="font-semibold text-slate-400">
      у самого корня. Делаем 1 вращение.
    </span>
  </p>
  <p>
    <span className="font-semibold text-slate-400">
      оба левые или оба правые. Вращаем деда, по
    </span>
  </p>
  <p>
    <span className="font-semibold text-slate-400">
      колено. Вращаем узел в разные стороны.
    </span>
  </p>
</div>
</div>

```

```

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
  <h4 className="text-indigo-400 font-bold text-slate-400">

```

```

<code className="text-rose-400">[60]</code>
углы), дерево будет превращаться в палку
сторону. Первая операция Splay(60) стои
<strong className="text-white">O(n)</strong>
глубину n. Вторая Splay(10) заставляет
Постоянный свинг убивает производительн
сложности <strong className="text-white"
</p>
</div>
<div className="mt-3 text-[10px] text-rose-
  Кэшировать свингующие пары на концах – фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
  <div>
    <p className="font-bold text-slate-300 mb
      3. Амортизация O(log n)
    </p>
    <p className="text-slate-400 leading-rela
      Парадокс: один запрос за{" "}
      <code className="text-emerald-400">O(n)</code>
      всю структуру вдвое более плоской за сч
      при двойных вращениях {
        <code className="text-emerald-400 font-
      }. Дорогая операция инвестирует в дешев
      операций. Поэтому амортизированное врем
      <strong className="text-white">O(log n)</strong>
    </p>
  </div>
</div>
<div className="mt-3 text-[10px] text-emera
  Каждое "растяжение" компенсируется "склад
</div>
</div>
</div>
</div>
</div>
);

```

```

      tate: 'idle' },
    });

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут роняться');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.color} мылом`);

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
  }, 500);
}, [dirtyStack, isWashing]);

```

```

    setDirtyStack([
      { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
      { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
      { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
    ]);
    setCleanRack([]);
    setIsWashing(false);
    setShowSoap(false);
    setLog([' Стол сброшен. Посуда снова грязная!']);
  });

```

```

const topIndex = dirtyStack.length - 1;

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950/40 p-6">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-tight">
          Ты складываешь грязные тарелки друг на друга.
          А когда приходит время мыть, ты берёшь именно последнюю.
          Попробуй вытащить нижнюю – всё рухнет!
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex items-center gap-3 flex-wrap">
      <button
        onClick={pushPlate}
        disabled={isWashing}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
          disabled:cursor-not-allowed text-white"
      >
        Добавить тарелку
      </button>
    </div>
  </div>
)

```

```

        <span>РАКОВИНА    </span>
    </div>

    {/* Мыльные пузыри при мытье */}
    {showSoap && {
        <div className="absolute inset-0 pointer-events-none" >
            <div className="absolute w-6 h-6 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute w-4 h-4 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute w-5 h-5 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute text-4xl anim-crash text-white" >
            </div>
        </div>
    }}

    {dirtyStack.length === 0 ? {
        <div className="flex-1 flex flex-col items-center justify-center" >
            <span className="text-6xl mb-3 animate-pulse text-white" >
            <p className="text-white font-black text-2xl" >
            <p className="text-slate-500 text-xs mt-1" >
            </div>
        } : {
            <div className={`flex-1 flex flex-col justify-center items-center`} >

                {/* Указатель TOP */}
                <div
                    className="absolute right-8 flex items-center justify-center"
                    style={{ bottom: `${24 + topIndex * 44}px` }} >
                    <span className="text-xs font-mono text-white" >
                    er gap-1">
                        <ArrowUp className="w-3.5 h-3.5" />
                        <span>ВЕРШИНА (TOP)</span>
                    </span>
                    <span className="text-blue-400 animate-pulse text-2xl" >
                    </div>

```



```

<div className="absolute top-4 right-4 flex items-center justify-center"
  order-emerald-800/80 text-[9px] font-mono">
  <Sparkles className="w-3.5 h-3.5 animate-pulse" />
  <span>ЧИСТО </span>
</div>

<div className="flex-1 flex flex-col justify-between"
  {cleanRack.length === 0 ? {
    <div className="flex-1 flex flex-col item-center justify-center"
      <span className="text-5xl mb-2"> </span>
      <p className="text-xs font-bold font-mono">Пусто!
      <p className="text-[10px] mt-1">Здесь будет сушилка
    </div>
  ) : {
    <div className="flex flex-col gap-2 w-full"
      {cleanRack.map(plate => {
        <div
          key={plate.id}
          className="w-full h-8 rounded-full border border-slate-200"
          items-center justify-center gap-2 opacity-0.8">
          <span className="text-sm"> {plate.name}
          <span className="text-[10px] text-emerald-500"> {plate.status}
        </div>
      })}
    </div>
  }}
</div>

{/* Столешница сушки */}
<div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-200" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-700 p-4"
  <div className="text-[10px] font-mono text-slate-200">

```

```

        while (j > 0 && s[i] !== s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                'Сходство уменьшается. Откат назад: откат j = pi[j - 1] = ${pi[j - 1]};
                j = pi[j - 1];
            });
        }

        if (s[i] === s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                'Сходство увеличивается.' });
            j++;
        } else {
            steps.push({ i, j, pi: [...pi], highlight:
                'Схождений нет.' });
        }

        pi[i] = j;
        steps.push({ i, j, pi: [...pi], desc: `Записываем ${j} в pi[${i}].` });

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true });
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string) {
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало. l=${l}, r=${r}, z=${z}` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=${i}, l=${l}, r=${r}, z=${z}` });
    }
}

```

```
-----  
export function StringAlgorithmsViz({ defaultMode = "kmp"  
  const [mode, setMode] = useState<"kmp" | "z">(defaultMode);  
  const [pattern, setPattern] = useState("aba");  
  const [text, setText] = useState("abacaba");  
  
  // Ensure inputs are valid dynamically  
  const safePattern = pattern || " ";  
  const safeText = text || " ";  
  
  const { s, steps, patternLength } = useMemo(() => {  
    if (mode === "kmp") return generateKmpSteps(safePattern, safeText);  
    else return generateZSteps(safePattern, safeText);  
  }, [mode, safePattern, safeText]);  
  
  const [autoPlay, setAutoPlay] = useState(false);  
  const [stepIdx, setStepIdx] = useState(0);  
  useVizStepSync(stepIdx, setStepIdx, steps.length - 1);  
  const timer = useRef<NodeJS.Timeout | null>(null);  
  
  // Reset when inputs or mode change  
  useEffect(() => {  
    setStepIdx(0);  
    setAutoPlay(false);  
  }, [s, mode]);  
  
  // Timer loop for autoPlay  
  useEffect(() => {  
    if (autoPlay) {  
      timer.current = setInterval(() => {  
        setStepIdx(prev => {  
          if (prev >= steps.length - 1) { setAutoPlay(false);  
            return prev + 1;  
          }  
        });  
      }, 1200);  
    }  
    return () => { if (timer.current) clearInterval(timer.current); };  
  }, [autoPlay, steps.length]);
```

```

px] flex items-center justify-start">
  <div className="flex gap-1.5 px-4 h-full"
    {s.split("").map((char, index) => {
      const isPattern = index < pattern.length
      const isSeparator = index === pattern.length

      let borderColor = "border-slate-400"
      let bgColor = "bg-slate-900"
      let textColor = isSeparator ? "text-white" : "text-slate-400"

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.include(index)) {
          borderColor = "border-amber-400"
          bgColor = step.match === index ? "bg-amber-400" : "bg-slate-900/40"
        }
      }

      // Z Logic
      if (mode === "z") {
        if (index >= step.l && index <= step.r) {
          bgColor = "bg-emerald-900"
          borderColor = "border-emerald-400"
        }
        if (step.zTarget === index) {
          borderColor = "border-amber-400"
          bgColor = step.match === index ? "bg-amber-400" : "bg-slate-900/40"
        }
      }
    })}

  return (
    <div key={index} className={
      /* L/R markers for Z */
      {mode === "z" && step.l <= index <= step.r
        ? "border border-amber-400"
        : ""}
    >
      <div className="absolute inset-0" style={{
        background: bgColor,
        border: borderColor,
        color: textColor,
        display: flex,
        align-items: center,
        justify-content: center,
        width: 100px,
        height: 100px,
        text-align: center,
        font-size: 1.2em,
        font-weight: bold,
        line-height: 1.2,
        padding: 5px 10px,
        margin: 0 auto,
        box-shadow: 0 0 0 2px transparent,
        transition: border-color 0.2s,
        background-color: 0.2s,
        color: 0.2s,
        text-decoration: none,
        pointer-events: none,
        user-select: none,
        cursor: default,
        opacity: 0.9,
        z-index: 1,
      }}>
        {char}
      </div>
    </div>
  )
}

```

```

        { /* Z fingers */
        {mode === "z" && step.i
          <div className="abs
e z-20">
            <span className
            </div>
        }}
        {mode === "z" && step.z
          <div className="abs
            <span className
            </div>
        }}
        {mode === "z" && step.z
          <div className="abs
            <span className
            </div>
        }}

        { /* Found flash effect
        {step.found && mode ===
          <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
        }}
        {step.found && mode ===
          <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
        }}
        </div>
      )
    }
  }
}
</div>
</div>

<div className="flex flex-col sm:flex-row g
  { /* Controls */
  <div className="flex bg-slate-900 borde
    <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov

```

Пройди шаги вручную, посмотри, как
ля пропуска работы (в Z).

</p>

</div>

{/* Code Sneak Peek */}

<div className="bg-slate-900 border border-

<div className="bg-slate-800 px-4 py-2 b

late-400 uppercase">

Код C++ ({mode === 'kmp' ? 'КМП

</div>

<pre className="p-4 text-xs font-mono te

{mode === 'kmp' ? `vector<int> pi(s.length());

for (int i = 1; i < s.length(); i++) {

int j = pi[i-1];

while (j > 0 && s[i] != s[j]) j = pi[j-1];

if (s[i] == s[j]) j++;

pi[i] = j;

}` : `int l = 0, r = 0;

for (int i = 1; i < n; i++) {

if (i <= r) z[i] = min(r - i + 1, z[i - l]);

while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]

if (i + z[i] - 1 > r) {

l = i;

r = i + z[i] - 1;

}

}`}

</pre>

</div>

</div>

);

}

```
      >
        {content}
      <span
        className={`absolute left-1/2 h-2 w-2 -translate-x-50%
          side === "top" ? "-bottom-[5px] border-b border-black" : ""
        }`
      />
    </span>
  </span>
);
};
```

ФАЙЛ 71/87: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";
```

```
interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}
```

```
interface TEdge {
  u: number;
  v: number;
```

```

const dfsStack: number[] = [];

const recordStep = (desc: string, currentNode: number) => {
  listSteps.push({
    desc,
    visited: new Set(visited),
    fullyProcessed: new Set(fullyProcessed),
    currentNode,
    topoList: [...topoList],
    dfsStack: [...dfsStack],
  });
};

recordStep(
  "Начало топологической сортировки. Используем кла",
  null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} ("${dagNodes[u].name}`,
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} о`,
        u,
      );
    }
  }
  fullyProcessed.add(u);
  topoList.push(u);
};

dfs(0);

return topoList;
}

export default topologicalSort;

```



```

    topoList: [],
    dfsStack: [],
  };

  useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1) {
      timer = setInterval(() => {
        setCurrentStepIdx((prev) => prev + 1);
      }, 1600);
    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIdx(0);
  };

  const getTopoListString = () => {
    return [...step.topoList]
      .reverse()
      .map((id) => dagNodes[id].name)
      .join(" → ");
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Clock className="w-5 h-5 text-indigo-400" />
          Топологическая сортировка (Зависимости обучен
        </h3>

        <div className="flex items-center gap-2 bg-slate-

```

```

        </marker>
        <marker
          id="dag-arrow-active"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#808080" />
        </marker>
      </defs>

      {/* Edges */}
      {dagEdges.map(({edge, idx}) => {
        const u = dagNodes.find((n) => n.id === edge.u)
        const v = dagNodes.find((n) => n.id === edge.v)

        const isActive =
          (step.currentNode === edge.u || step.currentNode === edge.v) ||
          step.visited.has(edge.v);

        return (
          <line
            key={`edge-${idx}`}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isActive ? "#818cf8" : "#334155"}
            strokeWidth={isActive ? "3" : "2"}
            markerEnd={`url(#${isActive ? "dag-arrow-active" : "dag-arrow"})`}
            className="transition-all duration-300"
          />
        );
      })}
    )}
  )}
}

```

```

        <text
          x={node.x}
          y={node.y - 4}
          textAnchor="middle"
          fill="white"
          fontSize="11px"
          fontWeight="bold"
          className="pointer-events-none"
        >
          ({node.label}) {node.name.split(" "
        </text>
        <text
          x={node.x}
          y={node.y + 12}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="9px"
          className="pointer-events-none"
        >
          {node.name.split(" ").slice(1).join
        </text>
      </g>
    );
  }}}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>
  </div>

```

```

    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-700 text-white"
        >
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-700 text-white"
        >
          Вперед
        </button>
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
```

```

    <div className="space-y-4">
      <SplayRotationsViz />
      <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
        <span aria-hidden="true"> </span>
        <span>
          Разобрался – проверь себя: ниже то же самое дерево ты
          не решишь.
        </span>
      </p>
      <SplayRotationSandbox />
    </div>
  );

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, что нужно для",
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">

```

```

    },
  },
  "splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Сверху – покадровый разбор поворота, снизу –",
    Component: () => <SplayRotationsPair />,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: () => <GraphDfsBfs /> },
  "top-sort": { title: "Топологическая сортировка", Component: () => <TopSort /> },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: () => <SccKosaraju /> },
  "graph-articulation": { title: "Точки сочленения", Component: () => <GraphArticulation /> },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: () => <BridgesCode /> },
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: () => <EulerPathVsCycle /> },
  "planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: () => <PlanarityEulerFormula /> },
  dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем",
    Component: () => (
      <>
      <ChapterImage vizType="dijkstra" />
      <DijkstraViz />
      </>
    ),
  },
  "bellman-ford": {
    title: "Форд–Беллман",
    Component: () => (
      <>
      <ChapterImage vizType="bellman-ford" />
      <BellmanFordViz />
      </>
    ),
  },
  floyd: {
    title: "Флойд–Уоршелл",
    Component: () => (
      <>
      <ChapterImage vizType="floyd" />

```



```

    setIsSearchDone(false);
    setSearchLog('Симуляция сброшена. Лосось вернулся на');
    setJumpPath(null);
    setFoundMatches([]);
    setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
    setActiveSearchCodeLine(4);
    return;
  }

  const char = textToScan[currentIndex];
  let curr = currentNode;
  let logMsg = `[Символ '${char}']`;

  let jumpType: 'normal' | 'fail' | null = null;
  void jumpType;
  let originalCurr = curr;

  if (TRIE_TRANSITIONS[curr][char] !== undefined) {
    const nextNode = TRIE_TRANSITIONS[curr][char];
    logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
    setJumpPath({ from: curr, to: nextNode, type: 'normal' });
    curr = nextNode;
    setActiveSearchCodeLine(3);
  } else {
    setActiveSearchCodeLine(2);
    let jumps = 0;
    while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
      curr = WATERFALL_NODES[curr].fail;
      jumps++;
    }
    const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
  }
};

```



```
// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
  {
    targetNodeId: 0,
    title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
    desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
      рние вершины сразу инициализируются с fail = R0
    codeLine: 1,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'ø',
    checkingBranchesFrom: null,
  },
  {
    targetNodeId: 1,
    title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
    desc: 'Мы обрабатываем первую вершину из очереди -
      просто не существует. fail[H] автоматически нап
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: null,
  },
  {
    targetNodeId: 7,
    title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
    desc: 'Обрабатываем дочернюю вершину #7 (S) на De
      ,
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: null,
  },
  {
    targetNodeId: 2,
```

```

        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'R',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 6,
        title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
        desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
            но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 9,
        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
        desc: ' МАГИЯ АХО-КОРАСИК! Обработываем #9 (SHE)
            ба плывет в #1 (H) и ищет ветку "E". У вершины
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
    },
    {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
            #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },

```

```

        activeMode === 'search'
          ? 'bg-blue-600 text-white shadow-lg sha
          : 'text-slate-400 hover:text-white'
        }` }
      >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
  /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
  <div className="grid grid-cols-1 lg:grid-cols-3
    {/* Управление шагами обучения */}
    <div className="bg-slate-900/90 p-5 rounded-x
      <div>
        <h5 className="text-indigo-300 font-bold
          <span> </span> Полный обход в ширину (В
        </h5>
        <p className="text-xs text-slate-300 mb-3
          Показаны все шаги очереди BFS (начиная
          ар 8 (SHE → HE)</strong>, где наглядно виде
        </p>
        <div className="space-y-1.5 mb-6 overflow
          {TRAINING_STEPS_INFO.map((step, idx) =>
            <button
              key={idx}
              onClick={() => setTrainStep(idx)}
              className={`w-full text-left px-3 p
                ${
                  idx === trainStep
                    ? 'bg-indigo-600 text-white fon
                    : 'bg-slate-800 text-slate-400
                }` }
            >
            <span className="line-clamp-1">{ste

```

```

    <div className="bg-slate-950 p-4 rounded-
      <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
        <span>1. let u = trie[r][char]; // Те
        {currentTrainInfo.codeLine === 1 && <
      тут</span>}
    </div>
    <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
        <span>2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}</span>
        {currentTrainInfo.codeLine === 2 && <
      к родителю</span>}
    </div>
    <div className={`p-1.5 rounded flex ite
l-4 border-amber-500 text-amber-300 font-bo
        <span>3. while (v !== root && trie[v]
        {currentTrainInfo.codeLine === 3 && <
      т, возврат в корень</span>}
    </div>
    <div className={`p-1.5 rounded flex ite
r-l-4 border-emerald-500 text-emerald-300 f
        <span>4. if (trie[v][char] !== undefin
      /span>
        {currentTrainInfo.codeLine === 4 && <
      ал IF!</span>}
    </div>
    <div className={`p-1.5 rounded flex ite
-l-4 border-blue-500 text-blue-300 font-bold'
        <span>5. else fail[u] = root; // Bero
        {currentTrainInfo.codeLine === 5 && <
      root</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-200 font-bold m

```

```

            idx === currentIndex
            ? 'bg-blue-600 text-white shadow'
            : idx < currentIndex
            ? 'bg-emerald-950/60 text-emera
            : 'bg-slate-800 text-slate-500'
        }` }
    >
    {char}
  </span>
)}}
{currentIndex >= textToScan.length && (
  <span className="bg-emerald-600 text-
    ГОТОВО ✓
  </span>
)}
</div>

<button
  onClick={stepSearchSimulation}
  disabled={isSearchDone || textToScan.len
  className={`w-full py-2.5 rounded-lg fo
    isSearchDone || textToScan.length ===
      ? 'bg-slate-700 text-slate-500 curs
      : 'bg-gradient-to-r from-blue-600 to
ue-900/40 active:scale-[0.98]`
    }` }
  >
    <span>{currentIndex === 0 ? 'Начать пла
    <ArrowRight className="w-4 h-4" />
  </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60
  <div>
    <h5 className="text-white font-bold mb-3
      <span>✂</span> Текущее действие в коде
    </h5>

```

```

    </div>
  })

  { /* SVG Анимация водопада */ }
  <div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
    -hidden shadow-2xl">
    { /* Водопадные фоновые эффекты */ }
    <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
    <div className="absolute top-0 left-1/2 -translate-x-1/2">
      <div className="w-8 bg-blue-400 h-full animate-pulse">
      <div className="w-12 bg-blue-300 h-full animate-pulse">
      <div className="w-6 bg-blue-500 h-full animate-pulse">
    </div>

    <div className="flex items-center justify-between">
      <h5 className="text-white font-bold text-base">
        <span> </span> Ветвящийся синий водопад (Борис)
      </h5>
      <div className="flex flex-wrap gap-4 text-xs">
        <span className="flex items-center gap-1 text-white">
          <span className="w-3 h-0.5 bg-blue-500 inline-block">
          </span>
          <span className="flex items-center gap-1 text-white">
            <span className="w-3 h-0.5 border-t border-blue-500 inline-block">
            </span>
          </span>
        </div>
      </div>

    { /* Само SVG дерево водопада */ }
    <div className="w-full overflow-x-auto custom-scrollbar">
      <svg viewBox="0 0 800 520" className="w-full h-full">

        { /* Горизонтальные линии и подписи уровней */ }
        {[
          { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
          { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
          { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
        ]}
      </svg>
    </div>
  </div>

```

```

        { /* Линия течения воды */ }
        <line
            x1={fromNode.x}
            y1={fromNode.y}
            x2={toNode.x}
            y2={toNode.y}
            stroke={isHighlighted ? '#3b82f6'
            strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
            className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
        />
        { /* Символ перехода */ }
        <circle cx={({fromNode.x + toNode.x} / 2)
            cy={({fromNode.y + toNode.y} / 2)
            r={10}
            fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'none'}
            stroke={isHighlighted ? '#3b82f6' : 'none'}
            strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
        />
        <text
            x={({fromNode.x + toNode.x} / 2)
            y={({fromNode.y + toNode.y} / 2) + 15
            fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'none'}
            fontSize="12"
            fontWeight="bold"
            textAnchor="middle"
        />
        {char}
        </text>

        { /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПЕРВЫХ ВЕТВЯХ */ }
        {isTrainingExamined && (
            <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
                (fromNode.y + toNode.y) / 2})`} >
                <rect x="-15" y="-12" width="30" height="24"
                fill={isMatchBranch ? '#10b981' : '#f43f5a'}
                stroke={isHighlighted ? '#3b82f6' : 'none'}
                strokeWidth="2" />
                <text x="0" y="5" fontSize="14"
                    {isMatchBranch ? ' ' : ''}
                />
            </g>
        )}
    </g>
}
);
});

```

```

        </text>
    })
  </g>
);
}}}
```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'target';
  const hasWords = node.words.length > 0;

  return (
    <g key={`node-${node.id}`} className="node" >
      {/* Внешнее свечение для активной вершины */}
      {isCurrent && (
        <circle cx={node.x} cy={node.y} r={node.radius * 1.5}
          fill="none" stroke="black" stroke-width={2} />
      )}
      {isTargetChild && (
        <circle cx={node.x} cy={node.y} r={node.radius * 1.5}
          fill="none" stroke="red" stroke-width={2} />
      )}

      {/* Основной круг вершины */}
      <circle
        cx={node.x}
        cy={node.y}
        r={isCurrent || isTargetChild ? '24px' : node.radius}
        fill={isCurrent ? '#2563eb' : isTargetChild ? 'red' : node.fill}
        stroke={isCurrent ? 'black' : isTargetChild ? 'red' : node.stroke}
        stroke-width={isCurrent || isTargetChild ? 2 : node.strokeWidth}
        className="transition-all duration-300" />

      {/* Текст внутри вершины */}
      <text
        x={node.x}
        y={node.y + 5}
        fill={isCurrent || isTargetChild ? 'white' : node.textColor}
        font-size={node.fontSize}
        font-weight={node.fontWeight}
        align="center" />
    </g>
  );
});

```



```

        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bezier(0.4, 0.6, 0.2, 0.8)'
        className="shadow-lg"
      />
      {/* Сама рыба */}
      <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezier(0.4, 0.6, 0.2, 0.8)'
        className="animate-float select-none"
      >
        {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
      </text>
    </g>

  </svg>
</div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
  <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{ width: '100%' }}>
    <h5 className="text-white font-bold mb-3 text-sm">
      <span> Улов лосося (Найденные слова) </span>
    </h5>
    {foundMatches.length === 0 ? (
      <p className="text-sm text-slate-400 italic">
        Пока ничего не поймали. Запустите шаги симуляции.
      </p>
    ) : (
      <div className="flex flex-wrap gap-2">
        {foundMatches.map((match, idx) => (
          <div
            key={idx}
            className="bg-emerald-950 border border-emerald-500 p-2 text-white text-sm"
          >
            {match}
          </div>
        ))}
      </div>
    )}
    </div>
  )}

```

```
*/

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase =
  "flex items-center justify-center rounded-md font-mono"
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);

  // Вкладка 2D = своё демо (свой скелет): компилятор по
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode === "2d"
  }, [chapterId, mode]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border"
      <div className="flex gap-2 mb-4 overflow-x-auto no-wrap"
        {(
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        ).map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              mode === key ? "bg-indigo-600 text-white"

```

```

<div className="space-y-5">
  <div className="overflow-x-auto pb-1">
    <div className="inline-block min-w-full">
      {/* индексы */}
      <div className="flex gap-1.5 mb-1 pl-[52px]">
        {A1.map(({_, i}) => {
          <div key={i} className={` ${cellBase} !h-5
            {i}
          </div>
        )}}
      </div>

      {/* массив a */}
      <div className="flex gap-1.5 items-center mb-1">
        <div className="w-[46px] shrink-0 text-right">
          {A1.map(({v, i}) => {
            const inCover = covered && i >= covered.f
            const inRange = i >= range.l && i <= range.r
            const isDrivenAdd = driven !== null && driven === i
            return (
              <div
                key={i}
                onMouseEnter={() => setHover({ kind: 'add' })}
                onMouseLeave={() => setHover(null)}
                onClick={() => setRange((r) => (i < r.l ? r.l : r.r))}
                className={` ${cellBase} cursor-pointer
                  isDrivenAdd
                    ? "bg-amber-500 text-black ring-2 ring-amber-500"
                    : inCover
                      ? "bg-indigo-500 text-white ring-2 ring-indigo-500"
                      : inRange
                        ? "bg-indigo-900/70 text-indigo-300"
                        : "bg-slate-800 text-slate-300"
                  `}
                title={`a[${i}] = ${v}`}
              >
                {v}
              </div>
            )
          )}
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
</div>

{ /* Пояснение под курсором */
<div className="min-h-[62px] bg-slate-900 border
  {driven !== null && !hover ? (
    <p className="text-slate-300">
      <b className="text-emerald-400">По шагам от.
      (последний:P[{driven}] = {pref[driven]}) –
      P[{Math.min(driven + 1, A1.length)}] = P[{d
      <span className="font-mono text-amber-400">
    </p>
  ) : hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0<
        отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hove
        </span>
      </p>
    )
  ) : hover?.kind === "a" ? (
    <p className="text-slate-300">
      <b className="text-indigo-400">
        a[{hover.i}] = {A1[hover.i]}
      </b>{" "}
      входит во все префиксы, начиная с{" "}
      <span className="font-mono text-emerald-300">
        подвинуть границу запроса.
      </p>
    )
  )
}

```

```

    }
    return p;
  }, [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() =>
                      setRect((r) =>
                        i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r
                    )
                  />
                );
              })}
            </div>
          })}
        </div>
      </div>
    </div>
  </div>
);

```

```

        isHover ? "bg-amber-500 text-wh
    }` }
    >
    {v}
    {corner && !isHover && {
        <span className="absolute -top-
ht">
        {corner}
        </span>
    }}
    </div>
    );
    }}}
    </div>
    )))
    </div>
    </div>
    </div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                P[{hover.i}][{hover.j}] = {P[hover.i][hov
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    } : {
        <p className="text-slate-500">Наведите курсор
    }}
    </div>

<div className="bg-slate-900 border border-indigo
    <p className="font-mono text-sm sm:text-base te
        сумма = <span className="text-emerald-400">A<
        <span className="text-rose-400">C</span> + <s
        <span className="text-indigo-300 font-bold">{
    </p>

```

```
// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {

```

```

        className={`px-3 sm:px-4 py-2 rounded-lg text-
          se-600 text-white" : "bg-slate-800 text-sla
    >
    3. Запрос 2D (Формула)
  </button>
</div>

  <AnimatePresence mode="wait">
    {tab === "1d" ? (
      <Viz1D key="1d" />
    ) : tab === "2d_build" ? (
      <Viz2DBuild key="2d_build" />
    ) : (
      <Viz2DQuery key="2d_query" />
    )}
  </AnimatePresence>
</div>
  );
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: num

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  useVizStepSync(step, setStep, maxStep);
  const covered = hover ? { from: hover.i, to: hover.i

  return (
    <div className="flex flex-col gap-5">

```



```

        return (
          <div key={idx} className="flex flex-col i
            <div
              className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
              inCover ? "bg-indigo-500 text-white
              }` }
            >
              {v}
            </div>
            <span className="text-[9px] text-slate-
          </div>
        );
      }}}
    </div>
    <div className="mt-2 text-xs min-h-[36px]">
      {hover && covered ? (
        <p className="text-slate-300">
          <b className="text-sky-400">
            ST[{hover.i}][{hover.j}] = {stID[hover.i
          </b>{" "}
          – это минимум на отрезке{" "}
          <span className="font-mono text-sky-300">
            [{covered.from}, {covered.to}]
          </span>{" "}
          длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{" "}
          <span className="font-mono text-slate-400">
            min({arrID.slice(covered.from, covered.
          </span>
        </p>
      ) : (
        <p className="text-slate-500">
          Наведите курсор (или тапните) на любое чи
        </p>
      )}
    </div>
  </div>

```

```

// Active computing state
let isActive = false;
let isSrc1 = false;
let isSrc2 = false;

if (step > 0 && step <= maxStep) {
  const currentStep = computeSteps[step];
  if (currentStep.i === i && currentStep.j === j) {
    isActive = true;
    if (currentStep.j === j + 1 && currentStep.i === i) {
      isSrc1 = true;
    }
    if (
      currentStep.j === j + 1 &&
      currentStep.i === i - (1 << (currentStep.i - i))
    ) {
      isSrc2 = true; // wait, no. Src2 is the source of the next step
    }
    // Let's re-eval exact source:
    // We are asking if THIS cell [i][j] is the source of the next step
    if (currentStep.j - 1 === j) {
      if (currentStep.i === i) isSrc1 = true;
      if (currentStep.i + (1 << (currentStep.i - i)) === i) {
        isSrc2 = true;
      }
    }
  }
}

return (
  <div key={j} className="relative flex" >
    {!isValid ? (
      <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" >
    ) : (
      <motion.div
        onMouseEnter={() => isVisible = true}
        onMouseLeave={() => setHover(false)}
        onClick={() => isVisible && setHover(true)}
        initial={{ opacity: 0, scale: 0.8 }}
        animate={{
          opacity: isVisible ? 1 : 0,
          scale: isVisible ? 1 : 0.8,
        }}
      >
    )}
  </div>
)

```

```

        strokeWidth="3"
      />
    </motion.svg>
  )}
</div>
);
}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-l
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1
            <span className="text-amber-400">
              {st1D[c.i + half][c.j - 1]}
            </span>
          ) ={" "}

```



```

        return (
          <div
            key={idx}
            onMouseEnter={() => { if(is
            onMouseLeave={() => { if(is
            onClick={() => {
              if(isCurr) {
                if(locked && locked.
                else setLocked({r, c
              }
            }}
            className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
          >
            {ST2D[r][c][step][step]}
          </div>
        );
      }}}
    </div>
  </div>
</div>
</React.Fragment>
)
  }}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-
  <div className="bg-slate-900 p-4 sm:p-6 rounded
    <h3 className="text-lg font-bold text-slate-3
      {k === 0 ? (
        <div className="text-slate-400 text-sm lea
          <p className="mb-4">При <span className=
            ы имеют размер <span className="font-bold t
              <div className="bg-[#0a0fle] rounded-xl l
                <span className="text-slate-500">// Ба
                <span className="text-indigo-400">ST</

```

```

    }}[k]][k]]</span>
    <span className="text-slate-400">
  </p>

  <div className="space-y-2 pt-3 pl-2" >
    {(() => {
      const prevL = 1 << (k - 1);
      return (
        <>
          <div className="flex items-center" >
            <span className="flex items-center justify-center shadow-[0_0_8px_#f43f5e]"></div> ST[{activeCell.r}
              <span className="text-slate-400" >{activeCell.c}[k-1][k-1]}</span>
            </div>
          <div className="flex items-center" >
            <span className="flex items-center justify-center shadow-[0_0_8px_#3b82f6]"></div> ST[{activeCell.r}
              <span className="text-slate-400" >{activeCell.c + prevL}[k-1][k-1]}</span>
            </div>
          <div className="flex items-center" >
            <span className="flex items-center justify-center shadow-[0_0_8px_#10b981]"></div> ST[{activeCell.r}
              <span className="text-slate-400" >{ST2D[activeCell.r + prevL][activeCell.c][k-1][k-1]}</span>
            </div>
          <div className="flex items-center" >
            <span className="flex items-center justify-center shadow-[0_0_8px_#f59e0b]"></div> ST[{activeCell.r}
              <span className="text-slate-400" >{activeCell.c + prevL}[activeCell.c + prevL][k-1][k-1]}</span>
            </div>
        </>
      )
    })()}
  </div>

```

```

<div className="flex flex-col xl:flex-row gap-6"
  <div className="flex-1 min-w-0 bg-slate-900"
    <h3 className="text-xl font-bold text-rose-500"
      <p className="text-slate-400 text-[13px] font-normal"
        ерх, r2, c2 - правый низ).</p>

```

```

    <div className="flex flex-col items-center"
      <div className="grid grid-cols-[repeat(8, 1fr)] gap-2"
        <div className="relative shadow-inner w-max max-w-full"
          {Array.from({length: 64}).map((_, idx) => {
            const r = Math.floor(idx / 8);
            const c = idx % 8;
            const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

            return (
              <div key={idx} className={`text-[clamp(9px,2.4vw,12px)] font-normal rounded text-slate-400 border-rose-500/50 border scale-105 z-10`
                {isInQuery ? `text-rose-500` : `text-slate-400`}
                {val2D[r][c]}
              </div>
            )
          })}
        </div>

```

```

    <div
      className="absolute border-[3px solid transparent] border-da
      style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: `calc(100% - ${8 * 2}px)`,
        width: `calc(100% - ${8 * 2}px)`,
      }}
    </div>

```

```

  {/* Показываем 4 угла в самой матрице */}
  <div className="absolute pointer-events-none"

```

```

    <h3 className="text-xl font-bold text-emerald-500">
    <p className="text-sm font-sans text-slate-500">
        Мы знаем, что максимальная степень дв
rounded text-white">{Lx}</span>. Аналогично
te">{Ly}</span>.
    </p>
    <p className="text-sm font-sans text-slate-500">
        Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0f1e] px-1 rounded b
роса. (Цветные прямоугольники слева).
    </p>

    <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto">
        <div className="absolute right-3 top-3">
            <span className="text-slate-500">// 1.
            <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
            <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
            <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
            <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b>

            <span className="text-slate-500">// 2.
            <span className="text-indigo-400">int<
            &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
            &nbsp;&nbsp;&nbsp;<span className="text-rose-900">r1</span>][<span className="text-white bg-r

            &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b>
            &nbsp;&nbsp;&nbsp;<span className="text-blue-500">r1</span>][<span className="text-white bg-b
            &nbsp;&nbsp;&nbsp;<span className="text-slate-500">

```



```

        mber-300 border-2"; badge = "BR"; }

        return (
            <div key={idx} className={
tion-all duration-300 relative shrink-0 ${c
                {txt}
                {badge && <div classNam
r-slate-700 text-white px-1 font-bold round
                </div>
            )
        }}}}
    </div>
  </div>
</div>

  <div className="mt-auto bg-slate-950 fon
)]">
    <div className="text-center font-bold
wrap bg-[#0a0fle] py-3 px-2 rounded-lg bord
    <span className="text-slate-400 fon
      <span className="text-rose-400">
      <span className="text-blue-400 m
n>
      <span className="text-emerald-40
span>
      <span className="text-amber-400 r
      &nbsp;<span className="text-slate-4
      <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
    </div>
  </div>

  </div>
</div>
)
}

```

```

        window.scrollTo(0, 0);
    } else {
        document.body.classList.remove('wtf-mode');
        if (aside) aside.style.display = '';
        if (headerMain) headerMain.style.display = '';
        if (questionsContainer) questionsContainer.style.display = '';
        if (wtfContainer) wtfContainer.classList.remove('wtf-mode');
    }
}

function setTheme(theme) {
    const body = document.body;
    body.classList.remove('theme-dark', 'theme-light', 'theme-terminal');
    body.classList.add('theme-' + theme);

    // Just some visual feedback on buttons
    document.querySelectorAll('[id^="theme-"]').forEach(btn => {
        btn.classList.remove('outline');
    });

    if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('bg-dark text-white transition-colors');
    if (theme === 'light') document.getElementById('btn-theme-light').classList.add('bg-light text-dark transition-colors');
    if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('bg-terminal text-white transition-colors');

    document.getElementById('btn-theme-dark').classList.add('px-3 py-1 rounded');
    document.getElementById('btn-theme-light').classList.add('px-3 py-1 rounded');
    document.getElementById('btn-theme-terminal').classList.add('px-3 py-1 rounded');
    document.getElementById('btn-theme-notebook').classList.add('px-3 py-1 rounded');
    document.getElementById('btn-theme-terminal').classList.add('px-3 py-1 rounded');
}

// Initialize theme and mode
setTheme('dark');
```

```

// Close on link click
document.querySelectorAll("#mobile-drawer a")
  .forEach(link => {
    link.addEventListener("click", () => {
      if (mobileDrawer && !mobileDrawer.closed) {
        mobileDrawer.toggleDrawer();
      }
    });
  });

const observerOptions = {
  root: null,
  rootMargin: '-10% 0px -70% 0px',
  threshold: 0
};

const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting && !document.querySelector("#mobile-drawer .active")) {
      // Highlight active TOC link locally
      const id = entry.target.id;
      document.querySelectorAll('aside .toc-link')
        .forEach(link => {
          if (link.getAttribute('href') === `#${id}`) {
            link.classList.add('bg-active');
          } else {
            link.classList.remove('bg-active');
          }
        });
    }
  });
});

document.querySelectorAll('section[id^="questions"]').forEach(section => {
  observer.observe(section);
});

</script>
</body>
</html>

```

```
        text-shadow: 1px 1px 0 #000, -1px -1px 0 #000;
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
    .uno-card.mini { width: 45px !important; height: 45px !important; }
    .uno-card.mini::before, .uno-card.mini::after { display: none; }

    /* Global formula scroll settings */
    .formula-scroll {
        overflow-x: auto;
        overflow-y: hidden;
        -webkit-overflow-scrolling: touch;
        scrollbar-width: none; /* Firefox */
        -ms-overflow-style: none; /* IE and Edge */
    }
    .formula-scroll::-webkit-scrollbar {
        display: none;
    }

    mjx-container {
        max-width: 100% !important;
    }
    mjx-container[display="true"] {
        overflow-x: auto;
        overflow-y: hidden;
        max-width: 100%;
        scrollbar-width: none;
        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar {
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font-size: 10px; }
    }
}
```

```

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light .bg-slate-800 { background-color: #0f172a !important; }
body.theme-light .bg-slate-700, body.theme-light .bg-slate-600 { background-color: #1e293b !important; }
body.theme-light .text-slate-200, body.theme-light .text-slate-300 { color: #c6c8ca !important; }
body.theme-light .text-slate-400 { color: #94989b !important; }
body.theme-light .border-slate-600, body.theme-light .border-slate-500 { border-color: #475569 !important; }
body.theme-light #top-controls { background-color: #f1f3f4; }

body.theme-notebook {
    background-color: #fcf8eb !important; /* light beige */
    color: #3b2818 !important; /* brown ink */
    background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-800, body.theme-notebook .bg-slate-700 {
    background-color: rgba(255, 255, 255, 0.7);
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-300 { color: #c6c8ca !important; }
body.theme-notebook .text-slate-400 { color: #94989b !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

```

```

<div class="bg-slate-800 p-1 rounded-lg flex
  <button onclick="setTheme('dark')" id="
transition-colors">Темная</button>
  <button onclick="setTheme('light')" id=
ext-white hover:bg-slate-700 transition-col
  <button onclick="setTheme('notebook')"
over:text-white hover:bg-slate-700 transiti
</div>
</div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
  <defs>
    <marker id="arrow-yellow" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
    </marker>
    <marker id="arrow-orange" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#f9731c"
    </marker>
    <marker id="arrow-pink" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#f472b2"
    </marker>
    <marker id="arrow-cyan" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
    </marker>
    <marker id="arrow-indigo" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
    </marker>
    <marker id="arrow-lime" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
    </marker>
    <marker id="arrow-green" markerWidth="6" ma
      <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">

```

```

        </g>
      </defs>
    </svg>

    <!-- Мобильная кнопка меню (Гамбургер) -->
    <button id="mobile-menu-btn" class="lg:hidden fixed
      0/50 z-50 hover:bg-blue-500 transition-transform
      <svg xmlns="http://www.w3.org/2000/svg" class="
        <path stroke-linecap="round" stroke-linejoin
      </svg>
    </button>

    <!-- Затемнение фона на мобилках при открытом меню
    <div id="drawer-overlay" class="lg:hidden fixed inset-0
      bg-black opacity-50 transition-opacity duration-300
    </div>

    <div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col
      <!-- Левое меню (Оглавление) -->
      <aside id="mobile-drawer" class="fixed inset-y-0 left-0
        translate-x-full transition-transform duration-300
        w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
        <div class="h-full flex flex-col p-6 overflow-y-auto
          lg:rounded-2xl lg:border lg:border-slate-700
          <div class="flex justify-between items-center
            <h3 class="font-black text-transparent bg-clip-text
              ider text-sm">
                Содержание
              </h3>
            <button id="close-drawer-btn" class="
              <svg xmlns="http://www.w3.org/2000/svg" class="
                <path stroke-linecap="round" stroke-linejoin="round"
              </svg>
            </button>
          </div>
        <nav class="space-y-3 text-sm font-medium
          <details class="group" open>
            <summary class="list-none cursor-pointer">

```

```

        <a href="#section-3-4" class="b
</div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-4" class:
er-amber-500 pl-3 font-bold text-amber-300">
        4. Степени и корни
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
    <a href="#q4-def" class="block l
    <a href="#q4-moivre" class="blo
    <a href="#q4-root" class="block
    <a href="#q4-count" class="bloc
</div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-:
        5. Группа корней из единицы
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
    <a href="#q5-def" class="block l
    <a href="#q5-group" class="bloc
    <a href="#q5-prim" class="block
    <a href="#q5-crit" class="block
    <a href="#q5-cyclic" class="blo
</div>
</details>

<details class="group">
    <summary class="list-none curso
```



```

        <div class="pl-6 space-y-2 text
          <a href="#q8-1" class="bloc
          <a href="#q8-2" class="bloc
          <a href="#q8-3" class="bloc
          <a href="#q8-4" class="bloc
          <a href="#q8-5" class="bloc
          <a href="#q8-6" class="bloc
          <a href="#q8-7" class="bloc
        </div>
      </details>

      <details class="group">
        <summary class="list-none curso
          <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
            9. Кольцо многочленов
          </a>
        </summary>
        <div class="pl-6 space-y-2 text
          <a href="#q9-1" class="bloc
          <a href="#q9-2" class="bloc
          <a href="#q9-3" class="bloc
          <a href="#q9-4" class="bloc
        </div>
      </details>

      <details class="group">
        <summary class="list-none curso
          <a href="#question-10" clas
der-rose-500 pl-3 font-bold text-rose-400">
            10. Делимость и Схема Г
          </a>
        </summary>
        <div class="pl-6 space-y-2 text
          <a href="#q10-1" class="blo
          <a href="#q10-2" class="blo
          <a href="#q10-3" class="blo

```

```
        <details class="group">
          <summary class="list-none cursor-pointer-emerald-500 pl-3 font-bold text-emerald-500">
            13. Корни многочленов.
          </a>
          </summary>
          <div class="pl-6 space-y-2 text-emerald-500">
            <a href="#q13-1" class="block">
            <a href="#q13-2" class="block">
            <a href="#q13-3" class="block">
          </div>
        </details>
```

```
        <details class="group">
          <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
            14. Кратность корня. Ко
          </a>
          </summary>
          <div class="pl-6 mt-2 space-y-2 text-fuchsia-500">
            <a href="#q14-1" class="block">
            <a href="#q14-2" class="block">
            <a href="#q14-3" class="block">
            <a href="#q14-4" class="block">
          </div>
        </details>
```

```
        <details class="group">
          <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
            15. Производная. Критер
          </a>
          </summary>
```

```

        </a>
        <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
            <span class="font-bold text-ind
        </a>
        <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
            <span class="font-bold text-fuc
        </a>

        <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
        ▲ Ликбез: Алгоритмы выживания
        </a>

</nav>
<script>
    document.querySelectorAll('details.
        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
="q5-"], div[id^="q6-"], div[id^="q7-"], di
v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.grou
        link.addEventListener('clie

```

```

        rootMargin: '-10% 0px -70% 0px',
        threshold: [0, 0.5]
    });

    sections.forEach(section => {
        observer.observe(section);
    });
});

```

```

function fallbackModal(contentRaw, ...args) {
    const modal = document.createElement('div');
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';
    modal.style.height = '100vh';
    modal.style.backgroundColor = 'black';
    modal.style.zIndex = '99999';
    modal.style.display = 'flex';
    modal.style.flexDirection = 'column-reverse';
    modal.style.alignItems = 'center';
    modal.style.justifyContent = 'center';
}

```

```

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column-reverse';
box.style.color = '#fff';

```

```

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

```

```

const subHeader = document.createElement('div');

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = 'black';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(closeBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

function downloadMD() {
    let clone = document.querySelector('#md');

    // Smart markdown parsing based on the content
    // Smart markdown parsing based on the content
    clone.querySelectorAll('h2').forEach(el => {
        clone.querySelectorAll('h3').forEach(el => {
            clone.querySelectorAll('.analog').forEach(el => {
                clone.querySelectorAll('li').forEach(el => {
                    clone.querySelectorAll('svg').forEach(el => {
                        let caption = '';
                        if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
                            caption = el.nextElementSibling.alt;
                        }
                    });
                });
            });
        });
    });
}

```

```
        document.getElementById('theme')
    } else if (theme === 'light') {
        body.classList.add('bg-white');
        document.getElementById('theme')
        style.innerHTML = `

        body.bg-white section h1 { color: #334155; }
        body.bg-white main { color: #334155; }
        body.bg-white main p { color: #334155; }
        /* Universal text color */
        body.bg-white .text-slate { color: #334155; }
        body.bg-white .text-slate { color: #334155; }
        body.bg-white .text-slate { color: #334155; }
        body.bg-white [class*="text-slate"] { color: #334155; }

        /* Specific backgrounds */
        body.bg-white aside > div { background-color: #f1f3f4; }
        body.bg-white [class*="background-color"] { background-color: #f1f3f4; }

        body.bg-white [class*="background-color"] { background-color: #f1f3f4; }
        body.bg-white .analogy-block { background-color: #f1f3f4; }
        body.bg-white .img-placeholder { background-color: #f1f3f4; }
        body.bg-white .info-block { background-color: #f1f3f4; }

        body.bg-white .analogy-block { background-color: #f1f3f4; }
        or: #94a3b8 !important; }
        body.bg-white .img-caption { background-color: #f1f3f4; }

        /* Tweak card blocks */
        body.bg-white .card-red { background-color: #f1f3f4; }

        body.bg-white aside { background-color: #f1f3f4; }
        #334155 !important; }
        body.bg-white .text-white { color: #334155; }

        /* Ensure active links are visible */
```

```

        fill: transparent !important;
    }
    `;
}

document.head.appendChild(style);
}
</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
    <!-- Кнопки Конвертации / Экспорта
    <div class="flex flex-col gap-2">
        <div class="text-xs text-slate-
            <button onclick="downloadWord()
ont-bold flex items-center justify-center g
                <span> </span> Word
            </button>
            <a href="algebra.pdf" target="_l
-sm font-bold flex items-center justify-cen
                <span> </span> PDF
            </a>
            <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
                <span> </span> Markdown
            </button>
            <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
                <span> </span> HTML
            </button>
        </div>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-
        <div class="flex items-center g

```

```

<li><a href="#question-8">8. Кл
<li><a href="#question-9">9. Ко
<li><a href="#question-10">10.
<li><a href="#question-11">11.
<li><a href="#question-12">12.
<li><a href="#question-13">13.
<li><a href="#question-14">14.
<li><a href="#question-15">15.
<li><a href="#question-17">17.
<li><a href="#question-18">18.
<li><a href="#question-19">19.
<li><a href="#question-20">20.
<li><a href="#question-21">21.
<li><a href="#question-22">22.
    </ul>
  </div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4">
      <div class="grid grid-cols-3 gap-4">
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
      </a>

```



```

        </a>
        <!-- Empty 3rd row cell -->
        <div class="hidden md:block"></div>

        <!-- Row 4: Факторизация / Структур
        <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-11" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 5: Корни / Уравнения -->
        <div class="hidden md:block"></div>
        <a href="#question-13" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t

```

```

        </a>
        <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
        <div class="text-xs text-indigo
        <div class="font-bold text-slate
        </a>
        <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
        <div class="text-xs text-fuchsia
        <div class="font-bold text-slate
        </a>

        <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
        <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

        <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
        <div class="text-xs text-red-500
        <div class="font-bold text-slate
        </a>
    </div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
    <div class="flex items-center mb-6">
        <div class="bg-red-500/20 p-3 r
            <svg class="w-8 h-8 text-red
rg/2000/svg">
                <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
            </svg>
        </div>
        <h2 class="text-3xl pr-4 border
    </div>

    <div class="prose prose-invert max-w

```

```
<p class="text-sm text-slate-500">
  <ul class="list-disc pl-5 space-y-2">
    <li><span class="text-w
, r_2$).</li>
    <li><span class="text-w
    <li><span class="text-w
счет ограничений (размер остатка) докажи, что
  </ul>
</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6">
  <h3 class="text-xl text-yellow-400">
  <p class="text-sm text-slate-500">
  <ul class="list-disc pl-5 space-y-2">
    <li><span class="text-w
валось.</li>
    <li><span class="text-w
Просто перепиши формулу с буквой $k$.</li>
    <li><span class="text-w
усок $k$, замени его на жареного карася из
  </ul>
</div>

<!-- Двойная делимость -->
<div class="bg-slate-800/80 p-6">
  <h3 class="text-xl text-orange-400">
  <p class="text-sm text-slate-500">
  <ul class="list-disc pl-5 space-y-2">
    <li><span class="text-w
вверх по уравнениям).</li>
    <li><span class="text-w
рху вниз, показывая, что $\delta \vdots r_i$
    <li><span class="text-w
х. Твой $d$ больше.</li>
  </ul>
</div>
```

(Правила экстрасенса)</h2>

<p class="text-sm text-slate-400">отношения), а ты их не помнишь. Юзай базовые нейность) для делимости.</p>

<ul class="space-y-4 pb-10">

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan">

<p class="text-sm t

<code class="text-x

</div>

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan">

<p class="text-sm t

х двух, потом последних.

<code class="text-x

</div>

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan">

<p class="text-sm t

есть сложение – есть ноль. Если есть умноже

<code class="text-x

</div>

<li class="bg-slate-800/50 p-4">

<div>

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся –
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
  const parts: string[] = [];
  for (const sheet of Array.from(document.styleSheets))
    try {
      const rules = (sheet as CSSStyleSheet).cssRules;
      parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен – пропускаем */
      }
    }
  }
  return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
```

```

} catch {
  /* если браузер не осилил регулярку – выгружаем текст */
}

const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
  const id = el.dataset.termId;
  if (!id) return;
  if (!ids.includes(id)) ids.push(id);
  const link = document.createElement("a");
  link.className = "term-hint";
  link.href = `#term-${id}`;
  link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
  link.textContent = el.textContent ?? "";
  el.replaceWith(link);
});

// <details> в файле лучше раскрыть: иначе код и разбросан
host.querySelectorAll("details").forEach((d) => d.setOpen());

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("\n");
}

```

```

    <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
    const blob = new Blob([html], { type: "text/html;charset=utf-8" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
    const num = title.match(/^\s*(\d+)/)?[1];
    const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
    return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

```

```

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
    const css = await collectCss();
    const { body, termIds } = prepareBody(chapter.content);
    const html = wrap({
        title: chapter.title,
        subtitle: chapter.category || "Универсальное пособие",
        css,
        body,
        gloss: glossarySection(termIds),
        origin: window.location.origin,
    });
}

```

```
}  
  
=====
```

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

```
=====
```

```
-----  
ФАЙЛ 80/87: scripts/audit-coverage.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ: 1.5 КБ  
-----
```

```
/**  
 * Аудит покрытия глав пособия: у каких тем нет «объяснений»  
 * (блок «Аналогия») и/или 2D-визуализации (интерактивные)  
 *  
 * node scripts/audit-coverage.mjs          # отчёт  
 * node scripts/audit-coverage.mjs --md      # markdown  
 */  
import fs from "node:fs";  
import path from "node:path";  
import { fileURLToPath } from "node:url";  
import { build } from "esbuild";  
  
const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),  
  "src/data");  
const TMP = path.join(ROOT, "tmp", "audit");  
  
fs.mkdirSync(TMP, { recursive: true });  
const bundle = path.join(TMP, "content.bundle.mjs");  
  
await build({  
  entryPoints: [path.join(ROOT, "src/data/content.ts")],  
  bundle: true,  
  format: "esm",  
  platform: "node",  
  outfile: bundle,  
  logLevel: "error",  
});
```



```

    for (const r of rows) {
      console.log(
        `| ${r.num} || "-" | ${r.title} | ${r.hasAnalogy
          ge)} |`
      );
    }
  } else {
    console.log(`Глав в пособии: ${rows.length}\n`);
    const dump = (title, list) => {
      console.log(`${title} (${list.length}):`);
      list.forEach((r) => console.log(`  * ${r.title} [$
        console.log();
      });
    dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
    dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
    dump("НЕТ 2D (аналогия есть)", gapViz);
    console.log(`Пропущенные номера билетов 1-24: ${missi
    console.log(`Визуализаторы без главы: ${orphanViz.join
  }

```

ФАЙЛ 81/87: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 *   node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa

```

```
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 4),
  );
}
console.log("\nГлав без единой подсказки:", bad.length,);
console.log("Терминов в глоссарии:", GLOSSARY.length,);
console.log("Не встретились:", GLOSSARY.filter((g) => !g).length,);
```

ФАЙЛ 82/87: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, } from "src";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };
```

```
    });

    console.log("[3/3] png -> pdf");
    console.log(`    страниц: ${pages.length}`);
    console.log(`    выход:   ${path.relative(ROOT, PDF)}`);
  }

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

ФАЙЛ 83/87: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ: 10.5 КБ

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недоступен)
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, ensureFile } from "scripts/export/paths";

/** Расширения, которые считаем «кодом» и включаем в дайджест */
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml",
]);
```

```

    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
    .split("\n")
    .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
      const ia = ORDER.indexOf(categoryOf(a));
      const ib = ORDER.indexOf(categoryOf(b));
      return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

/** Достаём заголовки глав из данных пособия (без запуска) */
function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("src/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
  }
}

```

```

push(HR);
push("                                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")} . ${c.title
});
push();

```

```

push(HR);
push("                                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`);
});
push();
push();

```

```

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  const content = fs.readFileSync(path.join(ROOT, rel
  const lines = content.split("\n");
  push(SUBHR);
  push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);

```

```
-----
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в кэш) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_and_filenames.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_and_filenames.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
  pointsize: 8,
  /** Максимум символов в строке (дальше – жёсткий перенос) */
  cols: 138,
  /** Строк содержимого на странице (+2 служебные: шапка и подвал) */
  rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return bytes + " bytes";
  if (bytes < 1024 * 1024) return (bytes / 1024).toFixed(1) + " KB";
  if (bytes < 1024 * 1024 * 1024) return (bytes / (1024 * 1024)).toFixed(1) + " MB";
  return (bytes / (1024 * 1024 * 1024)).toFixed(1) + " GB";
}
```

```
-----
import { promisify } from "node:util";
import {
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, "    ").replace(
    if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = "    ".repeat(Math.min((expanded.match(/^
let rest = expanded;
let first = true;
while (rest.length > 0) {
  const width = first ? PAGE.cols : PAGE.cols - indent;
  parts.push((first ? "" : indent) + rest.slice(0, width));
  rest = rest.slice(width);
  first = false;
}
return parts;
}

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)
  }
}
```

```
async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}.
    }
  }
}

await Promise.all(Array.from({ length: concurrency },
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.p
if (missing.length) throw new Error(`Не отрендерены с

const bytes = jobs.reduce((s, j) => s + fs.statSync(j
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAG
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message)
  process.exit(1);
});
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 86/87: [.github/workflows/deploy-pages.yml](#)

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

```
  uses: actions/setup-node@v7
  with:
    node-version: 24
    cache: npm

- name: Install dependencies
  run: npm ci --no-audit --no-fund

- name: Build for the repository subpath
  env:
    VITE_BASE: /algv0/
  run: npm run build

- name: Make GitHub Pages fallback
  run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
  uses: actions/upload-pages-artifact@v5
  with:
    path: ./dist
```

deploy:

```
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
```

steps:

```
  # Ранняя и понятная ошибка вместо невнятного «Fail»
  # когда в репозитории Pages не переключён на сбор
  - name: Preflight – Pages собирается через GitHub
    env:
      GH_TOKEN: ${ secrets.GITHUB_TOKEN }
    run: |
      bt=$(gh api "repos/${ github.repository }/pages" |
        echo "Pages build_type: ${bt}")
      if [ "${bt}" != "workflow" ]; then
```

```
    density:
      description: "DPI растеризации страниц (по умолчанию)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает
          # разрешаем то, что нужно пайплайну (text -> pdf)
```

```
uses: actions/upload-artifact@v7
with:
  name: full-code-export
  path: |
    public/export/full_code_all_pages.pdf
    public/export/full_code_all_pages.txt
  if-no-files-found: error
  retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action
    git add -- public/export/full_code_all_pages.
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full
    git push origin "HEAD:${GITHUB_REF_NAME}"
```